

DeepSeek Harness 源码深度解析

目录

1 DeepSeek Harness (dsh) - 项目概览

1.1 项目基本信息

1.1.1 核心架构理念

1.2 项目目录结构

1.3 主要功能模块

1.3.1 核心 API 主干 (`packages/core/` , 8 个包)

1.3.2 LLM 能力族 (`packages/llm/` , 5 个包)

1.3.3 能力接缝族 (执行环境类)

1.3.4 能力接缝族 (模型协作类)

1.3.5 持久化与检索

1.3.6 协议与集成

1.3.7 应用装配与 Boot

1.3.8 应用入口 (`apps/`)

1.4 核心依赖分析

1.4.1 框架与核心

1.4.2 构建与工具链

1.4.3 测试与质量

1.4.4 原生与系统

1.4.5 持久化

1.5 构建与部署

1.5.1 开发环境

1.5.2 构建命令矩阵

1.5.3 运行方式

1.5.4 测试命令

1.5.5 质量门禁

1.5.6 部署形态

1.6 环境配置

1.6.1 必需环境变量

1.6.2 可选环境变量

1.6.3 配置文件

1.6.4 Profile 组合顺序

2 系统架构说明

2.1 架构概述

2.1.1 架构模式

2.1.2 设计原则

2.2 系统分层

2.2.1 层级说明

2.2.2 层间依赖关系

2.3 核心组件

- 2.3.1 组件 1: AgentLoop (默认驱动)
- 2.3.2 组件 2: ToolRuntime (工具注册表与执行管线)
- 2.3.3 组件 3: SessionStore (会话事件日志)
- 2.3.4 组件 4: LLM 适配器接缝
- 2.3.5 组件 5: 执行世界 (fs + subprocess + sandbox + shell)
- 2.3.6 组件 6: Boot 组合器
- 2.3.7 组件 7: Typert 类型图

2.4 数据流

- 2.4.1 Turn/Step 请求处理流程
- 2.4.2 数据持久化流程
- 2.4.3 能力接缝数据流

2.5 外部集成

2.6 架构图

2.7 技术选型说明

2.8 扩展性考虑

- 2.8.1 扩展点地图
- 2.8.2 Profile/Bundle 组合

2.9 性能与安全

- 2.9.1 性能
- 2.9.2 安全

3 L1-L2 应用入口与 Boot 组合层

3.1 层职责

- 3.1.1 L1 应用入口
- 3.1.2 L2 Boot 组合

3.2 主要组件

- 3.2.1 apps/cli (L1)
- 3.2.2 apps/web (L1)
- 3.2.3 boot/app-boot (L2)
- 3.2.4 boot/cmdline (L2)
- 3.2.5 bundle/* (L2)

3.3 对外接口

3.4 与其他层的交互

3.5 关键代码路径

3.6 技术实现

4 L3 Host / Client 双半层

4.1 层职责

4.2 主要组件

- 4.2.1 host/ (服务端半)
- 4.2.2 client/ (浏览器半)

4.3 对外接口

4.4 与其他层的交互

4.5 关键代码路径

4.6 技术实现

4.7 注意事项

5 L4 Core API 主干层

5.1 层职责

5.2 主要组件

5.2.1 core/session (会话事件日志)

5.2.2 core/agent (Agent 接口与注册表)

5.2.3 core/agent-loop (默认驱动)

5.2.4 core/tools (工具注册表与执行管线)

5.2.5 core/system-prompt (提示装配)

5.2.6 core/scope (per-agent 作用域)

5.2.7 core/agent-default-model / core/agent-tool-presentation

5.3 对外接口

5.4 与其他层的交互

5.5 关键代码路径

5.6 技术实现

5.7 注意事项

6 L5 能力接缝层

6.1 层职责

6.2 主要组件

6.2.1 执行环境能力接缝

6.2.2 模型协作能力接缝

6.3 对外接口

6.4 与其他层的交互

6.5 关键代码路径

6.6 技术实现

6.7 注意事项

7 L6 持久化与检索层

7.1 层职责

7.2 主要组件

7.3 对外接口

7.4 与其他层的交互

7.5 关键代码路径

7.6 技术实现

7.7 注意事项

8 L7 协议与集成层

8.1 层职责

8.2 主要组件

8.3 对外接口

8.4 与其他层的交互

8.5 关键代码路径

8.6 技术实现

8.7 注意事项

9 core/agent-loop — 默认驱动

9.1 模块概述

- 9.2 主要功能
- 9.3 目录结构
- 9.4 核心流程
- 9.5 核心符号
- 9.6 Turn/Step 生命周期
- 9.7 事件域
- 9.8 技术栈
- 9.9 依赖关系
 - 9.9.1 依赖模块
 - 9.9.2 被依赖模块
- 9.10 相关文档
- 10 core/session — 会话事件日志
 - 10.1 模块概述
 - 10.2 主要功能
 - 10.3 目录结构
 - 10.4 核心流程
 - 10.5 核心符号
 - 10.6 事件类型体系 (SessionEventMap)
 - 10.7 技术栈
 - 10.8 依赖关系
 - 10.8.1 依赖模块
 - 10.8.2 被依赖模块
 - 10.9 相关文档
- 11 core/tools — 工具注册表与执行管线
 - 11.1 模块概述
 - 11.2 主要功能
 - 11.3 目录结构
 - 11.4 核心流程
 - 11.5 核心符号
 - 11.6 执行管线三阶段
 - 11.7 执行模式 (ToolExecutionMode)
 - 11.8 技术栈
 - 11.9 依赖关系
 - 11.9.1 依赖模块
 - 11.9.2 被依赖模块
 - 11.10 相关文档
- 12 工具执行管线 - 子功能概览
 - 12.1 功能简介
 - 12.2 支持的阶段/类型
 - 12.3 通用流程
 - 12.4 核心接口
 - 12.5 扩展机制
 - 12.6 配置选项
 - 12.7 性能特点

- 12.8 注意事项
- 13 数据架构
 - 13.1 文件清单
 - 13.2 文档结构
 - 13.3 关键发现
 - 13.4 关键代码位置
 - 13.5 渲染 PUML
 - 13.6 相关文档
- 14 数据架构说明
 - 14.1 概述
 - 14.1.1 设计理念
 - 14.1.2 数据层次
 - 14.2 主数据库设计 (SQLite 持久化后端)
 - 14.2.1 数据库连接与所有权
 - 14.2.2 核心表结构
 - 14.2.3 事务边界
 - 14.2.4 撕裂尾修复 (scanRows)
 - 14.3 JSONL 会话日志设计
 - 14.3.1 文件布局
 - 14.3.2 文件格式
 - 14.3.3 写透发布 (Write-Through Publication)
 - 14.3.4 追加写与回滚
 - 14.3.5 增量扫描与撕裂修复
 - 14.4 核心数据模型
 - 14.4.1 SessionEventMap 事件类型体系
 - 14.4.2 SessionHeader (会话头)
 - 14.4.3 SessionEvent (事件信封)
 - 14.4.4 Session 类
 - 14.4.5 Session.append 详解
 - 14.4.6 SessionStore 类
 - 14.5 数据关系图 (ER 图说明)
 - 14.6 数据流
 - 14.6.1 事件追加→派生→持久化→检索
 - 14.6.2 会话分叉 (fork) 数据流
 - 14.6.3 会话恢复 (resume) 数据流
 - 14.6.4 session-query FTS 检索流程
 - 14.7 辅助存储
 - 14.7.1 附件内容寻址存储 (attachment-local)
 - 14.7.2 存储 Hub (storage)
 - 14.7.3 Spill 存储 (spill-local)
 - 14.7.4 投影缓存 (session-projection-cache)
 - 14.7.5 设置存储 (settings-file)
 - 14.7.6 凭据存储 (credentials-local)
 - 14.8 数据一致性保证

- 14.8.1 事件溯源不变式
- 14.8.2 双后端一致语义
- 14.8.3 持久化协调器不变式
- 14.9 性能优化
- 14.10 数据迁移与版本管理
 - 14.10.1 双版本体系
 - 14.10.2 无迁移策略
 - 14.10.3 升级路径
- 14.11 安全性设计
 - 14.11.1 凭据引用语义
 - 14.11.2 进程限制与隔离
 - 14.11.3 跨进程安全
- 14.12 数据架构组件图
- 14.13 关键发现总结
- 15 第三方集成总览
 - 15.1 集成概述
 - 15.2 集成类别列表
 - 15.2.1 LLM 与 AI 框架 (详见 01-llm-integration.md)
 - 15.2.2 沙箱与执行环境 (详见 02-sandbox-execution.md)
 - 15.2.3 协议集成 (详见 03-protocols.md)
 - 15.2.4 框架与构建工具 (详见 04-framework-vendor.md)
 - 15.3 统一接口设计: 能力接缝三角色
 - 15.3.1 三角色定义
 - 15.3.2 各能力接缝对照
 - 15.3.3 注册与销毁
 - 15.3.4 凭据引用模式
 - 15.4 环境变量清单
 - 15.5 扩展指南
 - 15.5.1 添加新 LLM Provider
 - 15.5.2 添加新沙箱 Backend
 - 15.5.3 添加新协议或 Hook Dialect
 - 15.5.4 通用扩展流程
 - 15.6 关键设计原则
 - 15.6.1 接缝优先
 - 15.6.2 Per-request Resolution
 - 15.6.3 Fail-Closed / Fail-Loud
 - 15.6.4 Atomic Replace
 - 15.6.5 No Secret in Config
 - 15.6.6 Contracts-Ready Pattern
 - 15.7 关键发现
- 16 LLM 与 AI 框架集成
 - 16.1 集成概述
 - 16.1.1 设计哲学
 - 16.2 支持的服务/产品

16.3 集成方式

16.3.1 Service Definition: `LlmRuntime` 与 `LlmAdapter`

16.3.2 Provider: `DeepSeekAdapter` (直连 fetch)

16.3.3 Provider: `PiAiAdapter` (pi-ai 库)

16.3.4 凭据解析

16.4 代码实现

16.4.1 关键类与文件

16.4.2 Provider 插件 apply 流程

16.5 配置与环境变量

16.5.1 环境变量

16.5.2 Plugin Config (`dsh-llm-deepseek`)

16.5.3 Retry Policy Schema

16.6 错误处理

16.6.1 LlmError Code 表

16.6.2 重试策略

16.6.3 降级: 保留上一个 good 配置

16.7 扩展指南: 添加新 LLM Provider

16.7.1 直连方式 (参考 `dsh-llm-deepseek`)

16.7.2 pi-ai 库方式 (参考 `dsh-llm-pi-ai`)

16.7.3 Token 计量

16.8 关键 SDK 依赖版本

16.9 关键发现

17 沙箱与执行环境集成

17.1 集成概述

17.1.1 设计哲学

17.2 支持的服务/产品

17.3 集成方式

17.3.1 Service Definition: `SandboxProvider` 与 `SandboxPolicy`

17.3.2 Provider: `LocalSandboxProvider` (多 runner 链)

17.3.3 Provider: `dsh-sandbox-windows-acl` (koffi FFI)

17.3.4 Native Addon: `@deepseek-ai/node-addon-landlock-run`

17.3.5 Provider: E2B 远程沙箱 (POC)

17.3.6 node-pty (patched 1.1.0)

17.4 代码实现

17.4.1 关键类与文件

17.4.2 平台选择与 probe 流程

17.5 配置与环境变量

17.5.1 环境变量

17.5.2 Plugin Config (`dsh-sandbox-local`)

17.5.3 Plugin Config (`dsh-e2b`)

17.6 错误处理

17.6.1 错误码表

17.6.2 Runner Failure vs Denial 区分

17.6.3 Probe 失败 = fail-closed

- 17.6.4 ACL Grant 清理
- 17.7 扩展指南：添加新沙箱 Backend
 - 17.7.1 新增本地 runner
 - 17.7.2 新增独立 Provider 包（参考 E2B）
 - 17.7.3 SandboxPolicy 设计原则
- 17.8 关键 SDK 依赖版本
- 17.9 关键发现
- 18 协议集成
 - 18.1 集成概述
 - 18.1.1 设计哲学
 - 18.2 支持的服务/产品
 - 18.3 集成方式
 - 18.3.1 ACP Server（[dsh-acp](#)）
 - 18.3.2 SDK Wire Protocol（[dsh-sdk-protocol](#)）
 - 18.3.3 SDK Server（[dsh-sdk-jsonrpc-server](#)）
 - 18.3.4 SDK Client（[dsh-sdk-client](#)）
 - 18.3.5 Hook Protocol Library（[dsh-hook-protocol](#)）
 - 18.3.6 Claude Code Hook Bridge（[dsh-hooks-claude-code](#)）
 - 18.3.7 Codex Hook Bridge（[dsh-hooks-codex](#)）
 - 18.4 代码实现
 - 18.4.1 关键类与文件
 - 18.4.2 ACP 与 SDK 的差异
 - 18.4.3 Hook Bridge 架构
 - 18.5 配置与环境变量
 - 18.5.1 环境变量
 - 18.5.2 Plugin Config
 - 18.6 错误处理
 - 18.6.1 ACP 错误
 - 18.6.2 SDK Client 错误
 - 18.6.3 Hook 错误
 - 18.7 扩展指南
 - 18.7.1 添加新 ACP 方法
 - 18.7.2 添加新 SDK 方法
 - 18.7.3 添加新 Hook Dialect Bridge
 - 18.8 关键 SDK 依赖版本
 - 18.9 关键发现
- 19 框架与构建工具集成
 - 19.1 集成概述
 - 19.1.1 设计哲学
 - 19.2 支持的服务/产品
 - 19.2.1 Vendored 框架
 - 19.2.2 第三方构建工具
 - 19.3 集成方式
 - 19.3.1 Vendored Cordis Workspace

19.3.2 pnpm Workspace + allowBuilds

19.3.3 构建工具链

19.4 代码实现

19.4.1 关键脚本

19.4.2 Contracts-Ready 模式

19.4.3 Gate 系统

19.4.4 Vendored Scope 重写

19.5 配置与环境变量

19.5.1 环境变量

19.5.2 关键配置文件

19.6 错误处理

19.6.1 构建错误

19.6.2 CI 错误

19.6.3 Vendored 链接错误

19.6.4 包不变量错误

19.7 扩展指南

19.7.1 升级 Vendored Cordis

19.7.2 添加新子包到 workspace

19.7.3 添加新第三方构建依赖

19.8 关键 SDK 依赖版本

19.8.1 Vendored 框架

19.8.2 构建工具链版本矩阵

19.9 关键发现

20 相关资源 (Related Resources)

20.1 官方一手资源

20.2 社区与支持

20.3 橙皮书系列 & 同类框架横向对比

20.4 本分析的方法与工具链

20.5 延伸阅读 / 第三方解读

20.6 版本与时效说明

1 DeepSeek Harness (dsh) - 项目概览

本文档基于 AST 符号工具对源码符号进行验证分析生成。代码引用处的行号均来自符号工具验证。

1.1 项目基本信息

项	值
项目名称	DeepSeek Harness (<code>dsh</code> , npm scope <code>@deepseek-ai/dsh-*</code>)
根包名	<code>@deepseek-ai/dsh-root</code>
版本	<code>0.1.0-rc.5</code> (Developer Preview, 迭代中, 存在破坏性变更)
项目描述	由 DeepSeek AI 开发的开源 Agent Harness (智能体框架)。采用「一切皆插件」架构, 基于 <code>Cordis</code> 框架构建。
开发语言	TypeScript (全 ESM) , 附带 Python SDK 与原生 Node Addon
运行时	Node.js <code>^22.19.0 >=24.0.0</code> (CLI 源码启动走 tsx 的 ESM-only hook)
包管理器	pnpm <code>11.7.0</code> (workspace monorepo)
许可证	MIT
仓库地址	https://github.com/deepseek-ai/deepseek-harness

1.1.1 核心架构理念

- 一切皆插件**: 没有特权核心。扩展 dsh 的方式是在其他插件旁挂载一个插件; 注册是可逆的 effect (`ctx.effect()` / `ctx.on()`) , 插件卸载时自动回滚。
- 能力接缝 (Capability Seams)** : 每个可替换能力由三角色构成——Service Definition (接口声明)、Service Provider (实现)、Consumer (消费方, 通常是面向模型的工具) 。三者缺一不可。
- 模型可见即已记录**: 任何到达模型请求的内容都必须能从会话日志重建。新增模型可见输入需扩展 `SessionEventMap` 并从日志渲染。
- Profile / Bundle 组合**: 运行中的 `dsh` 是启动时按有序层组合的插件树。 `dsh-base` 是每个 profile 的第一层。

1.2 项目目录结构

```
deepseek-harness/
├─ packages/          # @deepseek-ai/dsh-<pkg> 工作区（49 个 group，详见第 3 节）
│   ├─ core/          # 产品 API 主干：session、system-prompt、tools、agent、agent-loop
│   ├─ api/           # 远程 BFF 装配 + Typert RPC 网关
│   ├─ typert/        # 类型图生成、加载、运行时注册
│   ├─ llm/           # LLM 能力：Service Definition + DeepSeek/pi-ai 适配器
│   ├─ e2b/           # E2B 沙箱 POC
│   ├─ shell/         # Bash 能力：执行接缝 + local/pwsh provider + 工具
│   ├─ subprocess/    # 子进程能力 + 本地进程树 provider
│   ├─ terminal/      # 持久 PTY 会话
│   ├─ code-runtime/  # 代码执行能力：worker-thread provider + Code Mode
│   ├─ sandbox/       # 进程限制接缝：bwrap/Landlock/Seatbelt
│   ├─ fs/            # 文件系统能力 + 策略
│   ├─ lsp/           # LSP 能力
│   ├─ skill/         # Skill provider 注册表 + 本地实现 + catalog/loader 工具
│   ├─ web/           # Web 能力：search/fetch provider + 工具
│   ├─ compaction/    # 压缩能力 + basic provider
│   ├─ context/       # 请求上下文插件（workspace 指令、时间上下文）
│   ├─ subagent/      # 子 agent 能力：provider 注册表 + 委派工具
│   ├─ jobs/          # 通用后台任务 + job_* 工具
│   ├─ workflow/      # workflow 接缝 + worker-thread 引擎 + ralph 工具
│   ├─ todo/          # todo_write 工具
│   ├─ plan/          # 计划协作状态（直接进入 + 审核退出）
│   ├─ preset/        # 预设 cordis.yml 的 per-session agent 组合
│   ├─ guard/         # 循环卫生守卫 + 工具超时强制器
│   ├─ extensions/    # Agent 运行时自修改（实时插件挂载/卸载）
│   ├─ hooks/         # Claude Code/Codex hook 桥 + wire-protocol 库
│   ├─ session/       # 持久会话数据：JSONL/SQLite 后端、投影、标题
│   ├─ session-query/ # 会话检索：FTS、lineage、语义过滤
│   ├─ settings/      # 用户设置接缝 + 文件 provider
│   ├─ credentials/   # 凭据引用接缝 + env/.env provider
│   ├─ storage/       # 非会话存储 hub + 后端
│   ├─ workspace/     # Workspace 实体
│   ├─ attachment/    # 持久附件身份 + 内容寻址存储
│   ├─ spill/         # Spill 能力：存储接缝 + 工具结果溢出策略
│   ├─ goal/          # 同会话目标持久化与生命周期
│   ├─ schedule/      # 会话本地定时跟进
│   ├─ feedback/      # 人类反馈
│   ├─ identity/      # 匿名身份
│   ├─ sdk/           # 进程外运行时 SDK：JSON-RPC 协议 + TS client + server
│   ├─ acp/           # 仅自动化的 Agent Client Protocol server
│   ├─ interaction/   # 人机协作面：审批/交互接缝、权限、命令、ask-user
│   ├─ boot/          # 共享 app-bin 启动胶水
│   ├─ host/          # Web-GUI host 半：API 网关 + HTTP 路由
│   ├─ client/        # Web-GUI 浏览器半：shell、wire、ui-* 插件
│   ├─ bundle/        # 可安装的 dsh --profile 补丁层（base/headless/web-app）
│   ├─ examples/      # 示例 bundle（agent-spine + CLI/ACP/JSON-RPC bin）
│   ├─ test-support/  # 测试基础设施（testkit、invariant、replay）
│   └─ util/          # 零依赖底层工具（Branded、路径、超时）
├─ apps/              # 产品装配
```

```
|   └─ cli/           # dsh CLI 入口（拥有 dsh bin: apps/cli/src/bin.ts）
|   └─ web/           # Web UI 前端（Vite，默认 http://127.0.0.1:3080）
└─ vendor/           # vendored Cordis 源码（cosmokit、schemastery 重 scope 为 @deepseek-ai/*）
└─ native/           # @deepseek-ai/node-addon-landlock-run 原生 addon 源码
└─ python/           # Python SDK + 捆绑运行时（python/sdk-runtime 为部署 workspace）
└─ examples/         # 可运行 cordis.yml 叶子（demo bundle）
└─ docs/             # 双语架构文档、生成目录、cookbook、postmortem
|   └─ subsystems/    # 子系统深度文档
|   └─ cookbook/      # 扩展 cookbook
|   └─ cordis-tutorial/
|   └─ postmortem/
└─ scripts/          # 仓库门禁与生成器（TS，经 tsx 运行）
└─ website/          # VitePress 文档站点投影
└─ .agents/          # Agent 工作流 + Agent Notes（notes/）
└─ patches/          # 依赖补丁（node-pty@1.1.0.patch）
└─ 配置文件: package.json / pnpm-workspace.yaml / tsconfig*.json / tsdown.config.ts / vitest*.config.ts / oxlintrc*.json /
  knip.json / lefthook.yml / .jscpd.json
```

1.3 主要功能模块

1.3.1 核心 API 主干（packages/core/，8 个包）

产品 API 脊柱，提供稳定 API。每个包贡献一个 `ctx` key。

包	职责	ctx key
core/session	追加只写的 <code>SessionEvent</code> 日志 + 内存存储 (符号验证: <code>Session</code> 类 L424-757、 <code>SessionStore</code> 类 L791-1154、 <code>SessionEventMap</code> 接口 L235-332)	ctx.sessions
core/system-prompt	Prompt section 与 tool schema 装配	ctx.systemPrompt
core/tools	作用域工具注册表 + 受保护的执行管线	ctx.tools
core/agent	Agent 接口、实时注册表、agent/* 事件	ctx.agents
core/agent-loop	实现该接口的默认驱动	ctx.agentLoop
core/agent-default-model	默认模型绑定	—
core/agent-tool-presentation	工具呈现	—
core/scope	每 agent 的作用域注册原语（库，无 ctx key）	—

1.3.2 LLM 能力族 (packages/llm/ , 5 个包)

包	职责
llm/llm	消息与流词汇表 + 适配器接缝 (ctx.llm)
llm/llm-deepseek	DeepSeek 官方适配器
llm/llm-pi-ai	@earendil-works/pi-ai 可选后端 (含 Google GenAI)
llm/llm-retry	重试策略
llm/token-meter	Token 计量

1.3.3 能力接缝族 (执行环境类)

包族	能力	核心 ctx key
shell/	Bash 执行 (local + pwsh)	ctx.shell
subprocess/	子进程 (本地进程树)	ctx.subprocess
terminal/	持久 PTY 会话	ctx.terminals
code-runtime/	代码执行 (worker-thread + Code Mode)	—
sandbox/	进程限制 (bwrap / Landlock / Seatbelt)	ctx.sandbox
fs/	文件系统 + 策略	ctx.fs
lsp/	语言服务器协议	—
e2b/	E2B 远程沙箱 (POC)	—

1.3.4 能力接缝族（模型协作类）

包族	能力	核心 ctx key
skill/	Skill provider 注册表 + 本地实现 + catalog/loader 工具	—
web/	Web search/fetch	—
compaction/	上下文压缩	—
context/	请求上下文（workspace 指令、时间）	—
subagent/	子 agent 委派	—
jobs/	后台任务 + job_* 工具	ctx.jobs
workflow/	workflow 接缝 + ralph 工具	—
todo/	todo_write 工具	—
plan/	计划协作状态	—
preset/	预设 cordis.yml 组合	—
guard/	循环卫生守卫 + 工具超时	—
extensions/	运行时自修改（实时插件挂载/卸载）	—

1.3.5 持久化与检索

包族	能力
session/	持久会话数据面：JSONL/SQLite 后端、投影、标题、报告
session-query/	会话检索：FTS、lineage、事件关系、语义过滤
storage/	非会话存储 hub + 后端
attachment/	持久附件身份 + 内容寻址存储
spill/	Spill 存储 + 工具结果溢出策略
settings/	用户设置接缝 + 文件 provider
credentials/	凭据引用接缝 + env-over-.env provider
workspace/	Workspace 实体
goal/	同会话目标持久化
schedule/	会话本地定时跟进
feedback/	人类反馈
identity/	匿名身份

1.3.6 协议与集成

包族	能力
api/	远程 BFF 装配 + Typert RPC 网关 (gateway + remotes)
typert/	类型图生成 (generator) + 加载 (loader) + 协议 (protocol) + 注册表 (registry)
sdk/	进程外运行时 SDK: JSON-RPC 协议 + TS client + server 插件
acp/	仅自动化的 Agent Client Protocol server
hooks/	Claude Code/Codex hook 桥 + wire-protocol 库
interaction/	人机协作面: 审批/交互接缝、权限、命令、ask-user 工具

1.3.7 应用装配与 Boot

包族	能力
bundle/	可安装 profile 补丁层: base (模型/工具/持久化/沙箱/审批/设置/凭据/遥测)、 headless (无服务器一次性运行)、 web-app (浏览器应用)
boot/	共享 app-bin 启动胶水
host/	Web-GUI host 半: API 网关 + HTTP 路由
client/	Web-GUI 浏览器半: shell、wire、object services、ui-* 插件

1.3.8 应用入口 (apps/)

应用	路径	说明
apps/cli	apps/cli/src/bin.ts	dsh CLI 入口, 经 node --import tsx/esm 启动; 含 config、reference、tests
apps/web	apps/web/	Web UI 前端, Vite 构建, index.html 入口, 默认服务在 http://127.0.0.1:3080

1.4 核心依赖分析

1.4.1 框架与核心

依赖	版本	用途
@deepseek-ai/cordis	vendored (workspace peer)	插件/上下文框架, 每个 harness 包的 peerDependency
@deepseek-ai/cosmokit	link:vendor/cosmokit (override)	Cordis 工具库 (vendored 重 scope)
@deepseek-ai/schemastery	link:vendor/schemastery (override)	配置 schema (vendored 重 scope)
@agentclientprotocol/sdk	0.25.1	ACP 协议 SDK
@earendil-works/pi-ai	(可选)	可选 LLM API 后端 (拉入 @google/genai)

1.4.2 构建与工具链

依赖	版本	用途
typescript	^6.0.3 (peer >=5 <7)	类型系统
tsdown	^0.22.2	运行时打包 (host/client 双 face)
tsx	^4.22.4	ESM-only TS 执行 hook (CLI 源码启动 + scripts)
vitest	^4.1.8	测试框架
@vitest/coverage-v8	^4.1.8	覆盖率 (per-file 100% 门禁)
oxlint	1.76.0	Lint (非 eslint)
oxlint-tsgolint	7.0.2001	oxlint 的 TS 类型规则
knip	^6.16.1	未使用代码检测
publint	^0.3.21	发布字段检查
jscpd	^5.0.12	重复代码检测
lefthook	^2.1.9	Git hooks
vite-tsconfig-paths	^6.1.1	Vite 路径解析
lightningcss	^1.32.0	CSS 处理
mermaid	11.16.0	图表渲染 (文档校验)

1.4.3 测试与质量

依赖	版本	用途
@testing-library/dom	^10.4.1	DOM 测试
@testing-library/react	^16.3.2	React 组件测试
fast-check	^4.8.0	属性测试
jsdom	29.1.1	DOM 模拟
execa	^10.0.0	进程执行
mdast-util-from-markdown / mdast-util-gfm	—	Markdown 解析（文档校验）

1.4.4 原生与系统

依赖	说明
node-pty 1.1.0 (patched)	跨平台 PTY 后端（含 Windows ConPTY）；补丁见 patches/node-pty@1.1.0.patch
koffi	Windows MoveFileExW 写透发布（JSONL 持久化）
@deepseek-ai/node-addon-landlock-run	Landlock 沙箱原生 addon（源码在 native/）
esbuild	原生二进制打包（允许 build 脚本）

1.4.5 持久化

- **SQLite**: 单调 SCHEMA_VERSION，无兼容性承诺（pre-release）。
- **JSONL**: 会话日志持久化，Windows 下用 MoveFileExW 写透发布。
- dsh-session 维持 SESSION_FORMAT_VERSION = 0，无兼容性承诺。

1.5 构建与部署

1.5.1 开发环境

```
# 前置: Node.js ^22.19 || >=24, pnpm 11.7.0
git clone https://github.com/deepseek-ai/deepseek-harness.git
cd deepseek-harness
pnpm install          # workspace 安装（触发 Lefthook postinstall）
pnpm run build        # build:lib:host + build:lib:client + build:web
pnpm dsh web          # 启动 Web UI (http://127.0.0.1:3080)
```

1.5.2 构建命令矩阵

命令	作用
<code>pnpm run build:lib:host</code>	<code>tsc -b tsconfig.host.json</code> + <code>tsdown --env.DSH_BUILD_FACE host</code>
<code>pnpm run build:lib:client</code>	<code>tsc -b tsconfig.client.json</code> + <code>tsdown --env.DSH_BUILD_FACE client</code>
<code>pnpm run build:web</code>	<code>pnpm --filter @deepseek-ai/dsh-web-frontend run build</code> (Vite)
<code>pnpm run clean</code>	<code>tsx scripts/clean.ts</code> (清理构建产物与已删包残留)
<code>pnpm run typecheck</code>	<code>build:lib:host</code> + <code>tsc -b tsconfig.client.json</code>

构建产物双 face 设计：`host` (Node 服务端) 与 `client` (浏览器侧)，通过 `DSH_BUILD_FACE` 环境变量切换。

1.5.3 运行方式

方式	命令	说明
从 npm 运行	<code>npx @deepseek-ai/dsh web</code>	无需克隆，启动 Web UI
从源码运行	<code>pnpm dsh web</code>	需先 build
Headless 任务	<code>pnpm dsh --profile headless "task"</code>	一次性运行，需 <code>DEEPSEEK_API_KEY</code>
查看插件树	<code>dsh --profile web --dump-config</code>	打印启动时实际加载的插件树
Demo	<code>pnpm run demo:cordis</code> / <code>demo:acp</code>	演示 (需 key)
LLM Mock	<code>pnpm run mock:llm</code>	本地 LLM mock server (测试用)
Web 开发	<code>pnpm run dev:web</code>	Vite 开发模式 (轮询)

1.5.4 测试命令

命令	作用
<code>pnpm run test</code>	<code>vitest run</code> (单元测试)
<code>pnpm run test:coverage</code>	CI 覆盖率门禁: <code>packages/**/*.src</code> 每文件 100%
<code>pnpm run test:e2e</code>	真实 API 测试, 无 <code>DEEPSEEK_API_KEY</code> 自动跳过
<code>pnpm run test:snapshot</code>	无 key 的 ACP/headless 回放; <code>-t <name></code> 过滤
<code>pnpm run test:snapshot:record</code>	重新录制期望输出 (需 key)
<code>pnpm run test:web</code> / <code>test:web:stress</code> / <code>test:web:perf</code>	Web 测试矩阵

1.5.5 质量门禁

```
pnpm run lint           # oxlint (先 build:Lib:host 再 lint)
pnpm run duplication    # jscpd (packages + scripts)
pnpm run knip           # --treat-config-hints-as-errors
pnpm run publint        # 发布字段
pnpm run constraints    # workspace 约束
pnpm run hygiene        # 综合卫生检查 (rescope + knip + publint + constraints + 许可证 + 包不变量 + cordis-config + node-next-
                        types + runtime-closure + vendored-links)
pnpm run doc-sync       # 所有文档门禁
pnpm run website:build  # VitePress 构建 (兼作死链检查)
```

1.5.6 部署形态

- npm 包发布: @deepseek-ai/dsh-* 系列, publishConfig.access: public。
- 单可执行文件: python/sdk-runtime 是部署根, 其依赖闭包是 exe 捆绑内容, 由 Python 运行时分发。
- 无 Docker/K8s: 仓库未包含容器化部署配置, 面向本地/CLI/SDK 集成场景。

1.6 环境配置

1.6.1 必需环境变量

变量名	用途	必需	示例
DEEPSEEK_API_KEY	DeepSeek 真实 API 调用 (e2e 测试、demo、headless 运行)	运行真实 API 时必需	sk-...

1.6.2 可选环境变量

变量名	用途	说明
DEEPSEEK_BASE_URL	DeepSeek API 基址覆盖	可选
DSH_BUILD_FACE	构建面选择	host / client
DSH_SNAPSHOT	快照测试模式	record / refresh / replay
NODE_PATH	managed workspace 模块解析	C:\...\node\workspace\node_modules

1.6.3 配置文件

文件	作用
<code>.env</code> (根)	<code>DEEPSEEK_API_KEY</code> 、 <code>DEEPSEEK_BASE_URL</code> (不提交)
<code>cordis.yml</code>	插件组合配置; <code>!!js</code> 允许 (非 <code>!js</code>) 用于 plugin <code>config</code> 与 entry <code>disabled</code> , 其余元数据保持字面量
<code>cordis.patch.yml</code>	profile 级 / home 级补丁, 按 id 替换整行 config 或插入新行
<code>tsconfig.host.json</code> / <code>tsconfig.client.json</code>	双 face TypeScript 构建
<code>tsdown.config.ts</code>	运行时打包配置
<code>vitest.*.config.ts</code>	多套测试配置 (unit/e2e/snapshot/web/stress/perf)
<code>.oxlintrc.json</code> / <code>.oxlintrc.staged.json</code>	Lint 规则
<code>knip.json</code>	未使用代码检测配置
<code>lefthook.yml</code>	Git hooks 配置
<code>.jscpd.json</code>	重复代码检测配置
<code>pnpm-workspace.yaml</code>	workspace 与 override (cosmokit/schemastery link 到 vendor)

1.6.4 Profile 组合顺序

层按以下顺序应用到空 entry 列表:

- 1. profile 列出的每个 bundle (按列出顺序)
- 2. profile 的 `cordis.patch.yml`
- 3. home 级 `cordis.patch.yml`
- 4. 任何 `--patch` overlay

任一行可通过 patch 替换。`dsh-base` 是每个 profile 的第一层 (模型适配器、工具、持久化、沙箱、审批策略、设置、凭据、遥测) 。

下一步: 系统架构

2 系统架构说明

本文档基于 AST 符号工具对源码符号验证生成。架构图见 `system-component-architecture.puml`。

2.1 架构概述

2.1.1 架构模式

DeepSeek Harness（`dsh`）采用**插件式 Agent Harness 架构**，核心设计原则源自 **Cordis** 框架。整体融合了以下架构模式：

模式	体现
插件架构（一切皆插件）	没有特权核心。模型适配器、工具注册表、会话日志、agent loop 本身都是插件，均可从配置替换。
能力接缝（Capability Seams）	每个可替换能力由三角色构成：Service Definition（接口）、Service Provider（实现）、Consumer（消费，常为面向模型的工具）。
事件驱动 / 事件溯源	会话是追加只写的事件日志（ <code>SessionEvent</code> ）；派生、分叉、恢复、遥测均从事件流推导。
Monorepo（pnpm workspace）	49 个 package group，按职责分组，统一 scope <code>@deepseek-ai/dsh-*</code> 。
分层组合（Profile / Bundle）	运行中的 <code>dsh</code> 是启动时按有序层组合的插件树，支持 patch 覆盖。
Host / Client 双半	Web-GUI 拆为 host（API 网关）与 client（浏览器侧）两半，各自独立打包。

2.1.2 设计原则

- 一切皆插件，注册是可逆 effect**：每个贡献通过 `ctx.effect()` / `ctx.on()` / `ctx.waterfall()` 进入共享上下文；`register()` 返回 disposer，插件卸载时自动回滚。
- 模型可见即已记录**：任何到达模型请求的内容必须能从会话日志重建。运行时不变式断言此约束。新增模型可见输入需扩展 `SessionEventMap` 并从日志渲染。
- 扩展插件依赖 Service Definition，而非具体 Provider**：`dsh-agent-loop` 可替换；UI、hook、工具插件使用 `dsh-agent`。换一个 provider 即迁移整个执行世界（如本地→远程沙箱）。
- 运行时不变式断言所有权关系**：检查权威事件流或可变数据，而非服务/方法存在性、插件元数据或固定示例。
- Pre-release 优先正确基础**：无外部消费者时，优先正确基础而非兼容垫片——可自由重命名/重打包并同步更新所有引用。

2.2 系统分层

2.2.1 层级说明

系统自顶向下分为 7 层（详见 [分层架构文档](#)）：

层	职责	代表包
L1 应用入口	CLI/Web 启动入口	apps/cli 、 apps/web
L2 Boot 组合	Profile/Bundle 层组合、命令行解析	boot/app-boot 、 boot/cmdline 、 bundle/*
L3 Host/Client 双半	API 网关、HTTP 路由、浏览器 shell	host 、 client
L4 Core API 主干	会话、agent、工具、prompt、loop	core/*
L5 能力接缝	执行环境 + 模型协作能力	shell 、 fs 、 llm 、 skill 、 web 等
L6 持久化与检索	会话持久化、存储、设置、凭据	session 、 storage 、 settings
L7 协议与集成	RPC、ACP、hooks、交互	api 、 typert 、 sdk 、 acp 、 hooks 、 interaction

所有层均构建于 vendored Cordis 框架之上。

2.2.2 层间依赖关系

- 向下依赖：上层依赖下层 Service Definition，不依赖具体 Provider。
- 无循环依赖：`dsh-agent-loop` 可被替换；能力 Consumer 通过 `ctx.tools` 注册，不直接依赖 loop。
- Cordis 是 peerDependency：每个 harness 包都将 `@deepseek-ai/cordis` 声明为 peerDependency (+ dev)，确保单一上下文实例。
- 依赖图由生成器维护：`pnpm run gen-module-graph` 生成 `docs/module-graph.md`，CI 新鲜度门禁校验。

2.3 核心组件

2.3.1 组件 1：AgentLoop（默认驱动）

- 职责：实现 `Agent` 接口的默认驱动，编排 Turn/Step 生命周期。
- 主要功能：claim 输入 → 装配 prompt+tools → 模型请求 → 工具执行 → 步结束 → 回合结束。
- 技术实现：基于 Cordis waterfall 事件（`agent/pre-step`、`agent/request`、`llm/stream`、`tools/*`）；`agent/turn-stopping` 是串行终端检查点（无 `next()`）。
- 代码路径：`packages/core/agent-loop/src/index.ts`
- 符号验证：`AgentLoop` 类 (L295-710)、`createAgent` 方法 (L605-621)、`ReactLoopAgent` 类（`agent.ts` L63-495）、`AgentLoopSettings` 接口 (L243-246)。

2.3.2 组件 2：ToolRuntime（工具注册表与执行管线）

- 职责：作用域工具注册表 + 受保护的执行管线。
- 主要功能：工具定义注册、pre/concurrent execute/post 三阶段调度、barrier 与有界滚动池、工具结果融合。
- 技术实现：`ToolDefinition` 接口描述工具 schema；`ToolLayer` 管理分层；`defineTool()` 工厂（`schema.ts` L544-616）。
- 代码路径：`packages/core/tools/src/index.ts`
- 符号验证：`ToolRuntime` 类 (L786-1862)、`ToolDefinition` 接口 (L221-287)、`ToolLayer` 类 (L713-753)、`dispatchToolBody` 方法 (L1531-1559)、`fuseToolSignals` 函数 (L1888-1915)。

2.3.3 组件 3：SessionStore（会话事件日志）

- 职责：追加只写的 `SessionEvent` 日志 + 内存存储；派生模型历史。
- 主要功能：事件追加、模型历史派生（`deriveMessages()`）、会话分叉（`fork`）、标题生成、遥测。
- 技术实现：事件溯源；`SessionEventMap` 声明合并扩展；raw `assistant/chunk` 保留回放与 UI 保真。

- 代码路径: `packages/core/session/src/index.ts`
- 符号验证: `Session` 类 (L424-757)、`SessionStore` 类 (L791-1154)、`SessionEventMap` 接口 (`types.ts` L235-332)、`SessionHeader` 接口 (L60-98)。

2.3.4 组件 4: LLM 适配器接缝

- 职责: 消息与流词汇表 + 适配器接缝。
- 主要功能: 流式 chunk、消息类型、provider 注册。
- 技术实现: `ctx.llm` waterfall; DeepSeek 官方适配器 + 可选 pi-ai 后端。
- 代码路径: `packages/llm/llm/src`、`packages/llm/llm-deepseek/src`

2.3.5 组件 5: 执行世界 (fs + subprocess + sandbox + shell)

- 职责: 文件系统访问、子进程生成、进程限制、Bash 执行。
- 主要功能: 共享一个执行世界——指向远程沙箱即可迁移 Bash、PTY、LSP。
- 技术实现: `ctx.fs` + `ctx.subprocess` + `ctx.sandbox` + `ctx.shell` ; 沙箱后端 bwrap (Linux) / Landlock (Linux, 原生 addon) / Seatbelt (macOS) 。
- 代码路径: `packages/fs/`、`packages/subprocess/`、`packages/sandbox/`、`packages/shell/`

2.3.6 组件 6: Boot 组合器

- 职责: 启动时按有序层组合插件树。
- 主要功能: 读取 profile → 加载 bundle 列表 → 应用 patch overlay → 构建插件树。
- 技术实现: 层应用顺序: 每个 bundle (profile 列出顺序) → profile `cordis.patch.yml` → home 级 → `--patch` overlay。
- 代码路径: `packages/boot/app-boot/src`

2.3.7 组件 7: Typert 类型图

- 职责: 类型图生成、加载、运行时注册。
- 主要功能: 跨语言 (TS/Python) 类型契约; RPC 网关的类型基础。
- 代码路径: `packages/typert/generator`、`packages/typert/loader`、`packages/typert/protocol`、`packages/typert/registry`

2.4 数据流

2.4.1 Turn/Step 请求处理流程

一个 **step** = 一次模型请求 + 其调用的工具。一个 **turn** = 零或多个 step: 在第一个输入被 claim 前打开, 在无待办时关闭。

```

turn/start
  claim 下一 step 输入 + 一条排队消息
  装配 prompt sections + tool schemas
  -> agent/pre-step (waterfall)      拒绝 | enter(messages)
    拒绝 / 首次 enter 重写为空 -> 无 step 关闭 turn
  step/start
    追加 entered messages 为 user/message
  从日志派生模型历史
  agent/request -> llm/stream -> assistant/chunk* -> assistant/message
  tool/call* -> tools/pre-execute -> tools/execute -> tools/post-execute -> tool/result*
  step/end
    工具欠另一次请求 / 下一 step 输入到达 -> claim -> 下一 step
  -> agent/turn-stopping (serial, 无 next)
turn/end

```

事件域划分： - **持久会话事件：** `turn/*`、`step/*`、`user/message`、`assistant/*`、`tool/*`（写入日志，可重放）。 - **实时扩展点：** `agent/pre-step`、`agent/request`、`llm/stream`、`tools/*`（waterfall，监听者须调 `next()` 委托）。 - **串行终端：** `agent/turn-stopping`（无 `next()`）。

输入通过单一 inbox 到达驱动。部分消息立即唤醒；注入的上下文在 inbox 中等待直至另一消息到达。`agent/pre-step` 决定模型所见——监听者可重写 claimed 消息或直接拒绝。

2.4.2 数据持久化流程

- **会话事件**追加到 `SessionEvent` 日志（内存）→ 由 `session/` 包持久化到 JSONL 或 SQLite。
- `deriveMessages()` 从日志派生模型历史；raw `assistant/chunk` 保留回放与 UI 保真。
- Fork、resume、transcripts、telemetry、persistence 均从事件流派生。
- **不变式：** 到达模型请求的任何内容必须可从日志重建——运行时断言此约束。

2.4.3 能力接缝数据流

```

模型 tool/call
  -> ctx.tools (ToolRuntime)
    -> tools/pre-execute (waterfall, 策略/审批)
    -> Consumer 工具 (如 Bash/file/web)
      -> Service Definition 接缝 (ctx.fs / ctx.shell / ctx.subprocess)
        -> Service Provider (local / sandbox / e2b)
    -> tools/post-execute (waterfall)
  -> tool/result (持久)

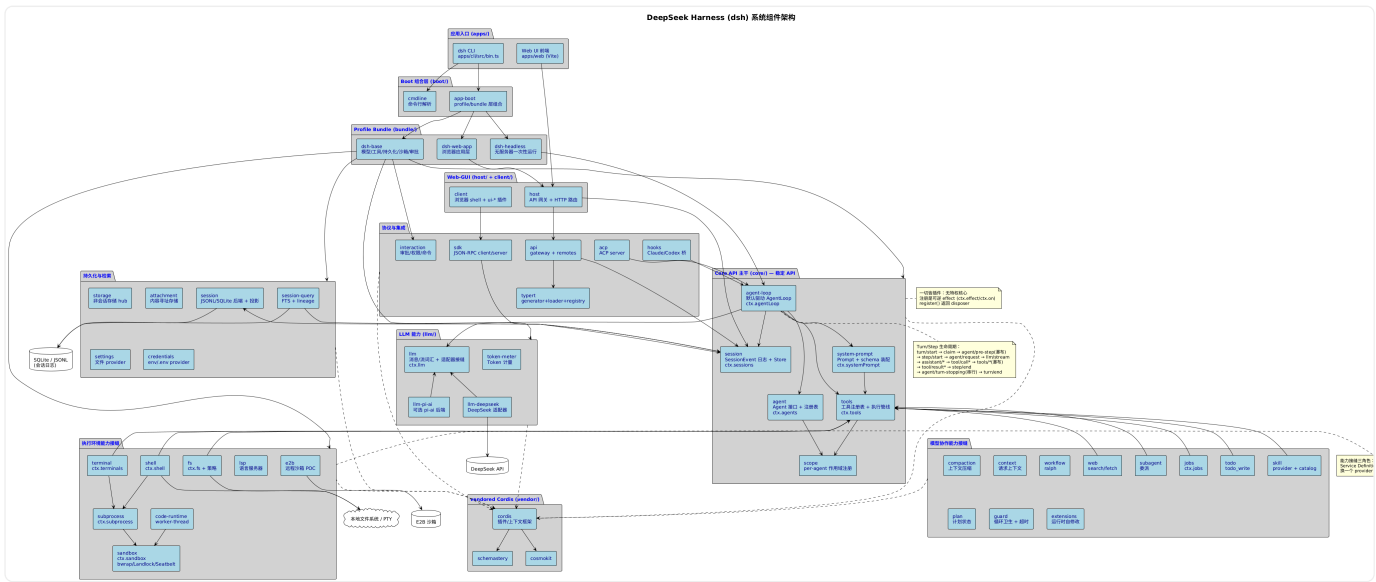
```

换一个 provider（如本地→E2B 远程沙箱）即可迁移整个执行世界，无需 provider 分叉。

2.5 外部集成

集成点	用途	实现包
DeepSeek API	LLM 模型调用	<code>llm/llm-deepseek</code>
Google GenAI (可选)	经 pi-ai 后端的可选 LLM	<code>llm/llm-pi-ai</code> (<code>@google/genai</code> build 脚本被拒)
E2B 云沙箱	远程代码执行 POC	<code>e2b/</code>
SQLite	结构化会话持久化	<code>session/</code> (monotonic SCHEMA_VERSION)
JSONL	流式会话日志持久化	<code>session/</code> (Windows 用 <code>MoveFileExW</code> 写透)
本地文件系统 / PTY	文件读写、Bash、终端	<code>fs/</code> 、 <code>shell/</code> 、 <code>terminal/</code> 、 <code>subprocess/</code>
Landlock / bwrap / Seatbelt	进程限制	<code>sandbox/</code> (含原生 add-on <code>native/landlock-run</code>)
LSP 服务器	语言智能	<code>lsp/</code>
ACP (Agent Client Protocol)	自动化协议	<code>acp/</code>
JSON-RPC	进程外 SDK 通信	<code>sdk/</code>
Claude Code / Codex hooks	hook 桥接	<code>hooks/</code>

2.6 架构图



2.7 技术选型说明

选型	理由
Cordis (vendored)	插件/上下文编程范式，支持时空可组合性；vendored 以锁定版本并重 scope 为 <code>@deepseek-ai/*</code> 。
TypeScript ESM	类型安全 + 原生 ESM；tsx ESM-only hook 跨引擎范围启动源码。
pnpm workspace	49 个包的高效依赖管理； <code>linkWorkspacePackages</code> + <code>overrides</code> 锁定 vendored 依赖。
oxlint (非 eslint)	性能优先的 lint； <code>oxlint-tsgolint</code> 补充类型规则。
Vitest	原生 ESM 测试；多配置 (unit/e2e/snapshot/web) 覆盖不同场景。
tsdown 双 face	host/client 分离打包，匹配 Host/Client 双半架构。
SQLite + JSONL 双后端	JSONL 流式写、SQLite 结构化查询；monotonic SCHEMA_VERSION 无兼容负担 (pre-release)。
能力接缝三角色	解耦接口与实现，使 provider 可替换迁移整个执行世界。

2.8 扩展性考虑

2.8.1 扩展点地图

目标	机制
添加模型 provider	在 <code>ctx.llm</code> 注册适配器
添加面向模型的能力	在 <code>ctx.tools</code> 注册；其 schema 加入 prompt 装配
给某会话不同能力集	组合 agent preset；service 行需 <code>isolate</code> realm
添加 shell 执行	注册 <code>ctx.shell</code> 后端；local 经 <code>ctx.subprocess</code> 生成
添加持久终端	注册 <code>ctx.terminals</code> 后端 + <code>dsh-tool-terminal</code>
添加人类命令	注册 <code>ctx.commands</code> ；无需模型 turn 即分发
添加后台工作	注册 <code>ctx.jobs</code> ； <code>job_*</code> 工具收集或停止
添加文件系统访问/策略	注册 <code>ctx.fs</code> provider 或监听 <code>fs/*</code> 事件
限制生成进程	使用 <code>ctx.sandbox</code> 后端；consumer 在 spawn 前包装 argv
拦截请求/工具/turn	用 <code>agent/*</code> 或 <code>tools/*</code> 事件； <code>agent/turn-stopping</code> 停止 turn
添加模型可见上下文	调 <code>agent.inject()</code> ；落入下次 admitted 请求
添加 UI/编辑器集成	驱动 <code>ctx.agents</code> 并从 <code>session/event</code> 渲染
添加 Web Client Chat 节点	注册 <code>ConversationNodeDefinition</code> + keyed renderer
添加持久会话状态	扩展 <code>SessionEventMap</code> ；从日志渲染并重放
生成会话标题	注册唯一 <code>ctx.sessionTitle</code> provider
管理同会话目标	用 <code>ctx.goals</code> ；经 <code>agent/*</code> 继续
分叉实时会话	<code>ctx.sessions.fork(source, boundary?, childSessionId?)</code>
作用域注册到一个 agent	用该 agent 的 <code>agent.ctx</code>

2.8.2 Profile/Bundle 组合

- 新 profile 在 Harness home 定义，列出所堆叠的 bundle。
- 新 bundle 在 `package.json` 的 `dsh.bundle` 字段指向补丁文件。
- `dsh-base` 是每个 profile 第一层；`dsh-web-app` / `dsh-headless` 添加应用特定层。
- 任意行可通过 `dsh --profile <name> --dump-config` 查看并用 patch 替换。

2.9 性能与安全

2.9.1 性能

- **工具并发调度**：pre/post 有序，execute 并发；barrier 与有界滚动池控制并发度。
- **流式响应**：`llm/stream` waterfall + `assistant/chunk` 持久化，保留 UI 保真与回放。

- 上下文压缩: `dsh-compaction-basic` 经 `agent/pre-step` 检测压力, `agent/request-error` 处理上下文溢出, 支持工具结果剪枝与摘要选择。
- 覆盖率门禁: `packages/*/*/src` 每文件 100% (CI gate) 。

2.9.2 安全

- 进程限制: `ctx.sandbox` 后端 (Landlock/bwrap/Seatbelt) 限制生成进程能力。
- 审批策略: `interaction/` 提供审批/交互接缝与权限预设; `tools/pre-execute` waterfall 注入策略。
- 凭据管理: `credentials/` 提供 env-over-`.env` provider; 凭据引用而非明文。
- `strictDepBuilds`: pnpm 10+ 默认拒绝带 install/build 脚本的依赖, 除非显式 allowlist。
- 补丁审查: `node-pty` 等依赖经 `patches/` 显式补丁。
- 不变式断言: 运行时断言所有权关系与「模型可见即已记录」, 防止状态漂移。

下一步: [分层架构说明](#)

3 L1-L2 应用入口与 Boot 组合层

参照 [系统组件架构图](#)。本文档覆盖 L1 (应用入口) 与 L2 (Boot 组合) 。

3.1 层职责

3.1.1 L1 应用入口

提供用户可达的启动入口: CLI (`dsh` 命令) 与 Web UI (浏览器) 。负责命令行解析、参数分发、启动 Boot 组合器。

3.1.2 L2 Boot 组合

将分散的插件包按 Profile/Bundle 有序层组合为一棵可运行的插件树。读取 profile 配置 → 加载 bundle 列表 → 应用 patch overlay → 构建 Cordis 上下文。

3.2 主要组件

3.2.1 apps/cli (L1)

- 路径: `apps/cli/`
- 职责: `dsh` CLI 入口, 拥有 `dsh` bin。
- 核心: `apps/cli/src/bin.ts` (经 `node --import tsx/esm` 启动) ; `config/` (CLI 配置)、`reference/`、`tests/` 。
- 启动命令: `pnpm dsh web` / `pnpm dsh --profile headless "task"` 。

3.2.2 apps/web (L1)

- 路径: `apps/web/`
- 职责: Web UI 前端, Vite 构建。
- 核心: `index.html` 入口、`vite.config.ts`、`src/`、`stress-tests/`、`tests/` 。
- 默认端口: `http://127.0.0.1:3080` 。

3.2.3 boot/app-boot (L2)

- 路径: packages/boot/app-boot/
- 职责: Profile/Bundle 层组合核心。层应用顺序: 每个 bundle (profile 列出顺序) → profile cordis.patch.yml → home 级 cordis.patch.yml → --patch overlay。
- 配置查看: dsh --profile <name> --dump-config 。

3.2.4 boot/cmdline (L2)

- 路径: packages/boot/cmdline/
- 职责: 命令行参数解析, 分发到对应 profile 启动。

3.2.5 bundle/* (L2)

- 路径: packages/bundle/
- 子包:
 - base : 每个 profile 的第一层——模型适配器、工具、持久化、沙箱、审批策略、设置、凭据、遥测。
 - web-app : 添加浏览器应用层。
 - headless : 添加无服务器一次性运行层。
- 声明方式: package.json 的 dsh.bundle 字段指向补丁文件; dsh.profile 列出 profile 的 bundle。

3.3 对外接口

接口	参数	说明
dsh web	—	启动 Web UI
dsh --profile <name>	profile 名	按指定 profile 启动
dsh --profile <name> --dump-config	—	打印插件树不启动
dsh --profile headless "task"	任务文本	一次性 headless 运行
dsh --patch <file>	补丁文件	叠加 overlay

3.4 与其他层的交互

- 上层依赖: 无 (最顶层) 。
- 下层调用:
 - L1 → L2: CLI 解析后调用 app-boot 组合插件树。
 - L2 → L3: 组合完成后启动 Host (Web 模式) 或直接驱动 AgentLoop (headless 模式) 。
 - L2 → L4/L5/L6: bundle 引入 Core、能力接缝、持久化插件。

3.5 关键代码路径

apps/cli/src/bin.ts	# dsh bin 入口
apps/web/index.html	# Web UI HTML 入口
packages/boot/app-boot/src/	# Profile/Bundle 组合
packages/boot/cmdline/src/	# 命令行解析
packages/bundle/base/	# dsh-base 第一层
packages/bundle/web-app/	# Web 应用层
packages/bundle/headless/	# Headless 层

3.6 技术实现

- **tsx ESM-only hook**: CLI 源码启动经 `node --import tsx/esm`，所达模块必须保持 ESM。
- **层组合是 patchable 的**: 每个 bundle 插入的行可被上层 patch 替换（按 id 替换整行 config 或插入新行）。
- **Pre-release 自由重构**: 无外部消费者，profile/bundle 可自由重命名并同步更新引用。

4 L3 Host / Client 双半层

参照 [系统组件架构图](#)。

4.1 层职责

Web-GUI 拆为 host (Node 服务端) 与 client (浏览器侧) 两半，各自独立打包。Host 提供 API 网关与 HTTP 路由；Client 提供浏览器 shell、wire 协议、object services 与大量 `ui-*` 插件。双半通过 tsdown 的 host/client 双 face 构建匹配。

4.2 主要组件

4.2.1 host/ (服务端半)

- **路径**: `packages/host/`
- **子包** (8 个) :
 - `apiproxy` : API 代理。
 - `webserver` : HTTP 路由服务器。
 - `frontend-static` : 静态前端资源服务。
 - `plugin-inventory` : 插件清单管理。
 - `directory-picker` / `directory-picker-auto` / `directory-picker-browse` / `directory-picker-native` : 目录选择器 (多实现)。

4.2.2 client/ (浏览器半)

- **路径**: `packages/client/`
- **子包** (约 30 个) :
 - **核心运行时**: `connection` (连接管理)、`runtime` (浏览器运行时)、`hmr` (热替换)、`locale` (国际化)、`web`、`web-react`。
 - **基础设施**: `modules`、`schema-form`、`ui-slots`、`ui-primitives`、`ui-layout`、`ui-sidebar`、`ui-theme`。
 - **会话交互**: `ui-conversation` (对话)、`ui-input-trigger`、`ui-message-feedback`、`ui-trajectory`、`ui-user-questions`。

- 功能面板: `ui-tool`、`ui-plan`、`ui-goal`、`ui-jobs`、`ui-skill`、`ui-subagent`、`ui-workflow-run`、`ui-deliverables`。
- 设置: `ui-settings`、`ui-settings-general`、`ui-settings-models`、`ui-settings-plugins`、`ui-settings-plugin-inventory`、`ui-permission-presets`、`ui-model-selection`、`ui-agent-preset`。
- 其他: `ui-attachment`、`ui-commands`、`ui-workspace`、`ui-directory-picker-browse`、`ui-directory-picker-native`。

4.3 对外接口

接口	说明
HTTP 路由 (host/webserver)	Web UI 与 API 的 HTTP 端点
API 代理 (host/apiproxy)	代理前端请求到后端服务
ConversationNodeDefinition (client)	注册 Web Client Chat 节点 + keyed renderer
<code>ui-*</code> 插件槽 (client)	浏览器侧可扩展 UI 插件接口

4.4 与其他层的交互

- 上层依赖: L1 (CLI/Web 启动入口) + L2 (Boot 组合引入 web-app bundle)。
- 下层调用:
 - host → L7 (api/gateway、typert) : API 网关经 Typert RPC。
 - host → L4 (core/session) : 读取会话事件渲染。
 - client → L7 (sdk) : 浏览器侧经 JSON-RPC SDK 通信。
 - client → L4 (ctx.agents) : 驱动 agent 并从 `session/event` 渲染。

4.5 关键代码路径

```
packages/host/apiproxy/src/
packages/host/webserver/src/
packages/host/frontend-static/src/
packages/client/connection/src/
packages/client/runtime/src/
packages/client/ui-conversation/src/
packages/client/ui-settings/src/
packages/client/web-react/
packages/client/tsdown.client.ts    # client face 构建配置
```

4.6 技术实现

- 双 face 构建: `tsdown --env.DSH_BUILD_FACE host` 与 `client` 分别打包, 匹配 Host/Client 双半。
- Vite 前端: `apps/web` 用 Vite 构建, `vite-tsconfig-paths` 解析路径。
- HMR: `client/hmr` 支持开发时热替换。
- 扩展点: 新增 Web Client Chat 节点 = 注册 `ConversationNodeDefinition` + keyed renderer。

4.7 注意事项

- Host 与 Client 分属不同 tsconfig (`tsconfig.host.json` / `tsconfig.client.json`) , 构建顺序为先 host 后 client (host 的类型契约需先就绪)。

- `contracts-ready` 模式: typecheck/lint/doc-typecheck 均要求先 `build:lib:host` 再跑 client gate, 因为类型契约跨包生成 (declaration merging、`SessionEventMap`) 。

5 L4 Core API 主干层

参照 [系统组件架构图](#)。本层是产品 API 脊柱, 提供稳定 API。所有符号行号均经 AST 工具验证。

5.1 层职责

提供 Agent Harness 的核心 API: 会话事件日志、Agent 接口与注册表、默认驱动循环、工具注册表与执行管线、系统提示装配、per-agent 作用域注册。本层是「一切皆插件」架构的中枢——所有能力接缝与扩展都挂载到本层提供的 `ctx` key 上。

5.2 主要组件

5.2.1 core/session (会话事件日志)

- 路径: `packages/core/session/`
- ctx key: `ctx.sessions`
- 职责: 追加只写的 `SessionEvent` 日志 + 内存存储; 派生模型历史; 会话分叉。
- 核心符号:
 - `Session` 类 (`index.ts` L424-757): 会话实例, `append` 方法 (L603-654) 在源头用 `snapshotJsonValue` 拒绝非可序列化数据。
 - `SessionStore` 类 (`index.ts` L791-1154): 会话存储, 含 `fork` (分叉)。
 - `SessionEventMap` 接口 (`types.ts` L235-332): 声明合并扩展的事件类型体系, 13 个事件类型分四类 (生命周期/模型可见表面/仅日志/种子标记)。
 - `SessionHeader` 接口 (`types.ts` L60-98): 会话头, 含 `parentSession` (分叉谱系)。
 - `SessionEvent` (`types.ts` L403-435): 事件联合类型。
- 不变式: 模型可见即已记录——到达模型请求的任何内容必须可从日志重建。

5.2.2 core/agent (Agent 接口与注册表)

- 路径: `packages/core/agent/`
- ctx key: `ctx.agents`
- 职责: `Agent` 接口、实时注册表、`agent/*` 事件。
- 事件域: `agent/inbox/*` (队列)、`agent/status` (状态)、`agent/pre-step` (瀑布)、`agent/request` (瀑布)、`agent/request-error` (瀑布)、`agent/turn-stopping` (串行终端, 无 `next()`) 。

5.2.3 core/agent-loop (默认驱动)

- 路径: `packages/core/agent-loop/`
- ctx key: `ctx.agentLoop`
- 职责: 实现 `Agent` 接口的默认驱动, 编排 Turn/Step 生命周期。
- 核心符号:
 - `AgentLoop` 类 (`index.ts` L295-710): 默认驱动实现。
 - `createAgent` 方法 (L605-621): 创建 agent。
 - `ReactLoopAgent` 类 (`agent.ts` L63-495): 反应式循环 agent。
 - `AgentLoopSettings` 接口 (L243-246)、`PreparedAgent` 接口 (L149-157) 。

- **Turn/Step 流程:** turn/start → claim → agent/pre-step → step/start → agent/request → llm/stream → assistant/* → tool/call* → tools/* → tool/result* → step/end → agent/turn-stopping → turn/end 。

5.2.4 core/tools (工具注册表与执行管线)

- **路径:** packages/core/tools/
- **ctx key:** ctx.tools
- **职责:** 作用域工具注册表 + 受保护的执行管线 (pre/concurrent execute/post 三阶段) 。
- **核心符号:**
 - ToolRuntime 类 (index.ts L786-1862) : 工具运行时, 含 dispatchToolBody 方法 (L1531-1559) 。
 - ToolDefinition 接口 (L221-287) : 工具定义 schema。
 - ToolLayer 类 (L713-753) : 分层工具管理。
 - defineTool 函数 (schema.ts L544-616) : 工具定义工厂。
 - DefineToolOptions 接口 (schema.ts L482-535) 。
 - fuseToolSignals 函数 (L1888-1915) : 工具信号融合。
 - ToolExecutionInput (L313-337)、ToolResult (L290-301)、ToolExecutionMode (L343-345) 。
- **执行模式:** barrier 与有界滚动池控制并发; pre/post 有序, execute 并发。

5.2.5 core/system-prompt (提示装配)

- **路径:** packages/core/system-prompt/
- **ctx key:** ctx.systemPrompt
- **职责:** Prompt section 与 tool schema 装配。
- **流程:** system-prompt/assemble waterfall——插件注册 prompt section, 驱动装配时合并。

5.2.6 core/scope (per-agent 作用域)

- **路径:** packages/core/scope/
- **ctx key:** 无 (库)
- **职责:** per-agent 的作用域注册原语。用某 agent 的 agent.ctx 将注册限定到该 agent。

5.2.7 core/agent-default-model / core/agent-tool-presentation

- **路径:** packages/core/agent-default-model/、packages/core/agent-tool-presentation/
- **职责:** 默认模型绑定、工具呈现模式 (ToolPresentationMode 在 tools/src/index.ts L650) 。

5.3 对外接口

ctx key	类型	说明
ctx.sessions	SessionStore	会话创建、追加、分叉、派生
ctx.agents	Agent 注册表	Agent 实时注册与 agent/* 事件
ctx.agentLoop	AgentLoop	默认驱动
ctx.tools	ToolRuntime	工具注册与受保护执行
ctx.systemPrompt	—	Prompt section 装配

5.4 与其他层的交互

- **上层依赖**: L2 (Boot 引入)、L3 (Host/Client 驱动与渲染)。
- **下层调用**:
 - agent-loop → `ctx.llm` (L5): 模型请求与流。
 - agent-loop → `ctx.tools` (本层): 工具调度。
 - agent-loop → `ctx.systemPrompt` (本层): prompt 装配。
 - agent-loop → `ctx.sessions` (本层): 事件追加。
 - tools → Consumer 工具 (L5): shell/fs/web/skill 等。
 - sessions → L6 (持久化): 事件日志持久化。

5.5 关键代码路径

```
packages/core/session/src/index.ts      # Session, SessionStore
packages/core/session/src/types.ts      # SessionEventMap, SessionHeader
packages/core/session/src/invariant.ts  # 会话不变式
packages/core/agent-loop/src/index.ts   # AgentLoop
packages/core/agent-loop/src/agent.ts   # ReactLoopAgent
packages/core/tools/src/index.ts         # ToolRuntime, ToolLayer
packages/core/tools/src/schema.ts        # defineTool, DefineToolOptions
packages/core/system-prompt/src/
packages/core/scope/src/
```

5.6 技术实现

- **事件溯源**: 会话状态是 `SessionEvent` 追加只写日志的派生物。`deriveMessages()` 从日志派生模型历史; raw `assistant/chunk` 保留回放与 UI 保真。
- **Waterfall 事件**: `agent/pre-step`、`agent/request`、`llm/stream`、`tools/pre-execute`、`tools/execute`、`tools/post-execute` 是瀑布——监听者须调 `next()` 委托。
- **串行终端**: `agent/turn-stopping` 无 `next()`, 是回合结束的权威检查点。
- **声明合并**: `SessionEventMap` 用 TypeScript declaration merging 扩展, 新事件类型由各包追加。JSDoc 需 `@mode` 与 payload `@param`; scoped keys 缺失需 `@dshScopeScan unsupported`。
- **可替换性**: `dsh-agent-loop` 可被替换; UI/hook/工具插件依赖 `dsh-agent` (接口) 而非具体 loop。

5.7 注意事项

- 本层是稳定 API 承诺的核心——改动需谨慎。
- `dsh-agent-loop` 是 swappable 的; 扩展插件应依赖 Service Definition (`dsh-agent`) 而非具体 Provider。
- 会话日志的不变式 (模型可见即已记录) 由运行时断言强制——新增模型可见输入必须扩展 `SessionEventMap` 并从日志渲染。

6 L5 能力接缝层

参照 [系统组件架构图](#) 与 [能力接缝文档](#)。

6.1 层职责

提供 Agent 可调用的全部能力，以**能力接缝（Capability Seams）**模式组织。每个接缝由三角色构成：- **Service Definition**：零依赖接口声明（声明能力存在）。- **Service Provider**：实现接口（绑定具体后端，如 local / sandbox / e2b）。- **Consumer**：使用能力（通常是面向模型的工具，注册到 `ctx.tools`）。

换一个 Provider 即可迁移整个执行世界（如本地→远程沙箱），无需 Consumer 分叉。

6.2 主要组件

6.2.1 执行环境能力接缝

接缝	包	ctx key	Provider	说明
Bash 执行	shell/	ctx.shell	local、pwsh	经 ctx.subprocess 生成进程
子进程	subprocess/	ctx.subprocess	本地进程树	进程树管理
持久终端	terminal/	ctx.terminals	local PTY	owner-scoped 会话，node-pty (ConPTY)
代码执行	code-runtime/	—	worker-thread	Code Mode Consumer
进程限制	sandbox/	ctx.sandbox	bwrap / Landlock / Seatbelt / Windows ACL	限制生成进程能力
文件系统	fs/	ctx.fs	local	文件读写 + 策略 + bash-backed 发现工具
LSP	lsp/	—	generic stdio	语言服务器协议
远程沙箱	e2b/	—	E2B 云	POC

6.2.2 模型协作能力接缝

接缝	包	ctx key	说明
LLM 适配	llm/	ctx.llm	消息/流词汇 + 适配器接缝; DeepSeek + pi-ai
Skill	skill/	—	provider 注册表 + 本地实现 + catalog/loader 工具
Web	web/	—	search/fetch provider + 工具
上下文压缩	compaction/	—	basic provider + 命令 Consumer
请求上下文	context/	—	workspace 指令、时间上下文
子 agent	subagent/	—	provider 注册表 + 委派工具
后台任务	jobs/	ctx.jobs	job_* 控制工具
工作流	workflow/	—	worker-thread 引擎 + ralph 工具
Todo	todo/	—	todo_write 工具
计划	plan/	—	计划协作状态（直接进入 + 审核 退出）
预设组合	preset/	—	预设 cordis.yml 的 per-session agent 组合
循环守卫	guard/	—	重复调用提醒 + 工具超时强制器
运行时自修改	extensions/	—	实时插件/服务检查 + 模型挂载/卸 载插件
目标	goal/	ctx.goals	同会话目标持久化
定时	schedule/	—	会话本地定时跟进
反馈	feedback/	—	人类反馈
身份	identity/	—	匿名身份

6.3 对外接口

能力接缝通过 ctx.* key 暴露。Consumer 工具通过 ctx.tools 注册，其 schema 自动加入 prompt 装配。

能力	注册方式
面向模型的工具	<code>ctx.tools</code> 注册; schema 加入 prompt
给某会话不同能力集	组合 agent preset; service 行需 <code>isolate</code> realm
添加 shell 执行	注册 <code>ctx.shell</code> 后端
添加持久终端	注册 <code>ctx.terminals</code> 后端 + <code>dsh-tool-terminal</code>
添加文件系统策略	注册 <code>ctx.fs</code> provider 或监听 <code>fs/*</code> 事件
限制生成进程	<code>ctx.sandbox</code> 后端; consumer 在 spawn 前包装 argv

6.4 与其他层的交互

- **上层依赖**: L4 (Core API) ——能力 Consumer 通过 `ctx.tools` 注册, Provider 通过 `ctx.*` 接缝挂载。
- **下层调用**:
 - 执行环境接缝 → 外部系统: 本地 FS/PTY、E2B 云、DeepSeek API。
 - shell/subprocess → sandbox: 生成进程前经 sandbox 包装 argv。
 - terminal → subprocess: PTY 经 subprocess 生成。
 - code-runtime → sandbox: 代码执行经 sandbox 限制。
- **横向**: 能力接缝之间通过 `ctx.tools` 协作 (如 fs 工具与 shell 工具共享执行世界) 。

6.5 关键代码路径

<code>packages/llm/llm/src/</code>	<code># LLM 接缝 (ctx.llm)</code>
<code>packages/llm/llm-deepseek/src/</code>	<code># DeepSeek 适配器</code>
<code>packages/shell/</code>	<code># Bash 能力 (ctx.shell)</code>
<code>packages/subprocess/</code>	<code># 子进程 (ctx.subprocess)</code>
<code>packages/terminal/</code>	<code># 持久终端 (ctx.terminals)</code>
<code>packages/sandbox/</code>	<code># 进程限制 (ctx.sandbox)</code>
<code>packages/fs/</code>	<code># 文件系统 (ctx.fs)</code>
<code>packages/lsp/</code>	<code># LSP</code>
<code>packages/skill/</code>	<code># Skill</code>
<code>packages/web/</code>	<code># Web search/fetch</code>
<code>packages/compaction/</code>	<code># 上下文压缩</code>
<code>packages/subagent/</code>	<code># 子 agent</code>
<code>packages/jobs/</code>	<code># 后台任务 (ctx.jobs)</code>
<code>packages/workflow/</code>	<code># 工作流</code>
<code>packages/extensions/</code>	<code># 运行时自修改</code>

6.6 技术实现

- **三角色解耦**: Service Definition (接口) 零依赖; Provider 绑定 SDK; Consumer 注册到 `ctx.tools` 。这使 provider 可替换迁移整个执行世界。
- **共享执行世界**: fs + subprocess + sandbox + shell 共享一个执行世界。指向远程沙箱即可迁移 Bash、PTY、LSP, 无 provider 分叉。
- **isolate realm**: 给某会话不同能力集时, preset 的 service 行需 `isolate` realm 隔离。
- **waterfall 策略注入**: `tools/pre-execute` waterfall 注入审批/策略; `fs/*`、`tools/*`、`telemetry/*` 事件附加策略而不导入 loop。
- **模型可见上下文**: `agent.inject()` 将上下文落入下次 admitted 请求; 注入的上下文在 inbox 中等待直至另一消息到达。

6.7 注意事项

- 扩展插件应依赖 Service Definition (如 `dsh-llm`)，而非具体 Provider (如 `dsh-llm-deepseek`)。
- 能力接缝三角色必须同时设计——一个角色单独不构成接缝。
- 添加新能力需同时考虑：接口声明、默认 provider、模型可见工具。
- `extensions/` 允许 agent 在运行时检查并挂载/卸载自己的插件 (自指 cordis toolset)。

7 L6 持久化与检索层

参照 [系统组件架构图](#)。详细数据模型见 [数据架构文档](#)。

7.1 层职责

提供会话日志的持久化、检索、投影，以及非会话存储（设置、凭据、附件、spill）。采用事件溯源 + 双后端（JSONL/SQLite）设计，pre-release 阶段无兼容性承诺。

7.2 主要组件

组件	路径	职责
会话持久化	<code>session/</code>	JSONL/SQLite 后端 + 投影 seam + 日志回溯标题 + 会话报告
会话检索	<code>session-query/</code>	逻辑语料、有界读、lineage、事件关系、语义过滤、SQLite FTS
非会话存储	<code>storage/</code>	存储 hub + 后端 + domain form
附件	<code>attachment/</code>	持久附件身份 + 内容寻址存储 (sha256)
Spill	<code>spill/</code>	存储接缝 + 本地实现 + 工具结果溢出策略
设置	<code>settings/</code>	用户设置 seam + 文件 provider
凭据	<code>credentials/</code>	凭据引用 seam + env-over- <code>.env</code> provider
Workspace	<code>workspace/</code>	Workspace 实体

7.3 对外接口

接口	说明
PersistenceBackend	JSONL/SQLite 共同实现的持久化接口 (coordinator.ts L127)
SessionCorpus	FTS5 检索语料 (corpus.ts L32)
SessionProjectionCache	投影缓存 (fail-soft, index.ts L71)
凭据解析	进程环境 > .credentials.yaml (0600) > cwd/.env > DSH_HOME/.env

7.4 与其他层的交互

- 上层依赖: L4 (ctx.sessions 追加事件 → 持久化) 。
- 下层调用: 外部存储介质 (SQLite 文件、JSONL 文件、本地 FS) 。
- 横向: session-query 从持久化日志派生 FTS 索引; attachment 为工具结果提供内容寻址存储。

7.5 关键代码路径

```
packages/session/                # 持久化后端 (JSONL/SQLite)
packages/session-query/src/      # FTS 检索
packages/storage/src/
packages/attachment/src/store.ts # 内容寻址存储 (sha256)
packages/spill/src/
packages/settings/src/
packages/credentials/src/
```

7.6 技术实现

- 事件溯源: 会话状态是 SessionEvent 日志的派生物; 持久化即追加事件到 JSONL/SQLite。
- 双后端等价语义: JSONL 与 SQLite 实现同一 PersistenceBackend 接口, 崩溃撕裂尾修复语义一致。
- 写透发布: POSIX 用 link() + 目录 fsync; Windows 用 MoveFileExW(MOVEFILE_WRITE_THROUGH) (koffi) 。
- 版本管理: SCHEMA_VERSION (SQLite 表布局)、SESSION_QUERY_SQLITE_SCHEMA_VERSION (FTS)、SESSION_FORMAT_VERSION (事件词汇表) 三版本正交, 不匹配即拒绝/丢弃重建。
- 内容寻址附件: 附件 ID = sha256:hex, 路径 objects/<前2位>/<sha256>, link() dedup。
- 凭据分层信任: 进程环境 (不可写, 胜出) > .credentials.yaml > cwd/.env > DSH_HOME/.env; CredentialRef → string 严格映射。

7.7 注意事项

- Pre-release 无迁移: 版本不匹配即丢弃重建, 只有 pristine 或当前自有版本可打开。
- 投影缓存是 fail-soft 的——失败仅 warning, 下次冷读从全日志重放自愈。
- 凭据永不入配置/日志/session event——apiKeyEnv 是 CredentialRef (branded 环境变量名) 。

8 L7 协议与集成层

参照 [系统组件架构图](#)。详细集成方案见 [第三方集成文档](#)。

8.1 层职责

提供进程外通信协议（Typert RPC、JSON-RPC、ACP）、hook 桥接（Claude Code/Codex）与人机协作面（审批、权限、命令、ask-user）。本层是 dsh 与外部世界（SDK 客户端、自动化系统、IDE hook、人类用户）的交互边界。

8.2 主要组件

组件	路径	职责
API 网关	<code>api/gateway</code> + <code>api/remotes</code>	远程 BFF 装配 + Typert RPC 网关
Typert	<code>typert/generator</code> + <code>loader</code> + <code>protocol</code> + <code>registry</code>	类型图生成、加载、运行时注册；跨语言类型契约
SDK	<code>sdk/</code>	进程外运行时 SDK：JSON-RPC 协议 + TS client + server 插件
ACP	<code>acp/</code>	仅自动化的 Agent Client Protocol server (@agentclientprotocol/sdk 0.25.1)
Hooks	<code>hooks/</code>	Claude Code/Codex hook 桥 + 共享 wire-protocol 库
交互	<code>interaction/</code>	审批/交互接缝、权限预设、命令、ask-user 工具

8.3 对外接口

接口	协议	说明
Typert RPC	Typert 类型图	跨语言（TS/Python）RPC 类型契约
JSON-RPC	JSON-RPC	dsh 自有完整客户端协议（含 subagent 一等支持）
ACP	Agent Client Protocol 0.25.1	automation-only，外部 trusted 客户端
Hook 桥	Claude Code/Codex wire-protocol	执行未修改的 hook 命令
<code>ctx.commands</code>	—	人类命令分发，无需模型 turn
审批/权限	<code>tools/pre-execute</code> waterfall	策略注入

8.4 与其他层的交互

- 上层依赖：L3（Host 经 api/gateway 暴露 RPC；Client 经 sdk 通信）。

- **下层调用**: L4 (api → core/session 读取会话; acp → agent-loop 驱动; hooks → agent-loop)。
- **横向**: interaction 的权限预设注入 `tools/pre-execute`; hooks 与 agent-loop 的 `agent/*` 事件桥接。

8.5 关键代码路径

```
packages/api/gateway/src/  
packages/api/remotes/src/  
packages/typert/generator/src/  
packages/typert/loader/src/  
packages/typert/protocol/src/  
packages/typert/registry/src/  
packages/sdk/server/src/server.ts    # HarnessSdkJsonRpcServer (L52)  
packages/acp/src/  
packages/hooks/  
packages/interaction/src/
```

8.6 技术实现

- **Typert 类型图**: generator 从 TS 类型生成类型图 artifact, loader 加载, registry 运行时注册。跨语言类型契约的基础。
- **SDK 与 ACP 平行**: ACP 是 automation-only 标准 (外部 trusted 客户端); SDK 是 dsh 自有完整客户端协议 (含 subagent 一等支持)。两者平行而非嵌套。
- **Hook bridge 兼容层**: 执行未修改的 Claude Code/Codex hook 命令; shared execution 在 `dsh-hook-protocol`, 每个 dialect bridge 只 own 自己的 payload/env/mapping。restrictive merge 保证一个 block 则整体 block。
- **contracts-ready 模式**: typecheck/lint/doc-typecheck 均要求先 `build:lib:host` ——因为类型契约跨包生成 (declaration merging, `SessionEventMap`)。
- **命令分发**: `ctx.commands` 注册的命令无需模型 turn 即分发。

8.7 注意事项

- ACP 是 automation-only, 不面向交互式人类用户。
- Hook 命令在 dsh 之外定义, bridge 只负责执行与映射, 不修改 hook 逻辑。
- Typert 类型图是 generated artifact——`pnpm run gen-cordis-api / verify-cordis-api` 维护新鲜度。
- `interaction/` 的权限预设与 `tools/pre-execute` 紧耦合——审批策略变更需同步考虑工具执行管线。

9 core/agent-loop — 默认驱动

9.1 模块概述

`core/agent-loop` 实现 `Agent` 接口的默认驱动, 编排 Turn/Step 生命周期。它是整个 Agent Harness 的「心脏」——驱动模型请求、工具执行、事件追加的循环。`ctx.agentLoop` 是其 ctx key。本模块是 **swappable** 的——扩展插件应依赖 `dsh-agent` (接口) 而非具体 loop。

9.2 主要功能

1. **Turn/Step 生命周期编排**: `prepare()` 准备步骤 → 模型请求 → 工具执行 → 步结束。
2. **Agent 创建与管理**: `createAgent()` 创建 agent, `create()` 工厂方法。

3. 会话恢复: `resume()` / `resumeWith()` 恢复会话继续运行。
4. 配置身份管理: `restoreOrCreateConfigured()` / `waitForDrainingConfiguredIdentity()`。
5. 反应式循环: `ReactLoopAgent` (`agent.ts` L63-495) 提供反应式驱动变体。

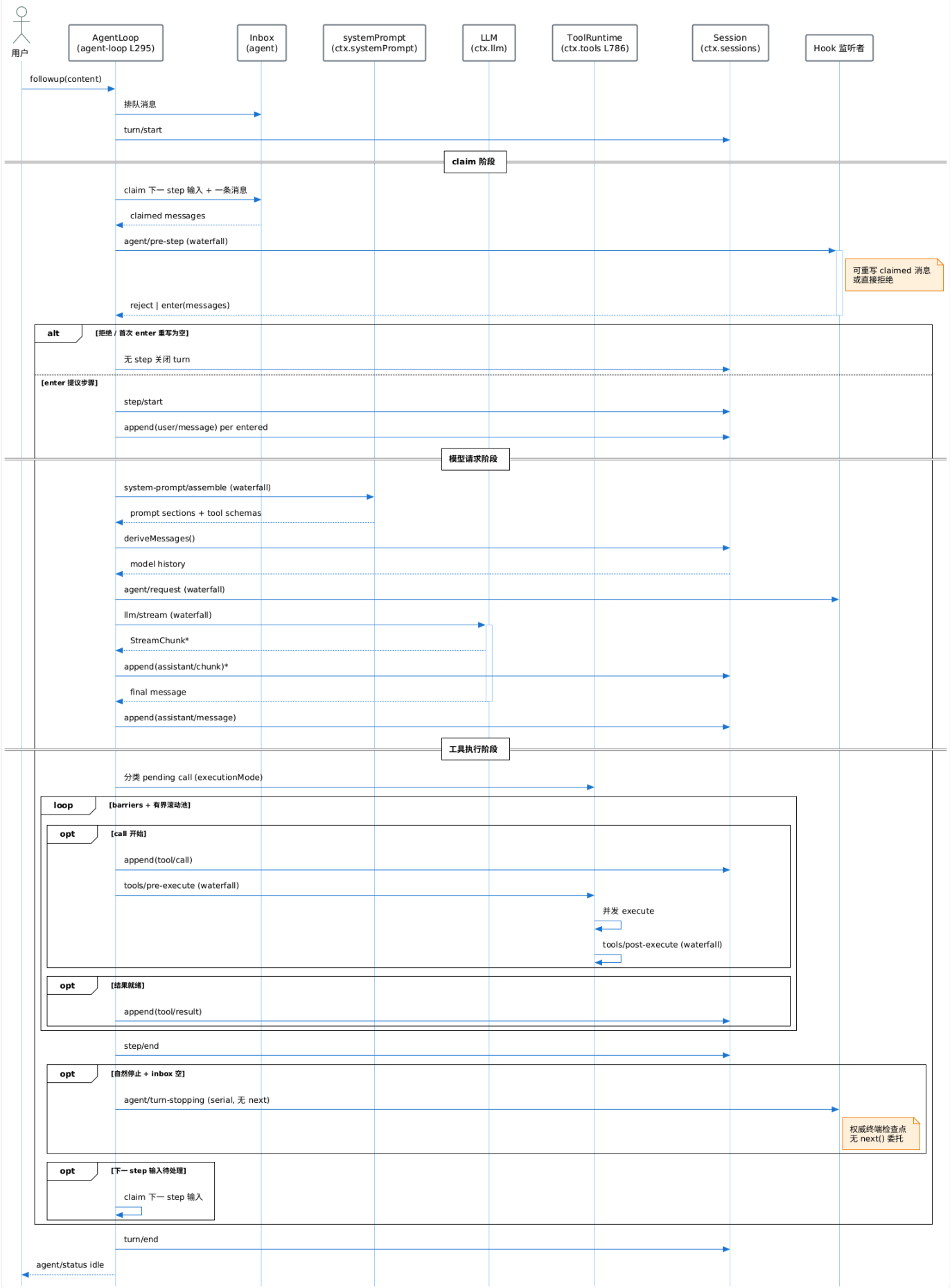
9.3 目录结构

```
packages/core/agent-loop/src/  
├─ index.ts          # AgentLoop 类 (L295-710), 配置, createAgent  
├─ agent.ts          # ReactLoopAgent 类 (L63-495)  
└─ (其他辅助模块)
```

9.4 核心流程

- `Turn/Step` 生命周期时序图 — 完整回合驱动流程

Turn/Step 生命周期 (AgentLoop 默认驱动)



 后处理步骤会在上方链接后自动插入 PNG 图片嵌入。

9.5 核心符号

符号	类型	位置	说明
<code>AgentLoop</code>	Class	<code>index.ts</code> L295-710	默认驱动实现
<code>AgentLoop/constructor</code>	Method	L318-381	构造
<code>AgentLoop/prepare</code>	Method	L458-577	准备步骤 (claim + pre-step)
<code>AgentLoop/create</code>	Method	L588-597	工厂
<code>AgentLoop/createAgent</code>	Method	L605-621	创建 agent
<code>AgentLoop/setupAndPublish</code>	Method	L624-644	设置并发布
<code>AgentLoop/resume</code>	Method	L652-658	恢复会话
<code>AgentLoop/resumeWith</code>	Method	L661-709	带输入恢复
<code>AgentLoop/restoreOrCreateConfigured</code>	Method	L406-427	恢复或创建配置身份
<code>AgentLoop/waitForDrainingConfiguredIdentity</code>	Method	L430-450	等待排空
<code>ReactLoopAgent</code>	Class	<code>agent.ts</code> L63-495	反应式循环变体
<code>AgentLoopSettings</code>	Interface	<code>index.ts</code> L243-246	设置
<code>PreparedAgent</code>	Interface	L149-157	已准备 agent
<code>ConfiguredAgentIdentities</code>	Interface	L202	配置身份

9.6 Turn/Step 生命周期

一个 **step** = 一次模型请求 + 其调用的工具。一个 **turn** = 零或多个 step。

```
turn/start
  claim 下一 step 输入 + 一条排队消息
  装配 prompt sections + tool schemas
  -> agent/pre-step (waterfall)      拒绝 | enter(messages)
step/start
  追加 entered messages 为 user/message
  从日志派生模型历史
  agent/request -> llm/stream -> assistant/chunk* -> assistant/message
  tool/call* -> tools/pre-execute -> tools/execute -> tools/post-execute -> tool/result*
step/end
  -> agent/turn-stopping (serial, 无 next)
turn/end
```

9.7 事件域

事件	类型	说明
agent/inbox/*	实时	队列管理（spliced/inserted/claimed）
agent/status	实时	状态变更（running/idle）
agent/pre-step	waterfall	步前拦截，可重写/拒绝消息
agent/request	waterfall	请求构造
agent/request-error	waterfall	请求错误处理（重试/保留）
agent/turn-stopping	serial（无 next）	回合结束权威检查点

9.8 技术栈

技术	用途
TypeScript ESM	实现
Cordis	waterfall / serial 事件
事件溯源	状态从 SessionEvent 派生

9.9 依赖关系

9.9.1 依赖模块

- core/session（ctx.sessions 追加事件）
- core/tools（ctx.tools 工具调度）
- core/system-prompt（ctx.systemPrompt prompt 装配）
- llm/llm（ctx.llm 模型请求与流）
- @deepseek-ai/cordis

9.9.2 被依赖模块

- bundle/headless（headless 运行驱动）
- acp/（ACP server 驱动）
- hooks/（hook 桥接 agent 事件）

9.10 相关文档

- 系统架构 - Turn 流程
 - Agent 生命周期
-

10 core/session — 会话事件日志

10.1 模块概述

`core/session` 是 DeepSeek Harness 的产品 API 脊柱之一，提供追加只写的 `SessionEvent` 日志与内存存储。采用事件溯源（Event Sourcing）模式：会话状态是事件日志的派生物，所有模型可见内容必须可从日志重建。`ctx.sessions` 是其 ctx key。

10.2 主要功能

- 1. 事件追加与日志: `Session.append()` 在源头用 `snapshotJsonValue` 拒绝非可序列化数据，保证耐久日志回放相同事件。
- 2. 模型历史派生: `deriveMessages()` 从事件流派生模型可见历史；raw `assistant/chunk` 保留回放与 UI 保真。
- 3. 会话分叉: `SessionStore.fork(source, boundary?, childSessionId?)` 从事件流分叉新会话。
- 4. 会话恢复: 从持久化日志恢复会话状态。
- 5. 事件类型体系: `SessionEventMap` 声明合并扩展，13 个事件类型分四类。

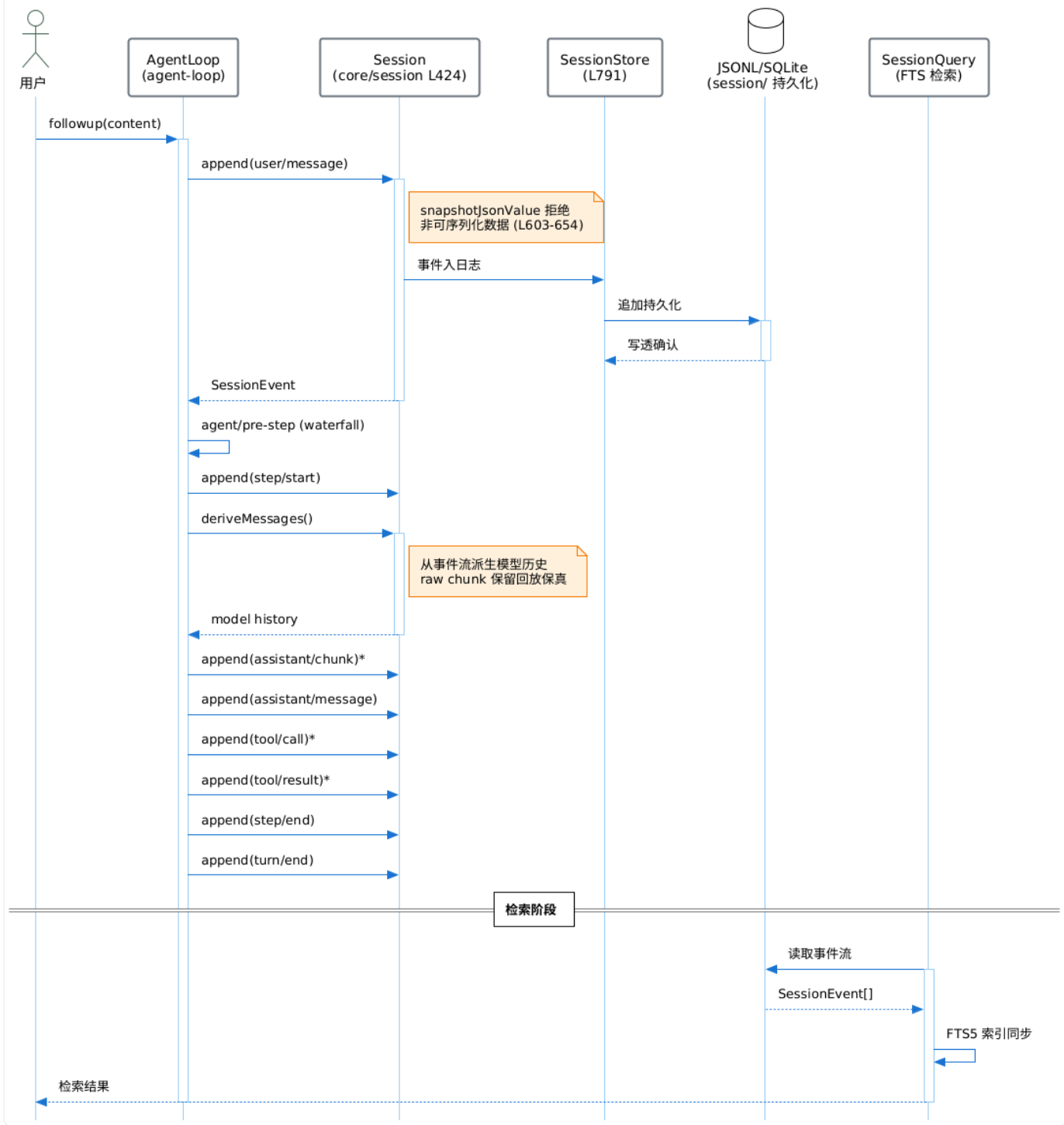
10.3 目录结构

```
packages/core/session/src/
├─ index.ts           # Session 类 (L424-757), SessionStore 类 (L791-1154)
├─ types.ts           # SessionEventMap (L235-332), SessionHeader (L60-98), SessionEvent (L403-435)
├─ invariant.ts        # 会话不变式, SessionTrace
├─ preparation.ts      # SessionPreparation
├─ surface.ts          # SessionSurface
└─ py-types.ts         # Python SDK 类型
```

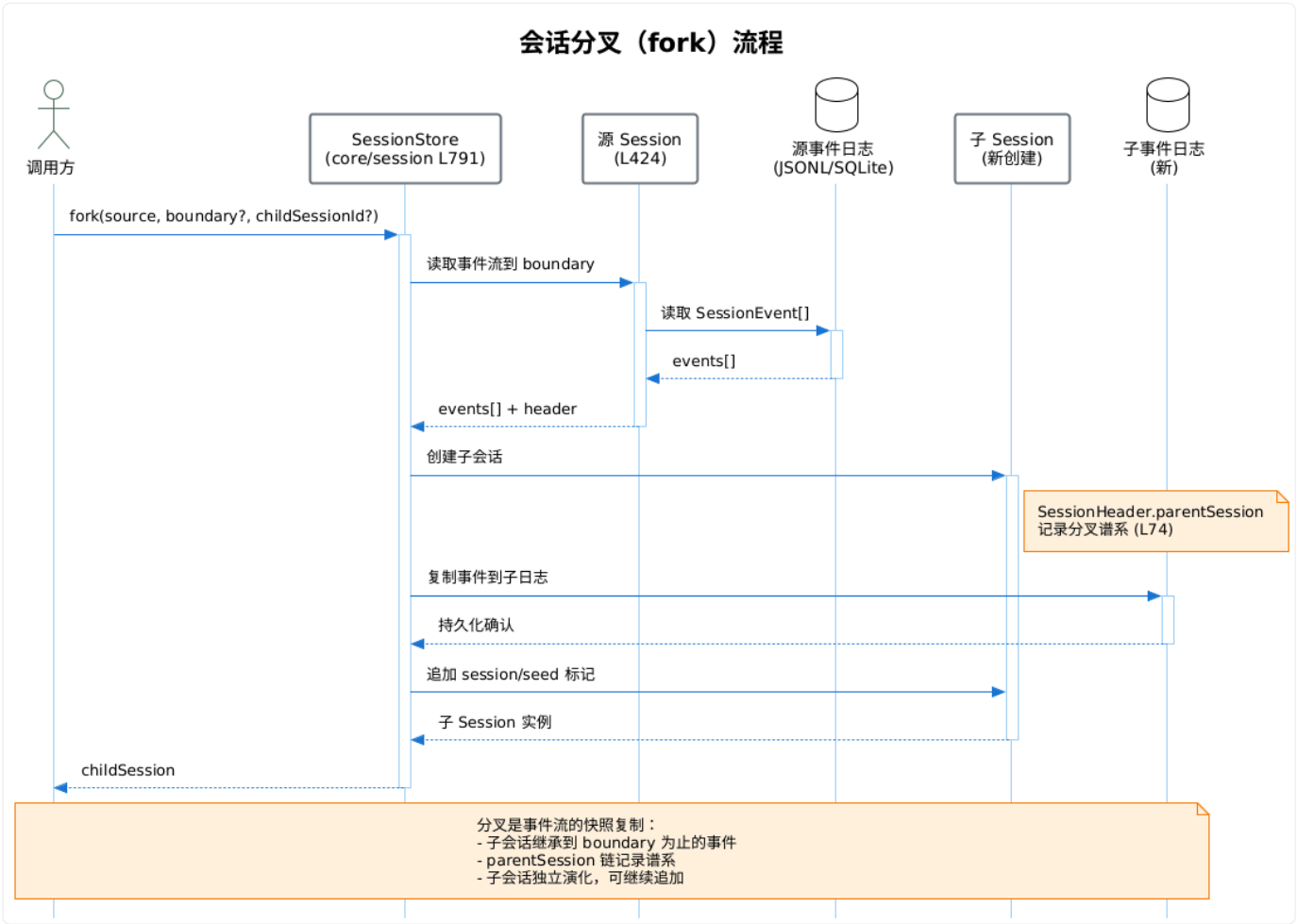
10.4 核心流程

- 事件追加与派生时序图 — 事件追加 → 派生 → 持久化 → 检索
- 会话分叉时序图 — fork 流程

会话事件溯源：追加 → 派生 → 持久化



事件追加与派生时序图



会话分叉时序图

i 后处理步骤会在上方链接后自动插入 PNG 图片嵌入。

10.5 核心符号

符号	类型	位置	说明
<code>Session</code>	Class	<code>index.ts</code> L424-757	会话实例, <code>append</code> (L603-654)
<code>SessionStore</code>	Class	<code>index.ts</code> L791-1154	会话存储, 含 <code>fork</code>
<code>SessionEventMap</code>	Interface	<code>types.ts</code> L235-332	事件类型体系 (声明合并扩展)
<code>SessionHeader</code>	Interface	<code>types.ts</code> L60-98	会话头, 含 <code>parentSession</code>
<code>SessionEvent</code>	Variable	<code>types.ts</code> L403-435	事件联合类型
<code>CreateSessionOptions</code>	Interface	<code>types.ts</code> L105-121	创建会话选项
<code>SessionForkError</code>	Class	<code>index.ts</code> L778-783	分叉错误
<code>validateSessionHeader</code>	Function	<code>index.ts</code> L95-135	头校验

10.6 事件类型体系 (SessionEventMap)

类别	事件	持久	说明
生命周期	<code>turn/start</code> 、 <code>turn/end</code> 、 <code>step/start</code> 、 <code>step/end</code>	✓	Turn/Step 边界
模型可见表面	<code>user/message</code> 、 <code>assistant/chunk</code> 、 <code>assistant/message</code> 、 <code>tool/call</code> 、 <code>tool/result</code>	✓	模型请求可见内容
仅日志	<code>session/seed</code> 等	✓	不影响模型历史
种子标记	header/version 标记	✓	会话元数据

完整 13 类事件见 `types.ts` L235-332 (AST 验证)。

10.7 技术栈

技术	用途
TypeScript ESM	实现
Cordis	插件上下文 (<code>ctx.sessions</code>)
事件溯源	状态派生自事件日志

10.8 依赖关系

10.8.1 依赖模块

- `@deepseek-ai/dsh-brand`、`@deepseek-ai/dsh-invariants`、`@deepseek-ai/dsh-llm`、`@deepseek-ai/dsh-scope`、`@deepseek-ai/dsh-typert-protocol`、`@deepseek-ai/cordis`

10.8.2 被依赖模块

- `core/agent-loop` (追加事件)、`session/` (持久化)、`session-query/` (检索)、`api/` (渲染)、`host/`、`client/`

10.9 相关文档

- 事件溯源功能说明
- 数据架构

11 core/tools — 工具注册表与执行管线

11.1 模块概述

`core/tools` 提供作用域工具注册表与受保护的执行管线。它管理工具定义注册、schema 装配、三阶段调度（pre/concurrent execute/post）、barrier 与有界滚动池并发控制、工具结果融合。`ctx.tools` 是其 ctx key。所有面向模型的能力（shell、fs、web、skill 等）都作为 Consumer 工具注册到本模块。

11.2 主要功能

- 1. 工具注册: `register()` 注册 `ToolDefinition`，其 schema 加入 prompt 装配。
- 2. 分层管理: `ToolLayer` 管理分层工具，支持 collapse/restrict。
- 3. 三阶段执行管线: pre-execute（waterfall 策略/审批）→ concurrent execute → post-execute（waterfall）。
- 4. 并发控制: barrier 与有界滚动池（`maxParallelSubCalls`）控制并发度。
- 5. 执行模式: `executionMode` / `modeFor` 分类工具调用（同步/异步/延迟）。
- 6. 结果融合: `fuseToolSignals` 融合多工具信号。
- 7. 取消与守卫: `callerCancelled` / `cancellationResult` / `guard` / `guardReason`。
- 8. Code Mode: `createRunCodeTool`（`code-mode.ts` L293-672）代码执行工具。
- 9. SDK 类型生成: `renderToolsSdk` / `renderToolsSdkPy` 生成 TS/Python SDK schema。

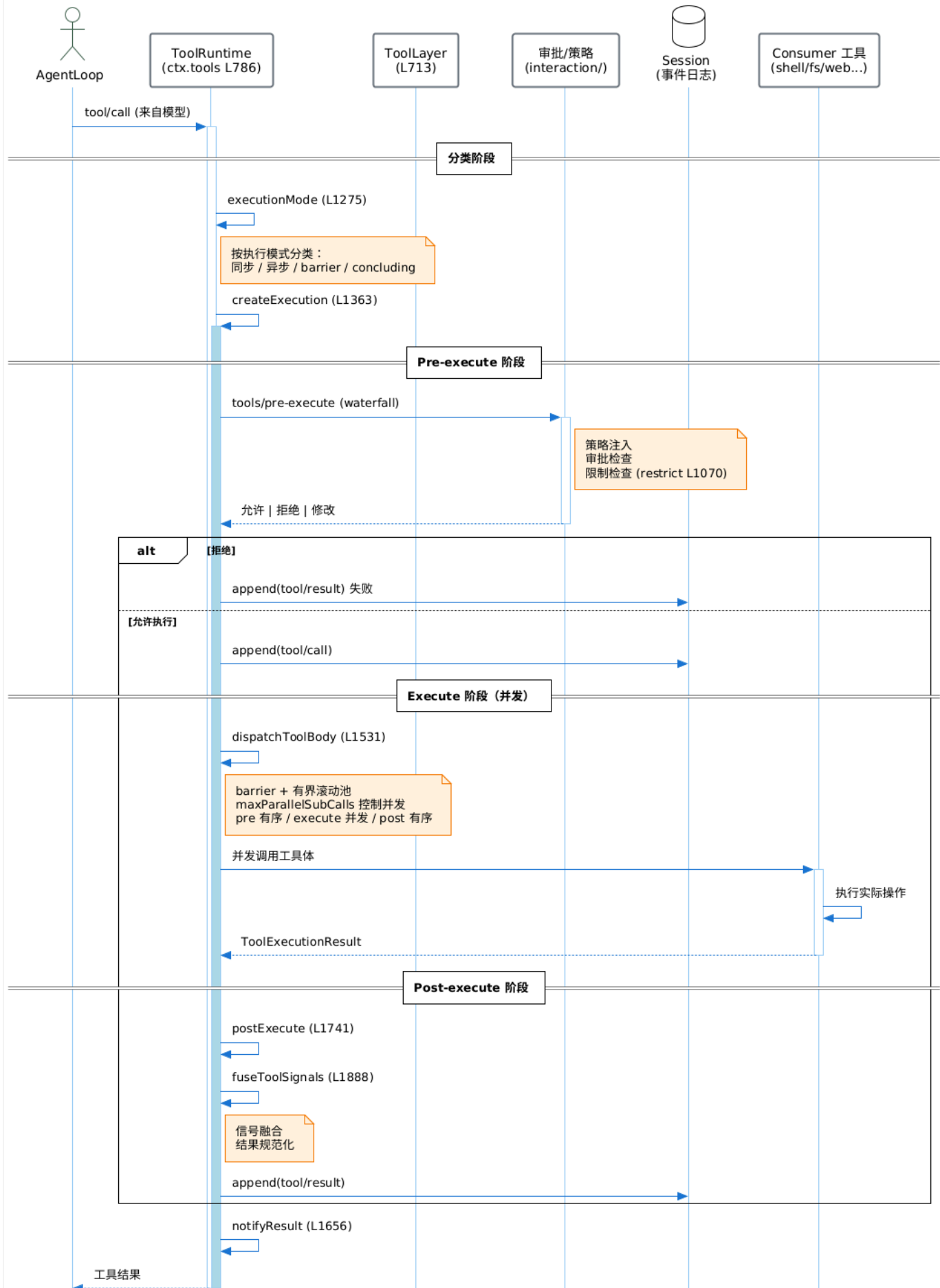
11.3 目录结构

```
packages/core/tools/src/
├─ index.ts           # ToolRuntime 类 (L786-1862), ToolLayer (L713-753), ToolDefinition (L221-287)
├─ schema.ts         # defineTool (L544-616), DefineToolOptions (L482-535)
├─ code-mode.ts      # createRunCodeTool (L293-672)
├─ presentation.ts   # ToolCallKind, ToolCallView, ToolResultView
├─ ts-types.ts       # renderToolsSdk, ToolSdkSchema
├─ py-types.ts       # renderToolsSdkPy
├─ testing.ts        # 测试工具
└─ invariant.ts      # ToolStage
```

11.4 核心流程

- 工具执行管线时序图 — 三阶段调度流程

工具执行管线：三阶段调度



- 三阶段管线特点：
1. pre-execute 是 waterfall（策略/审批可拦截）
 2. execute 并发，受 barrier 与滚动池控制
 3. post-execute 是 waterfall（结果后处理）
 4. 所有 tool/call 与 tool/result 持久到会话日志

工具执行管线时序图

 后处理步骤会在上方链接后自动插入 PNG 图片嵌入。

11.5 核心符号

符号	类型	位置	说明
ToolRuntime	Class	index.ts L786-1862	工具运行时核心
ToolRuntime/register	Method	L1036-1061	工具注册
ToolRuntime/restrict	Method	L1070-1097	工具限制
ToolRuntime/guard	Method	L1109-1115	守卫
ToolRuntime/view	Method	L1151-1192	视图
ToolRuntime/schemas	Method	L1233-1235	schema 列表
ToolRuntime/schemaOf	Method	L1255-1266	单工具 schema
ToolRuntime/executionMode	Method	L1275-1284	执行模式分类
ToolRuntime/createExecution	Method	L1363-1450	创建执行
ToolRuntime/prepareExecution	Method	L1462-1506	准备执行
ToolRuntime/dispatchToolBody	Method	L1531-1559	派发工具体
ToolRuntime/dispatchScheduledExecution	Method	L1568-1598	派发调度执行
ToolRuntime/postExecute	Method	L1741-1780	后执行
ToolRuntime/serviceAsk	Method	L1688-1728	服务询问 (ask-user)
ToolRuntime/notifyResult	Method	L1656-1675	通知结果
ToolRuntime/callerCancelled	Method	L1509-1514	调用方取消
ToolDefinition	Interface	L221-287	工具定义 schema
ToolLayer	Class	L713-753	分层工具管理
ToolExecutionInput	Interface	L313-337	执行输入
ToolResult	Interface	L290-301	工具结果
ToolExecutionMode	Variable	L343-345	执行模式
defineTool	Function	schema.ts L544-616	工具定义工厂
fuseToolSignals	Function	L1888-1915	信号融合
createRunCodeTool	Function	code-mode.ts L293-672	Code Mode 工具

11.6 执行管线三阶段

阶段	方法	事件	说明
Pre-execute	<code>prepareExecution</code> (L1462)	<code>tools/pre-execute</code> (waterfall)	策略注入、审批、限制检查
Execute	<code>dispatchToolBody</code> (L1531)	<code>tools/execute</code> (waterfall)	并发执行工具体
Post-execute	<code>postExecute</code> (L1741)	<code>tools/post-execute</code> (waterfall)	结果后处理、信号融合

11.7 执行模式 (ToolExecutionMode)

模式	说明
同步	立即执行并返回结果
异步/延迟	经 <code>dispatchScheduledExecution</code> 调度
barrier	需等待的前置屏障
concluding	收尾执行

11.8 技术栈

技术	用途
TypeScript ESM	实现
Cordis	waterfall 事件 (pre/execute/post)
有界滚动池	并发控制

11.9 依赖关系

11.9.1 依赖模块

- `core/scope` (作用域注册)
- `@deepseek-ai/cordis`

11.9.2 被依赖模块

- `core/agent-loop` (工具调度)
- 所有 Consumer 工具包: `shell/`、`fs/`、`terminal/`、`web/`、`skill/`、`todo/`、`subagent/`、`jobs/` 等
- `guard/` (工具超时强制器注入 `tools/execute`)
- `interaction/` (审批策略注入 `tools/pre-execute`)

11.10 相关文档

- 工具执行管线子功能分析
 - 系统架构 - 能力接缝
-

12 工具执行管线 - 子功能概览

12.1 功能简介

工具执行管线是 `core/tools` 的核心子功能，负责将模型发起的 `tool/call` 安全、有序、并发地执行并产出 `tool/result`。它采用三阶段 **waterfall + 有界滚动池** 架构，在策略注入、并发控制、结果融合之间取得平衡。

12.2 支持的阶段/类型

1. Pre-execute 阶段 — 详细说明

- 策略注入、审批检查、限制检查
- waterfall 事件，可拦截/修改/拒绝
- 适用场景：权限控制、参数校验、审计

2. Execute 阶段 — 详细说明

- 并发执行工具体
- barrier 与有界滚动池控制
- 适用场景：实际工具操作 (shell/fs/web/skill)

3. Post-execute 阶段 — 详细说明

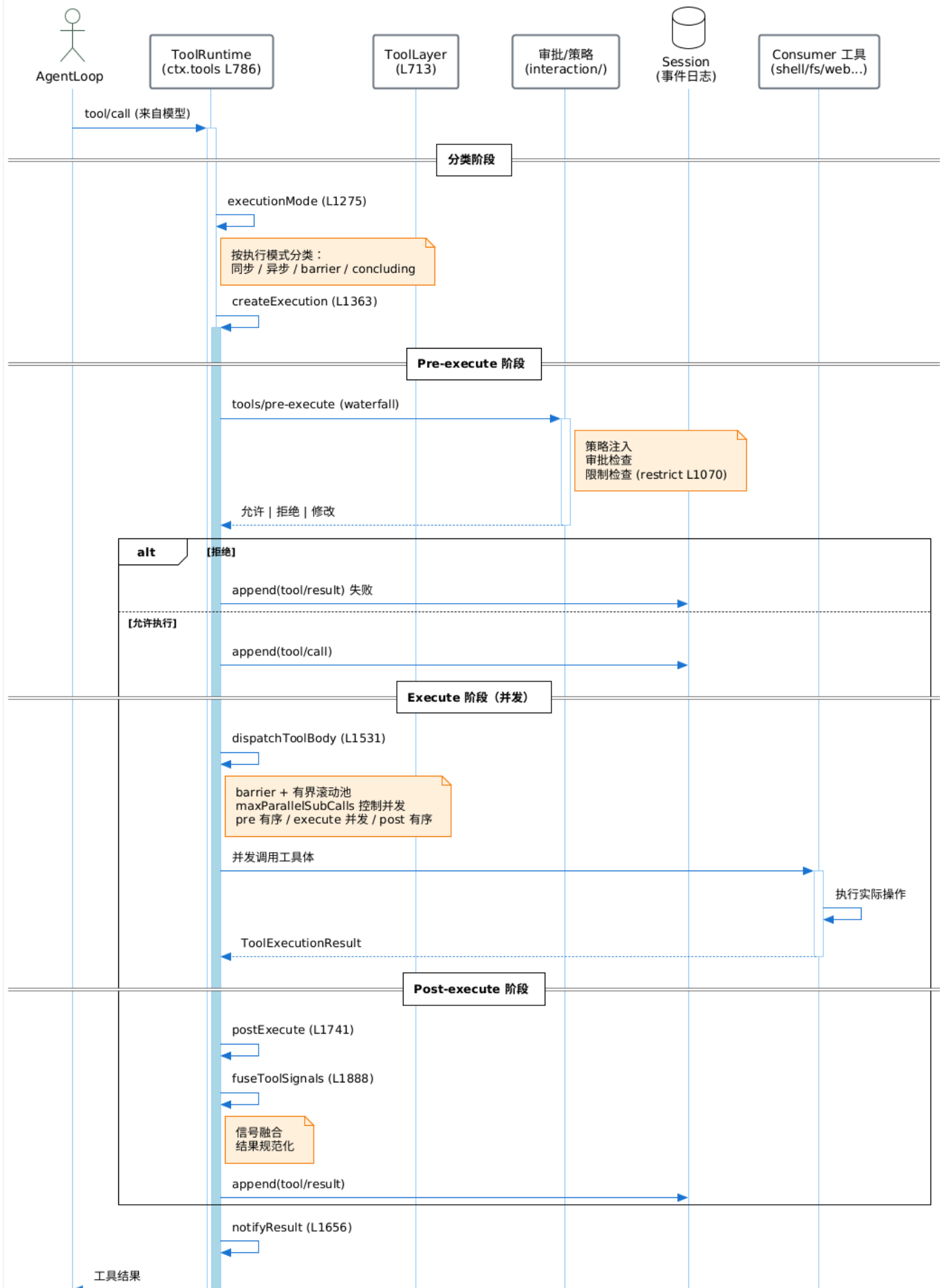
- 结果后处理、信号融合
- waterfall 事件
- 适用场景：结果规范化、副作用记录、压缩

12.3 通用流程

```
tool/call (来自模型)
-> ToolRuntime.executionMode 分类 (L1275)
-> ToolRuntime.createExecution (L1363)
-> [Pre] prepareExecution (L1462) + tools/pre-execute waterfall
    -> 审批/策略/限制 (interaction/, guard/)
-> [Execute] dispatchToolBody (L1531) + tools/execute waterfall
    -> Consumer 工具体 (shell/fs/web...)
    -> barrier + 有界滚动池 (maxParallelSubCalls)
-> [Post] postExecute (L1741) + tools/post-execute waterfall
    -> fuseToolSignals (L1888)
-> notifyResult (L1656)
-> append(tool/result)
```

详见 [执行管线时序图](#)。

工具执行管线：三阶段调度



- 三阶段管线特点：
- 1. pre-execute 是 waterfall（策略/审批可拦截）
 - 2. execute 并发，受 barrier 与滚动池控制
 - 3. post-execute 是 waterfall（结果后处理）
 - 4. 所有 tool/call 与 tool/result 持久到会话日志

工具执行管线时序图

12.4 核心接口

接口/符号	位置	说明
ToolExecutionInput	index.ts L313-337	执行输入
ToolExecution	L378-383	执行描述
ToolExecutionMode	L343-345	执行模式枚举
ToolExecutionResult	L579	执行结果
ToolExecutionSuccess	L555-565	成功结果
ToolExecutionFailure	L568-576	失败结果
PreToolDecision	L587-590	Pre 阶段决策
PostToolDecision	L596-599	Post 阶段决策
ToolRestriction	L679-684	工具限制
CompiledToolRestriction	L687-690	编译后限制
ToolGuard	L710	守卫
FusedToolSignal	L768-771	融合后信号

12.5 扩展机制

- 注册新工具：defineTool() (schema.ts L544-616) 定义 + ToolRuntime.register() (L1036-1061) 注册。
- 注入策略：监听 tools/pre-execute waterfall (如 interaction/ 审批、guard/ 超时) 。
- 结果后处理：监听 tools/post-execute waterfall。
- 限制工具：ToolRuntime.restrict() (L1070-1097) 添加限制。
- 守卫：ToolRuntime.guard() (L1109-1115) + guardReason() (L1118-1127) 。

12.6 配置选项

选项	类型	默认值	说明
<code>maxParallelSubCalls</code>	number	—	最大并发子调用数
<code>defaultMode</code>	ToolExecutionMode	—	默认执行模式
<code>layers</code>	ToolLayer[]	—	工具分层
<code>concludingExecutions</code>	—	—	收尾执行集合
<code>cancellationStates</code>	—	—	取消状态映射
<code>deferredContexts</code>	—	—	延迟上下文

12.7 性能特点

- **并发执行**：execute 阶段并发，受 `maxParallelSubCalls` 上限。
- **barrier 协调**：前置 barrier 需等待，保证顺序依赖。
- **有界滚动池**：避免无限制并发导致资源耗尽。
- **信号融合**：`fuseToolSignals` (L1888-1915) 合并多工具信号，减少冗余。
- **pre/post 有序**：保证策略与结果处理的确定性顺序。

12.8 注意事项

- Pre-execute 的 waterfall 决策是权威的——一个 block 则整体 block (restrictive merge) 。
- 取消传播：`callerCancelled` (L1509-1514) → `cancellationResult` (L1517-1524) 。
- `serviceAsk` (L1688-1728) 支持工具向用户提问 (ask-user) 。
- Code Mode 工具 (`createRunCodeTool` `code-mode.ts` L293-672) 是特殊的代码执行工具。

13 数据架构

本目录包含 DeepSeek Harness (`dsh`) 项目的数据架构分析文档，基于 AST 符号工具对源码符号验证生成。

13.1 文件清单

文件	内容
<code>data-architecture.md</code>	完整数据架构文档 (中文, 13 章)
<code>data-architecture.puml</code>	数据架构组件图 (PlantUML)
<code>data-model-er.puml</code>	数据模型 ER 图 (PlantUML)
<code>data-flow.puml</code>	数据流程图 (PlantUML, 含 5 个流程)

13.2 文档结构

`data-architecture.md` 包含 13 章：

1. **概述** — 设计理念（事件溯源 + 双后端 + 无兼容承诺）与数据层次
2. **主数据库设计** — SQLite 连接管理、SCHEMA_VERSION、核心 3 表、事务边界、撕裂尾修复
3. **JSONL 会话日志设计** — 文件布局、格式、写透发布（POSIX/Windows）、追加写与回滚、增量扫描
4. **核心数据模型** — SessionEventMap 13 个事件类型、SessionHeader、SessionEvent 信封、Session/SessionStore 类
5. **数据关系图说明** — ER 图关系总览
6. **数据流** — 事件追加→派生→持久化→检索、fork、resume、FTS 索引同步
7. **辅助存储** — attachment 内容寻址、storage hub、spill、settings、credentials、投影缓存
8. **数据一致性保证** — 事件溯源不变式、双后端一致语义、协调器不变式
9. **性能优化** — 写后批合并、chunk 打包、Zstd、seek 读、FTS5、投影缓存等
10. **数据迁移与版本管理** — 双版本体系、无迁移策略、升级路径
11. **安全性设计** — 凭据引用、进程限制、跨进程安全
12. **数据架构组件图** — 组件分层引用
13. **关键发现总结** — 10 条核心发现

13.3 关键发现

1. **事件溯源是核心**：会话状态是 `SessionEvent` 追加只写日志的派生物
2. **双后端等价语义**：JSONL 与 SQLite 实现同一 `PersistenceBackend` 接口，崩溃修复语义一致
3. **Pre-release 无迁移**：所有 schema 版本不匹配即拒绝，派生索引丢弃重建
4. **写透发布平台分化**：POSIX 用 `link()` + 目录 fsync；Windows 用 `MoveFileExW(MOVEFILE_WRITE_THROUGH)`
5. **模型可见即已记录**：`Session.append` 用 `isJsonValue` 在源头拒绝非可序列化数据
6. **内容寻址附件**：附件 ID 即 sha256，读取时 digest 校验
7. **投影缓存 fail-soft**：派生状态检查点失败仅 warning，下次冷读自愈
8. **凭据分层信任**：进程环境 > 托管 `.credentials.yaml` > `.env`
9. **FTS5 派生索引可弃**：版本不匹配即 `resetDerivedSchema` 重建
10. **双版本体系**：`SESSION_FORMAT_VERSION`（事件词汇表）与 `SCHEMA_VERSION`（表布局）正交

13.4 关键代码位置

模块	路径	关键符号
核心数据模型	<code>packages/core/session/src/types.ts</code>	<code>SessionEventMap</code> (L235-332)、 <code>SessionHeader</code> (L60-98)、 <code>SessionEvent</code> (L403-435)
会话运行时	<code>packages/core/session/src/index.ts</code>	<code>Session</code> (L424-757)、 <code>SessionStore</code> (L791-1154)、 <code>append</code> (L603-654)、 <code>fork</code> (L1080-1094)
SQLite 后端	<code>packages/session/session-persistence-sqlite/src/schema.ts</code>	<code>SCHEMA_VERSION=15</code> (L20)、3 表 DDL (L116-147)、 <code>scanRows</code> (L232-270)
SQLite 后端	<code>packages/session/session-persistence-sqlite/src/index.ts</code>	<code>appendBatch</code> (L284)、 <code>commitRepair</code> (L309)、 <code>loadStoredFrom</code> (L225)
JSONL 后端	<code>packages/session/session-persistence-jsonl/src/format.ts</code>	<code>SessionLogScanner</code> (L272)、 <code>scanLog</code> (L388)、 <code>encodeSegment</code> (L121)
JSONL 后端	<code>packages/session/session-persistence-jsonl/src/index.ts</code>	<code>appendBatch</code> (L421)、 <code>materialize</code> (L513)、 <code>appendLines</code> (L650)
Windows 写透	<code>packages/session/session-persistence-jsonl/src/win32.ts</code>	<code>publishNewFileWin32</code> (L116)、 <code>ensureDurableDirectoryWin32</code> (L130)
持久化协调	<code>packages/session/session-persistence/src/coordinator.ts</code>	<code>PersistenceBackend</code> 接口 (L127)、 <code>sessionFormatVersionRefusal</code> (L77)
写后批合并	<code>packages/session/session-persistence/src/write-behind.ts</code>	<code>SessionWriteBehind</code> (L22)
检索派生 schema	<code>packages/session-query/session-query-sqlite/src/schema.ts</code>	<code>SESSION_QUERY_SQLITE_SCHEMA_VERSION=8</code> (L8)、FTS5 表 (L126-168)
检索引擎	<code>packages/session-query/session-query-sqlite/src/index.ts</code>	<code>SqliteSessionQueryEngine</code> (L196)
逻辑语料	<code>packages/session-query/session-query/src/corpus.ts</code>	<code>SessionCorpus</code> (L32)
附件存储	<code>packages/attachment/attachment-local/src/store.ts</code>	<code>saveImageFile</code> (L136)、 <code>readImageFile</code> (L204)
存储 Hub	<code>packages/storage/storage/src/backend.ts</code>	<code>StorageBackend</code> (L17)、 <code>KvFacet</code> (L30)、 <code>KvUnit</code> (L66)
Domain 规范	<code>packages/storage/storage-domain/src/spec.ts</code>	<code>DomainSpec</code> (L35)、 <code>defineDomain</code> (L79)
Spill 存储	<code>packages/spill/spill-local/src/store.ts</code>	<code>saveTextFile</code> (L107)、 <code>privateRoot</code> (L27)
投影缓存	<code>packages/session/session-projection-cache/src/index.ts</code>	<code>SessionProjectionCache</code> (L71)

模块	路径	关键符号
投影注册表	packages/session/session-projection/src/index.ts	ProjectionDefinition (L42)
设置文件	packages/settings/settings-file/src/index.ts	FileSettingsProvider (L105)
凭据文件	packages/credentials/credentials-local/src/index.ts	分层信任 (L1-35)、assertOwnerOnly (L103)

13.5 渲染 PUML

```
# 使用 PlantUML 渲染（需 Java）
plantuml data-architecture.puml data-model-er.puml data-flow.puml

# 或使用 VS Code PlantUML 插件预览
```

13.6 相关文档

- 01-项目概览
- 02-系统架构

14 数据架构说明

本文档基于 AST 符号工具对源码符号验证生成。代码引用处的行号均来自符号工具验证。配套图：数据架构组件图、数据模型 ER 图、数据流程图。

14.1 概述

14.1.1 设计理念

DeepSeek Harness（dsh）的数据架构围绕三条核心原则展开：

原则	含义	体现
事件溯源（Event Sourcing）	会话状态是追加只写事件日志的派生物；事件是不可变事实，派生视图从事件流折叠而来。	<code>Session.append</code> 仅追加； <code>deriveMessages</code> 从事件派生模型历史；投影（projection）以 <code>init/apply/view</code> 三函数折叠。
双后端持久化	同一事件日志可通过 JSONL 或 SQLite 后端持久化，二者实现同一 <code>PersistenceBackend</code> 接口，崩溃语义一致。	<code>JsonlSessionPersistence</code> 、 <code>SqliteSessionPersistence</code> 均继承 <code>SessionPersistence</code> 并实现 <code>PersistenceBackend</code> 。
Pre-release 无兼容承诺	项目处于 <code>0.1.0-rc.5</code> Developer Preview，无外部消费者，优先正确基础而非兼容垫片——schema 版本不匹配即拒绝，不做迁移。	SQLite <code>SCHEMA_VERSION = 15</code> 不匹配即抛错； <code>SESSION_FORMAT_VERSION</code> 不匹配即 <code>SessionFormatUnsupportedError</code> 。

14.1.2 数据层次

系统数据自顶向下分四层：

层	内容	代表符号
L1 事件流（权威）	追加只写的 <code>SessionEvent</code> 日志	<code>Session.log</code> 、 <code>SessionEventMap</code>
L2 派生视图	从事件流折叠出的模型历史、投影状态	<code>Session.deriveMessages</code> 、 <code>SessionProjectionRegistry</code>
L3 持久化后端	把事件流落到磁盘（JSONL/SQLite）	<code>PersistenceCoordinator</code> 、 <code>PersistenceBackend</code>
L4 辅助存储	附件、设置、凭据、spill、投影缓存	<code>attachment-local</code> 、 <code>settings-file</code> 、 <code>credentials-local</code> 、 <code>spill-local</code> 、 <code>session-projection-cache</code>

只有 L1 是权威；L2~L4 都是派生或缓存，丢失后可从 L1 重建。

14.2 主数据库设计（SQLite 持久化后端）

SQLite 后端把会话头与事件映射为行，委托 `PersistenceCoordinator` 编排写路径。代码：`packages/session/session-persistence-sqlite/src/schema.ts`、`packages/session/session-persistence-sqlite/src/index.ts`。

14.2.1 数据库连接与所有权

- 打开：`openDatabase(path, journalMode)`（`schema.ts:81`）以 `BEGIN IMMEDIATE` 持写锁校验所有权，避免其他连接在检查与初始化之间抢占 schema。
- 所有权校验（`schema.ts:105-114`）：
 - `user_version = 0` 且 (`application_id ≠ 0` 或存在用户对象) → 拒绝（无版本 schema 或外部 identity）
 - `user_version ≠ 0` 且 `≠ SCHEMA_VERSION` → 拒绝不兼容版本
 - `user_version = SCHEMA_VERSION` 且 `application_id ≠ SESSION_PERSISTENCE_SQLITE_APPLICATION_ID` → 拒绝
- 常量（`schema.ts:20-23`）：
 - `SCHEMA_VERSION = 15`（monotonic，破坏性变更时 bump）
 - `SESSION_PERSISTENCE_SQLITE_APPLICATION_ID = 0x44534850`（ASCII “DSHP”）
- Journal Mode（`schema.ts:70`）：`'wal' | 'delete' | 'truncate' | 'persist'`；默认 `wal`，回滚日志模式用于 WAL 共享内存不工作的网络挂载；排除 `memory / off`（静默丢弃日志耐久性）。

- **写入争用**：后端不设 busy timeout，不重试 locked-database 错误——其他连接持写事务时操作立即拒绝。

14.2.2 核心表结构

后端在初始化时创建三张 STRICT 表（`schema.ts:116-147`）：

```
-- 存储身份单例 (store identity singleton)
CREATE TABLE IF NOT EXISTS persistence_state (
  singleton INTEGER PRIMARY KEY CHECK (singleton = 1),
  store_id TEXT NOT NULL
) STRICT;

-- 会话元数据 (out-of-log header)
CREATE TABLE IF NOT EXISTS sessions (
  id TEXT PRIMARY KEY,
  version INTEGER NOT NULL,          -- SESSION_FORMAT_VERSION
  created_at INTEGER NOT NULL,
  cwd TEXT,
  parent_session TEXT,              -- 分叉谱系
  seed_length INTEGER,             -- 继承的种子事件数
  origin TEXT,                     -- 'subagent' 或 null
  delegation_depth INTEGER,
  agent_preset TEXT,
  incarnation TEXT NOT NULL,        -- 物化时分配的稳定身份
  revision INTEGER NOT NULL         -- 单调递增的日志变更令牌
) STRICT;

-- 事件行 (1:1 映射 SessionEvent)
CREATE TABLE IF NOT EXISTS events (
  session_id TEXT NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
  seq INTEGER NOT NULL,             -- 会话内单调序列号
  type TEXT NOT NULL,
  time INTEGER NOT NULL,
  data TEXT NOT NULL,              -- JSON 文本
  source_event_seqs TEXT,           -- JSON number[] 或 null
  surface_op TEXT,                  -- JSON SurfaceOp 或 null
  ignorable INTEGER,               -- 1 iff ignorable: true
  PRIMARY KEY (session_id, seq)
) STRICT
```

关键设计：

- **惰性物化**：`sessions` 行仅在首次 `append` 时写入（`writeRow`，`index.ts:385`），所以「已创建但从未追加」的会话无行、不出现在 `list` 中——与 JSONL 后端「无文件直到首次追加」语义一致。
- **incarnation**：物化时分配的 `randomUUID()`，作为 `storeIdentity` 的一部分，让 `readStoredRevision` 区分同一 id 下的不同生命周期。
- **revision**：每个变更事务 +1（`index.ts:296`），让 `SessionPersistenceRevision` 表达「同一身份下的精确前缀」。
- **ON DELETE CASCADE**：seam 无删除 API；行累积直到外部带外清理时级联删除事件。
- **STRICT 表**：所有列类型严格校验，避免 SQLite 弱类型导致的脏数据。

14.2.3 事务边界

- **appendBatch**（`index.ts:284-302`）：`BEGIN` → `writeRow`（若未物化）→ 逐行 `INSERT` 事件 → `UPDATE sessions SET revision = revision + 1` → `COMMIT`。物化写与首批事件原子提交，崩溃不会留下「物化但空」的会话。失败 `ROLLBACK`，存储日志保持未触碰。

- `commitRepair` (`index.ts:309-338`) : `BEGIN` → `DELETE FROM events WHERE seq >= tornMarker` (截断撕裂尾) → `INSERT closers` (合成关闭器) → `UPDATE revision` → `COMMIT` 。一次事务完成崩溃修复, `COMMIT` 后存储行 == 平衡日志。
- `loadStoredFrom` (`index.ts:225-238`) : `seek-capable` 后缀读, `SELECT ... WHERE seq >= fromSeq ORDER BY seq` , 让 `readFrom` 随后缀规模 (而非整日志规模) 扩展。

14.2.4 撕裂尾修复 (scanRows)

`scanRows` (`schema.ts:232-270`) 找出有序事件行的保留前缀:

1. **Pass 1**: 解析每行 `data` ; JSON 解析失败的行标记为洞 (hole) 。
2. 找最后 `turn/end` : 从末尾向前扫描, 定位最后一个有效的 `turn/end` 。
3. 保留前缀: 连续前缀 (含完整的中断 `turn`) ; 通过最后 `turn/end` 之前的洞抛错 (已提交损坏) , 之后的洞停止 (容忍撕裂尾) 。
4. `tornFrom` : 保留前缀之后的第一 `seq` , 作为 `commitRepair` 的物理删除起点。

这与 JSONL 后端的 `SessionLogScanner` 给出相同的崩溃尾语义。

14.3 JSONL 会话日志设计

JSONL 后端把会话头与事件存为每会话一个追加只写文件, 委托 `PersistenceCoordinator` 编排。代码: `packages/session/session-persistence-jsonl/src/format.ts` 、 `packages/session/session-persistence-jsonl/src/index.ts` 、 `packages/session/session-persistence-jsonl/src/win32.ts` 。

14.3.1 文件布局

每会话一个目录, 按项目分组 (`format.ts:176-208`) :

```
<root>/                                -- 配置的会话根目录
├─ _no-cwd/                             -- cwd 未定义的会话
│   └─ <encoded-session-id>/
│       └─ session.jsonl.zstd
└─ --<project-key>--/                    -- cwd 编码为可读目录键
    └─ <encoded-session-id>/
        └─ session.jsonl.zstd           -- 默认 Zstandard 压缩
```

- `encodeSegment` (`format.ts:121-136`) : 把任意字符串单射编码为安全路径段。 `SessionId` 是未校验的 branded string, 必须先编码才能用于路径——中和 `../` 、绝对路径、NUL、分隔符; `~` 转义为 `~XXXX` , 保证可逆。
- `projectKey` (`format.ts:147-167`) : 把项目路径编码为可读目录键, 分隔符变 `-` , 不安全码元用 `~XXXX` , 截断到 251 字节, 前后加 `--` 边界。
- `logPath` (`format.ts:201-208`) : `sessionDir/projectDir/encodeSegment(id) + session<suffix>` 。
- 物理编码 (`format.ts:17-26`) : 默认 `zstd` (`.jsonl.zstd`) , 可选 `none` (`.jsonl`) ; 切换编码需独立/全新根, 预发布格式无迁移。

14.3.2 文件格式

首行是头记录, 后续每行一个事件 (`format.ts:33-44, 221-224`) :

```
{ "type": "session", "version": 0, "id": "session-1", "createdAt": 1234567890, "cwd": "/work", "delegationDepth": 0 }
{ "type": "turn/start", "seq": 0, "time": 1234567890, "data": { "turn": 0 } }
{ "type": "user/message", "seq": 1, "time": 1234567891, "data": { ... }, "sourceEventSeqs": [], "surfaceOp": "append" }
...
```

- `HeaderLine` (`format.ts:33-44`) : `type: 'session'` 标记, 让读者从头行中区分出事件行; `delegationDepth` 总是数字 (默认 0) ; 拒绝已退休的 `sandboxMode` / `approvalPolicy` 字段 (`format.ts:72-74`) 。
- `eventLines` (`format.ts:221-224`) : 批次序列化为 JSONL 文本 (无尾换行) 。 `packChunks` 开启时, 连续 `assistant/chunk` `delta` 事件打包为 `text-chunks` / `reasoning-chunks` / `tool-call-chunks` 存储行 (无损, 实测约 60% 压缩) ; 关闭则一事件一行, 与打包前布局字节相同。读取布局无关——

`scanLog` 总是解码行，开关只影响新写字节。

- `SESSION_FORMAT_VERSION`：事件词汇表版本，写入头行；后端拒绝任何其他版本（无迁移）。

14.3.3 写透发布 (Write-Through Publication)

POSIX 路径（`index.ts:529-569`）：

1. `mkdir -p` 创建 `root/project/session` 三级目录，每级 `mode: 0o700`。
2. 每级创建后 `syncDirPosix`（`index.ts:636-643`）—— `open(dir, 'r') + handle.sync()` fsync 父目录元数据。
3. `writeSyncedTempFile`（`index.ts:606-616`）：`open(tmp, 'wx', 0o600) → writeFile → handle.sync()` fsync 文件数据。
4. `link()` 发布（`index.ts:549`）：用 `link(tmp, finalPath)` 而非 `rename()` —— `link` 在最终路径已存在时 `EEXIST` 失败，两进程并发物化同一 id 不会互相覆盖。
5. `syncDirPosix(dir)` fsync 目录，让新 link 元数据 crash-durable。
6. `rm(tmp)` 最佳努力清理临时硬链接，失败不影响已发布的日志。

Windows 路径（`win32.ts`、`index.ts:573-591`）：

Windows 不通过 Node 暴露父目录 fsync 契约，改用原生耐久命名空间原语（`win32.ts:1-12`）：

- `publishNewFileWin32`（`win32.ts:116-120`）：`MoveFileExW(existing, replacement, MOVEFILE_WRITE_THROUGH)`（`MOVEFILE_WRITE_THROUGH = 0x8`）。目的地必须不存在；移动必须同卷（无 copy fallback）。
- `ensureDurableDirectoryWin32`（`win32.ts:130-142`）：逐级创建——每级先用 `mkdtemp` 创建随机 staging 兄弟，再 `publishNewFileWin32` 移到最终名；与另一创建者的竞争仅在验证赢家是目录后才接受。
- 延迟加载 `Koffi`（`win32.ts:42-51`）：非 Windows 进程永不加载 `koffi`，避免不必要的原生依赖。

14.3.4 追加写与回滚

`appendLines`（`index.ts:651-679`）：

1. `open(path, 'a')` 以追加模式打开。
2. 记录 `before = handle.stat().size`。
3. `writeFile(content) + handle.sync()` fsync。
4. 失败回滚（`index.ts:667-675`）：关闭句柄后 `rollbackAppend`（`index.ts:681-689`）`truncate(path, before) + sync()`，恢复到追加前大小——因为不变的游标会重试批次，留下部分字节会制造重复 seq。
5. 双重失败抛 `AggregateError`。

14.3.5 增量扫描与撕裂修复

`SessionLogScanner`（`format.ts:272-378`）增量扫描完整 JSONL 事件记录：

- 构造：从一行完整头记录开始（`parseHeaderRecord`，`format.ts:249-264`），先 `refuseForeignFormatVersion` 拒绝异类格式版本，再 `isHeaderLine` 结构校验。
- `write(chunk)`（`format.ts:297-323`）：按 `\n` 分割，跨 chunk 的不完整记录用 `fragments` 缓冲拼接；只对完整记录解码 UTF-8。
- `consumeEventLine`（`format.ts:347-377`）：解码一行，更新连续前缀；遇 `turn/end` 时若有未抛的 issue 则抛出（已提交损坏）；seq 间隙在已提交区抛错，在撕裂尾停止。
- `finish`（`format.ts:341-344`）：忽略无尾换行的最终记录（撕裂尾），返回头、连续事件前缀、安全截断字节偏移。
- `scanLog`（`format.ts:388-394`）：完整缓冲兼容包装，先单独取头记录再委托给 scanner。
- `parseHeaderMeta`（`format.ts:404-413`）：仅解析头行——`list()` 用它读元数据而无需解析整日志，让会话选择器随会话数（而非每会话大小）扩展。

14.4 核心数据模型

核心数据模型在 `packages/core/session/src/types.ts` 与 `packages/core/session/src/index.ts`。

14.4.1 SessionEventMap 事件类型体系

SessionEventMap （ types.ts:235-332 ） 是会话事件词汇表，声明合并可扩展。13 个事件类型分四类：

14.4.1.1 生命周期边界（turn/step）

事件	数据	说明
turn/start	{ turn: number }	在 loop claim 排队输入或运行 pre-step 前打开 turn。拒绝/空输入/取消/失败可能无 step 关闭它。
turn/end	{ turn: number; reason: TurnEndReason }	关闭 turn。loop 在 turn 边界不 await flush —— dsh-session-checkpoint-policy 拥有每请求耐久检查点。成功提交 turn；拒绝实时上报不阻塞后续工作。
step/start	{ turn: number; step: number }	打开 turn 内的 step（一次模型调用 + 其请求的工具执行）。
step/end	{ turn: number; step: number }	关闭 step。

14.4.1.2 模型可见表面（surface events）

事件	数据	说明
user/message	UserMessage	用户角色消息：直接人类 prompt、合成 agent.inject() 上下文（文件变更通知、子目录 AGENTS.md、skill 内容、cron 通知）、或输入的目标延续回合。三者都逐字投影 content；source 区分。
assistant/chunk	{ turn; step; chunk: StreamChunk }	原始流 chunk——token 级回放保真。
assistant/message	{ turn; step; message: AssistantMessage; usage?: TokenUsage }	一个 step 的组装助手消息（派生历史用此）。携带 step 的 usage ——模型输出与记账同行，无独立 usage 记录。适配器未上报时 usage 缺失。
tool/call	{ turn; step; callId: CallId; name: string; arguments: string }	模型请求一次工具调用：name + 模型原样产生的未解析 arguments JSON 串。callId 配对 tool/result。
tool/result	{ turn; step; message: ToolResultMessage; error?: {name; code}; meta?: JsonValue }	工具调用的模型面结果。meta 对核心不透明（产出工具拥有其形状并在 presentResult 读回），但必须 JSON 可序列化——Session.append 用 isJsonValue 运行时校验所有事件数据，非可序列化 meta 在源头拒绝，耐久日志在回放时重现相同卡片。

14.4.1.3 仅日志状态（log-only）

事件	数据	说明
todo/write	<code>{ todos: TodoItem[] }</code>	整列表快照；回放时最新写胜出。仅日志 UI 状态， 从不 派生历史。
request/header	<code>{ header: EpochHeader; reason: RequestHeaderReason }</code>	下次请求的完整头，在 step 内分发前追加。仅日志；最新快照重建请求头。
request/context	<code>RequestContext</code>	下次请求的路由元数据，仅在路由或容量变更时记录。不参与请求重建或头相等。

14.4.1.4 生命周期种子标记

事件	数据	说明
session/end-seed	<code>Record<string, never></code>	标记构造器种子结束。此事件之前有更小 seq 的事件来自种子 (resume/fork/replay) ；此生命周期不产出它们。仅日志事件，是 <code>Session.firstLiveSeq</code> 的耐久投影。载荷为空——位置与 <code>time</code> 承载含义。定位存储历史中的 最后一个 。已以它结尾的种子不重标记，所以重开未触碰会话不增长日志。 <code>Session</code> 构造器是唯一合法写者。

14.4.2 SessionHeader（会话头）

`SessionHeader` （`types.ts:60-98`）是会话的不可变元数据：

字段	类型	说明
version	readonly number	磁盘格式版本，创建时从 <code>SESSION_FORMAT_VERSION</code> 盖戳。后端加载时拒绝任何其他版本（无迁移）。
id	readonly SessionId	会话 id（镜像 <code>Session.id</code> ）。
createdAt	readonly number	非负安全整数 Unix epoch 毫秒。
cwd	readonly string?	创建会话时的绝对工作目录。
parentSession	readonly SessionId?	分叉源（种子谱系）。
seedLength	readonly number?	通过种子继承的前导事件数；持久化此边界让 <code>resume/replay</code> 区分父历史与子工作。
origin	readonly 'subagent'?	子 agent 创建时的粗粒度产品分类——展示元数据， 不 证明子可延续。
delegationDepth	readonly number?	委派深度：顶层缺省（零），子 agent 为父深度 + 1。持久化让递归预算穿越 <code>restart/resume</code> ——纯运行时深度会让 <code>resumed</code> 子重置为顶层。
agentPreset	readonly string?	此会话 agent 组合自的 preset id。耐久因为 preset 决定会话的工具与 prompt—— <code>resume</code> 恢复不同组合会重放模型无法再行动的历史。

14.4.3 SessionEvent（事件信封）

`SessionEvent`（`types.ts:403-435`）是事件信封类型：

```
type SessionEvent<T extends SessionEventType = SessionEventType> = {
  [K in SessionEventType]: {
    type: K
    seq: number // 会话内单调序列号
    time: number // Unix epoch 毫秒
    data: SessionEventMap[K]
    ignorable?: true // 读者可不识别时安全跳过
  } & (K extends SurfaceEventType ? {
    sourceEventSeqs?: number[] // 引用的更早事件 seq（如 chunk 组成 message）
    surfaceOp?: SurfaceOp // 事件如何进入表面
  } : object)
}[T]
```

`ignorable` 语义（`types.ts:412-421`）：缺省意味着 required——读者遇不识别 `type` 且无此标记时**必须**拒绝重建会话，而非静默丢弃事件（不识别的 required 事件可能改变后续日志解释）。写者仅在纯信息记录（丢失不影响重建）上设 `true`；默认 required 让遗忘的标记过度拒绝（不便）而非静默恢复被掏空的会话。

14.4.4 Session 类

`Session` 类（`index.ts:424-757`）是事件日志的运行时持有者：

成员	行号	职责
<code>log: SessionEvent[]</code>	L425	追加只写的事件数组（权威源）。
<code>surfaceManager</code>	L427	表面管理器， <code>validateNext</code> 校验下一事件合法性。
<code>header: SessionHeader</code>	L442	不可变会话头。
<code>firstLiveSeq</code>	L471	此生命周期产生的第一个 seq（种子边界）。
<code>eventsSnapshot</code>	L550	缓存的事件快照（ <code>invalidated on append</code> ）。
<code>append</code>	L603-654	追加事件（见 4.5）。
<code>requestHeader</code>	L669-679	从日志派生下次请求头。
<code>requestContext</code>	L690-698	从日志派生请求上下文。
<code>deriveMessages</code>	L725-746	从事件派生模型历史（核心派生视图）。

14.4.5 Session.append 详解

`Session.append`（`index.ts:603-654`）是事件入口，体现「模型可见即已记录」原则：

- `snapshotJsonValue(data)`（L617-620）：深快照 + JSON 可序列化校验。非可序列化数据抛 `session event "... carries non-JSON-serializable data`——在源头拒绝，保证耐久日志能回放相同事件。
- `assertSupportedRequestHeader`（L623）：校验请求头事件合规。
- `snapshotJsonValue(surfaceMetadata)`（L625-629）：表面元数据同样快照+校验。
- 重入保护**（L621-622, L635-652）：`entry.appending` 标记防止追加时重入；`finally` 块清理标记并在请求 detach 时执行。
- `deepFreeze`（L627）：事件对象深冻结，保证不可变。
- `surfaceManager.validateNext`（L633）：表面管理器校验事件序列合法性。
- `log.push`（L641）：追加到权威日志。
- `eventsSnapshot = undefined`（L642）：失效快照缓存。
- 回调分发**（L637-640, L643-647）：通过 `collectSessionCallbacks` 收集 `session/event` 监听器并 `invokeContainedSessionObservers` 触发。

14.4.6 SessionStore 类

`SessionStore` 类（`index.ts:791-1154`）管理多个 Session：

SessionHeader 1—* SessionEvent (头 → 事件流)
SessionEvent *—1 SessionEventMap[K] (信封 → 类型化载荷)
Session 1—1 SessionHeader (运行时 → 头)
SessionStore 1—* Session (store → 会话集合)
Session 1—* SessionEvent (会话 → 日志)

持久化层 (SQLite) :

sessions 1—* events (ON DELETE CASCADE)

持久化层 (JSONL) :

HeaderLine 1—* event lines (首行 → 事件行)

派生层:

SessionEvent *—1 SurfaceNode (表面事件 → 表面节点)

Session 1—* ProjectionSnapshot (会话 → 投影快照)

辅助存储:

Session 1—* AttachmentRef (会话 → 附件引用)

AttachmentRef *—1 ContentObject (引用 → 内容寻址对象 sha256:hex)

Session 1—* SpillFile (会话 → spill 文件)

14.6 数据流

详见 [data-flow.puml](#)。



14.6.1 事件追加→派生→持久化→检索

```
[agent loop]
  | produce event
  ▼
Session.append(type, data, surfaceIntent?)
  | snapshotJsonValue(data) ← JSON 可序列化校验
  | deepFreeze(event)
  | surfaceManager.validateNext(event)
  | log.push(event) ← 权威日志追加 (内存)
  | session/event 回调分发
  ▼
[SessionWriteBehind.enqueue(event)] ← 持久化协调器订阅
  | structuredClone(event)
  | pending.push(event)
  | armTimer(maxDelayMs=200)
  ▼
[deadline or flush]
  | startWrite(batch)
  ▼
PersistenceBackend.appendBatch(meta, events, isMaterialized)
  |— SQLite: BEGIN → writeRow? → INSERT events → UPDATE revision → COMMIT
  |— JSONL: materialize? | appendLines (open 'a' → writeFile → fsync → rollback on fail)
  ▼
[SessionProjectionRegistry.apply(state, event)] ← 投影驱动
  | init/apply/view 纯函数折叠
  ▼
[SessionProjectionCache] ← 节流写后缓存
  | writeEveryEvents / writeIntervalMs / turn/end / disposal 触发
  | fail-soft 写入 storage domain session_projcache
  ▼
[SessionQueryEngine] ← FTS 索引派生
  | SessionCorpus.listSessions / load / projectMany
  | live-preferred: live session 优先于 persisted
  ▼
SqliteSessionQueryEngine.searchSessions / searchEvents
  |— temp.live_docs (FTS5) ← live 会话
  |— persisted_docs (FTS5) ← 持久化会话
```

14.6.2 会话分叉 (fork) 数据流

```
SessionStore.fork ( index.ts:1080-1094 ) :
```

```

fork(source, boundary?, childSessionId?)
  | childSessionId 已存在 → SESSION_ALREADY_EXISTS
  | _resolveForkSource(source) ← 解析为 live Session
  | _forkSeed(liveSource, boundary) ← 取种子事件（到 boundary 或末尾）
  ▼
create(childSessionId, {
  seed, ← 父事件前缀（深拷贝）
  meta: {
    cwd: liveSource.header.cwd,
    parentSession: liveSource.id, ← 谱系
    seedLength: seed.length, ← 种子边界
  }
})
  ▼
Session.create(id, seed, header)
  | log = [...seed] ← 种子事件成为子日志前缀
  | firstLiveSeq = seed.length ← 种子边界
  ▼
首次 append → session/end-seed 标记种子结束
  | 后续事件是子生命周期产出

```

关键点：种子事件被深拷贝，父子日志独立；`parentSession` + `seedLength` 让 resume/replay 区分父历史与子工作；`session/end-seed` 是耐久投影，让冷读识别种子边界。

14.6.3 会话恢复 (resume) 数据流

```

SessionStore.prepare(id, { seedSource: 'persistence' })
  | Session.fromRestore(id, seed, meta)
  ▼
PersistenceCoordinator.load(id)
  | PersistenceBackend.loadStored(id)
  | — SQLite: SELECT sessions row + events ORDER BY seq → scanRows
  | — JSONL: readFile → scanLog (header + SessionLogScanner)
  | tornMarker? → commitRepair (截断撕裂尾 + 合成关闭器)
  ▼
返回 StoredPrefix { meta, events, revision }
  | adoptSessionEvent(events)
  ▼
Session 构造器
  | log = events
  | firstLiveSeq = 最后一个 session/end-seed 位置
  ▼
live 注册 → SessionStore.enter

```

14.6.4 session-query FTS 检索流程

`SqliteSessionQueryEngine` (`session-query-sqlite/src/index.ts:196`) 维护两个 FTS5 索引：

```

[Observation]                                [Indexing]
|                                              |
├─ live sessions                             |
|  (SessionCorpus.listSessions)              |
|  → temp.live_sessions                     |
|  → temp.live_docs (FTS5)                  |
|                                              |
├─ persisted sessions                        |
|  (Persistence.listSnapshots)              |
|  → 比对已索引 revision                    |
|  → 变更则 inspectPersisted                |
|      → buildSessionEventSearchDocuments  |
|      → persisted_sessions                  |
|      → persisted_docs (FTS5)              |
|                                              |
|                                              ▼
[Query]
| normalizeSessionRequest / normalizeEventRequest
| buildSessionWhere / buildEventWhere ← 参数化 SQL 片段
| assertFts5OuterPredicateCount (<=14)
| assertPortableBindingCount (<=32766)
|
| ▼
SELECT ... FROM live_docs/persisted_docs
WHERE <filters> AND docs MATCH ?
ORDER BY ... LIMIT ? OFFSET ?
|
| ▼
makeSnippet (FTS5 highlight, \uFDD0\uFDD1 标记)
|
| ▼
SessionSearchPage / SessionEventSearchPage

```

7. `link(tmp, target)` 硬链接发布——`EEXIST` 时读现有文件校验 digest (dedup 路径)，不匹配抛 `ATTACHMENT_CORRUPT`。
 8. `syncDirectory(bucket) + syncDirectory(objects)` fsync 目录条目，关闭并发桶创建窗口。
 9. `unlink(tmp)` 清理临时。
- **读取流程** (`readImageFile`, `store.ts:204-231`)：读 + `digest` 校验 + `probeImage` 重派生头字段（不重复光栅解码，避免历史回放时的每请求像素放大）。
 - **目录 fsync** (`syncDirectory`, `store.ts:76-87`)：POSIX `open(dir, O_RDONLY)` + `handle.sync()`；Windows 跳过（NTFS 元数据日志拥有条目耐久性）。
 - **版本化根**：`DSH_HOME/attachments/v1`，预留版本演进。

14.7.2 存储 Hub (storage)

代码：`packages/storage/storage/src/backend.ts`、`packages/storage/storage-domain/src/spec.ts`。KV 面向后端的通用存储：

- **StorageBackend** (`backend.ts:17-27`)：拥有一个介质，暴露可选 `kv` facet 与 `close`。
- **KvFacet.open(descriptor)** (`backend.ts:42`)：打开一个单元，介质无痕迹时创建。
- **KvUnit** (`backend.ts:66-104`)：
 - `loadAll()`：读全快照（所有表记录 + global singleton）。
 - `putRecord(table, key, value)`：耐久 upsert（覆盖语义）。
 - `deleteRecord(table, key)`：耐久删除（幂等，缺键 no-op）。
 - `setGlobal(value)`：写 global singleton（仅当 `hasGlobal`）。
 - `close()`：排空在飞写入并释放。
 - **不序列化并发写**——写顺序是调用者责任（domain 层每单元一条写链）；只保证单调用在介质上原子且解析后耐久。
- **UNIT_NAME_RE** (`backend.ts:10`)：`/^[a-z][a-z0-9_]*$/`，单元/表名安全作为文件名与 SQL 标识符段。
- **DomainSpec** (`spec.ts:35-44`)：`name + version + global? + tables`。`defineDomain` (`spec.ts:79-98`) 在模块加载时校验：名不匹配 `UNIT_NAME_RE`、版本非非负整数、global schema 接受 `null` (`null` 是「从未写」哨兵) 都抛错。
- **后端实现**：`storage-json` (YAML/JSON 文件)、`storage-sqlite` (SQLite 表)。

14.7.3 Spill 存储 (spill-local)

代码：`packages/spill/spill-local/src/store.ts`。会话作用域临时文件存储工具结果溢出：

- **privateRoot** (`store.ts:27-30`)：`mkdtempSync(join(tmpdir(), 'dsh-spill-'))`，私有（0700）每进程目录。可预测的世界可读路径会让其他本地用户读 spilled 工具输出或预创建符号链接；`mkdtemp` 给不可预测后缀。
- **encodeSegment** (`store.ts:48-63`)：镜像 JSONL 路径编码器，但保留空名策略（`""` → `"~"`）本地化。
- **sessionDir** (`store.ts:73-76`)：`<root>/session-<sha256(sessionId).slice(0,12)>`，短稳定哈希作用域目录。
- **saveTextFile** (`store.ts:107-120`)：`mkdir 0700 + open(path, 'wx', 0o600)` 独占写。文件名是 `randomBytes(6).hex + '-' + safeName`——不可预测（防共享根中的符号链接种植）且可读。

14.7.4 投影缓存 (session-projection-cache)

代码：`packages/session/session-projection-cache/src/index.ts`。派生状态的耐久检查点：

- **存储域**：`session_projcache`（通过 `storageDomain.open(projectionCacheDomainSpec)`），shipped json backend 与 `workspace.json` 并置。
- **记录形状**：`(sessionId, key, ver, seq, val)`——会话 id + 投影键 + 单元 stateVersion + 水位 seq + 状态值。
- **写后节流** (`index.ts:42-60`)：两个触发器（`writeEveryEvents` 计数 / `writeIntervalMs` 间隔）+ 两个强制点（`turn/end` + 会话 disposal 即 live→cold 时刻）。
- **fail-soft** (`index.ts:68-69`)：每次耐久写失败仅 log warning，缓存自愈于下次写或冷读——丢失写只意味着下次冷读更长尾重放。
- **冷读阶梯** (`index.ts:119`)：`cached row → persistence readFrom tail → registry restore → durable write-back`。
- **身份校验** (`index.ts:101-105`)：`recordFor` 校验记录的 `identity` 与 `expected` 匹配——会话 id 命名槽位而非生命周期，重建的 id 或持久化存储在存活缓存下交换不能让旧记录种子不相关日志折叠出的状态。

- **版本失配**: `ver` mismatch 丢弃行而非迁移——`stateVersion` bump 让旧单元的持久化行被丢弃而非前向折叠成垃圾。

14.7.5 设置存储 (settings-file)

代码: `packages/settings/settings-file/src/index.ts`。单文档多命名空间:

- **位置**: 默认 `<DSH_HOME>/settings.yaml` (或 `.json`) ; `resolveSpec` (`index.ts:55-67`) 解析格式。
- **写** (`index.ts:81-92`) : `patchNode` 递归 map 做保评论叶子级 diff——未触碰节点与已改变 pair 的键节点保留评论、锚点、格式; 非 map 值 (数组/标量) 整体替换。
- **跨进程锁**: `withFileLock` + `writeFileAtomic` , 每次写重新读文档再打补丁。
- **热重载**: chokidar 监视 + `debounceMs=100` 写沉淀窗口; `operations` Promise 链串行化重载与写, 避免写渲染并发重载正在替换的文本。
- **自写抑制**: `text` 缓存上次成功解析/持久化的原文; watcher 事件内容等于此缓存即 no-op。

14.7.6 凭据存储 (credentials-local)

代码: `packages/credentials/credentials-local/src/index.ts`。分层信任 + 引用语义:

```
inherited process environment    (只读, 胜出)
> $DSH_HOME/.credentials.yaml    (provider 管理, 可写)
> <invocation cwd>/.env          (只读 fallback)
> $DSH_HOME/.env                 (只读 fallback)
```

- **环境胜出** (`index.ts:6-19`) : `DEEPSEEK_API_KEY=... dsh`、CI secret、容器 `-e` 是本次运行的显式意图, 不能从内部编辑, 所以必须**可见地**只读而非静默影子写。
- **托管存储胜过 .env**: Models 页面写的 key 立即生效, 即使旧 key 在用户 `.env` 中。
- **严格映射**: `.credentials.yaml` 是 `CredentialRef → string` 严格映射, **不是** dotenv 文件——Harness 拥有且从不物化到环境的存储不能同时充当用户的环境层。
- **权限校验** (`index.ts:103-122`) : `assertOwnerOnly` 在读内容前拒绝 group/other 可读文件 (POSIX `GROUP_OTHER_BITS = 0o077`) ; Windows 跳过 (ACL 不可表达) ; 提示 `chmod 600` 。
- **写**: 与 settings 相同的 `withFileLock` + `writeFileAtomic` + 保评论 diff + 0600。
- **YAML 错误描述** (`index.ts:135-140`) : `describeYamlError` 不引用源文本——解析器消息嵌入出错行, 而行里持有 secret。

14.8 数据一致性保证

14.8.1 事件溯源不变式

不变式	机制
模型可见即已记录	任何到达模型请求的内容必须能从会话日志重建。 <code>Session.append</code> 用 <code>isJsonValue</code> 运行时校验所有事件数据；新增模型可见输入需扩展 <code>SessionEventMap</code> 并从日志渲染。运行时不变式断言此约束。
追加只写	<code>Session.log</code> 只通过 <code>append</code> 增长；事件 <code>deepFreeze</code> 不可变； <code>eventsSnapshot</code> 在 <code>append</code> 时失效。
事件序列单调	<code>seq = log.length</code> （ <code>append</code> L628），单调递增；撕裂尾修复在已提交区检测 <code>seq</code> 间隙即抛错。
表面一致性	<code>surfaceManager.validateNext</code> 校验事件序列合法性； <code>sourceEventSeqs</code> 记录事件引用的源（如 <code>chunk→message</code> ）。
种子边界耐久	<code>session/end-seed</code> 是 <code>firstLiveSeq</code> 的耐久投影； <code>resume/replay</code> 据此区分父历史与子工作。

14.8.2 双后端一致语义

JSONL 与 SQLite 后端实现同一 `PersistenceBackend` 接口，崩溃语义一致：

语义	JSONL	SQLite
撕裂尾检测	<code>SessionLogScanner</code> ：无尾换行的最终记录视为撕裂尾	<code>scanRows</code> ：JSON 解析失败或 <code>seq</code> 间隙在最后 <code>turn/end</code> 之后停止
已提交损坏	通过最后 <code>turn/end</code> 的洞抛错	通过最后 <code>turn/end</code> 的洞抛错
修复	<code>commitRepair</code> ： <code>truncate(offset)</code> + <code>fsync</code> + <code>append closers</code>	<code>commitRepair</code> ： <code>DELETE WHERE seq >= tornMarker</code> + <code>INSERT closers</code> in one tx
物化原子性	<code>materialize</code> ：temp-write + <code>fsync</code> + <code>link()</code> + <code>dir fsync</code>	<code>appendBatch</code> ： <code>BEGIN</code> + <code>writeRow</code> + <code>INSERT events</code> + <code>COMMIT</code>
目录耐久	POSIX: <code>syncDirPosix</code> ；Windows: <code>MoveFileExW(MOVEFILE_WRITE_THROUGH)</code>	SQLite WAL journal
list 不解析全日志	<code>parseHeaderMeta</code> 仅头行	<code>SELECT * FROM sessions</code>

14.8.3 持久化协调器不变式

`PersistenceCoordinator` （`packages/session/session-persistence/src/coordinator.ts`）保证：

- 写后批合并：`SessionWriteBehind` （`write-behind.ts`）固定 200ms 窗口批合并；失败保留批次在 `pending` 头部，`automaticPaused` 暂停自动路径直至 `flush`。
- barrier 一致性：`flush` 的 `drainBarrier` 等待在飞写后逐批排空，在观察到空队列的同一 `job` 中关闭 `barrier` 入口，避免后来 `enqueue` 被搁置在已结算 `barrier` 后。
- 版本拒绝：`sessionFormatVersionRefusal` （`coordinator.ts:77-81`）方向感知——更新版本提示「升级 harness」，更旧版本提示「无升级路径」。

14.9 性能优化

优化	位置	效果
写后批合并	<code>SessionWriteBehind</code> (200ms 窗口)	减少磁盘 I/O 次数，合并多个事件为一次 <code>fsync</code> 。
chunk 打包	<code>packChunkRuns</code> (JSONL <code>packChunks: true</code> 默认)	连续 <code>assistant/chunk delta</code> 打包为 <code>text-chunks</code> / <code>reasoning-chunks</code> 行，实测约 60% 日志压缩。
Zstandard 压缩	JSONL 默认 <code>compression: 'zstd'</code>	帧式压缩，独立可解码的首帧含头记录。
seek-capable 后缀读	SQLite <code>loadStoredFrom</code>	<code>SELECT WHERE seq >= fromSeq</code> , <code>readFrom</code> 随后缀规模扩展。
头行只读 list	JSONL <code>parseHeaderMeta</code> / SQLite <code>SELECT * FROM sessions</code>	会话选择器随会话数扩展，不随每会话大小。
FTS5 索引	<code>persisted_docs</code> / <code>live_docs</code> (<code>unicode61 tokenizer</code>)	全文检索跨会话/事件，UNINDEXED 列减少索引体积。
派生索引丢弃重建	<code>resetDerivedSchema</code> (schema 版本不匹配)	派生索引无迁移负担，随时可重建。
投影缓存	<code>session-projection-cache</code>	冷读阶梯：缓存命中避免全日志重放； <code>ver</code> 校验避免垃圾折叠。
投影未变引用	<code>ProjectionDefinition.apply</code> 返回同引用	<code>Object.is</code> 不变产生零下游工作。
事件快照缓存	<code>Session.eventsSnapshot</code>	<code>events()</code> 多次读共享一份；append 时失效。
投影驱动水位居中	<code>asOfSeq</code> 共享水位	一致读切跨所有注册单元。
STRICT 表	SQLite <code>STRICT</code> 表声明	列类型严格校验，避免弱类型脏数据。

14.10 数据迁移与版本管理

14.10.1 双版本体系

版本	范围	当前值	语义
SESSION_FORMAT_VERSION	会话事件词汇表 (per session)	0	盖戳到每个 SessionHeader.version ; JSONL 头行 + SQLite sessions.version 列。后端拒绝任何其他版本。
SCHEMA_VERSION	SQLite 持久化 schema (表布局)	15	盖戳到 PRAGMA user_version ; 破坏性表布局变更时 bump。不匹配即拒绝，无迁移。
SESSION_QUERY_SQLITE_SCHEMA_VERSION	派生检索索引 schema	8	不匹配 resetDerivedSchema 原地丢弃重建。
ProjectionDefinition.stateVersion	投影单元折叠语义	各单元自定	mismatch 丢弃持久化缓存行而非迁移。
DomainSpec.version	存储 domain 格式	各 domain 自定	介质盖戳不同版本在 open 时 version-mismatch 拒绝。

14.10.2 无迁移策略

项目处于 pre-release，无外部消费者，所以：

- SQLite：只有 pristine 新数据库或当前自有 SCHEMA_VERSION 才能打开；无版本 schema 对象、外部 application identity、所有其他版本被拒绝而非迁移。
- JSONL：只加载已配置编码和当前 SESSION_FORMAT_VERSION ；更改压缩需独立/全新根；预发布格式无迁移。
- 派生索引：版本不匹配即丢弃重建，无迁移负担。
- 投影缓存：ver mismatch 丢弃行，下次冷读从全日志重放。
- 存储 domain：版本 mismatch 在 open 时拒绝。

14.10.3 升级路径

升级 harness 打开旧数据：

- 更新版本 (v > 当前)：提示「log was written by a newer harness — upgrade the harness to open it」。
- 更旧版本 (v < 当前)：提示「older than the supported version, and this build ships no upgrade path for it」。

14.11 安全性设计

14.11.1 凭据引用语义

- 引用而非值：CredentialRef 是 POSIX 标识符 (^[A-Za-z_][A-Za-z0-9_]*\$)，凭据存储是 CredentialRef → string 映射。Harness 拥有的存储从不物化到环境，不能同时充当用户环境层。
- 分层信任：进程环境 (不可写, 胜出) > 托管 .credentials.yaml (可写) > 项目 .env (只读) > DSH_HOME .env (只读)。
- 权限校验：assertOwnerOnly 在读前拒绝 group/other 可读文件 (POSIX 0o077 位检查)；提示 chmod 600。
- 0600 文件模式：创建与替换都以 0600 进行。
- 错误消息脱敏：describeYamlError 不引用源文本，避免解析器消息嵌入的出错行泄露 secret。

14.11.2 进程限制与隔离

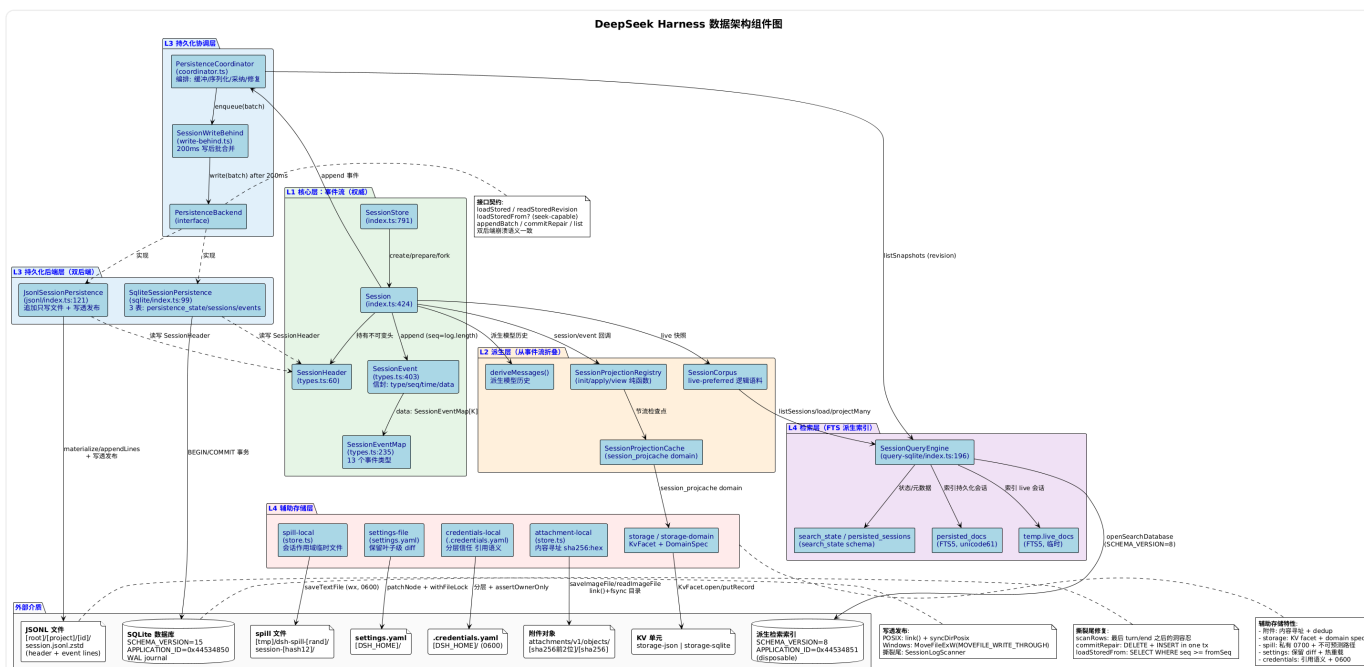
机制	位置	说明
owner-only 文件模式	所有持久化文件 0o600 ； 目录 0o700	阻止其他本地用户读会话日志、附件、spill、设置、凭据。
私有 spill 根	mkdtempSync(tmpdir/dsh-spill-) 0700	不可预测路径，防符号链接种植。
独占创建	open(path, 'wx', 0o600)	spill/attachment 临时文件独占创建，已存在路径（含符号链接）失败。
内容寻址附件	sha256:hex ID + digest 校验	附件 ID 即内容指纹，读取时重新校验 digest，不匹配抛 ATTACHMENT_CORRUPT 。
路径编码	encodeSegment 单射编码	SessionId 等未校验 branded string 在路径使用前中和 ../、绝对路径、NUL、分隔符。
Windows 耐久命名空间	MoveFileExW(MOVEFILE_WRITE_THROUGH)	同卷移动，无 copy fallback，目的地必须不存在。
SQLite application_id	0x44534850 / 0x44534851	保护无关数据库免被持久化写入覆盖。
STRICT 表	SQLite STRICT 声明	列类型严格校验，防弱类型注入。
参数化 SQL	buildSessionWhere / buildEventWhere 返回 SqlWhere	所有用户输入走占位符绑定，防 SQL 注入。
谓词预算	SQLITE_FTS5_OUTER_PREDICATE_LIMIT=14 、 SQLITE_PORTABLE_VARIABLE_LIMIT=32766	防止 FTS5 规划器超预算、跨平台兼容。
YAML 不引用源	describeYamlError	凭据 YAML 解析错误不泄露 secret。

14.11.3 跨进程安全

- 跨进程写锁： withFileLock （settings/credentials）保证多进程并发写不互相覆盖。
- link() 而非 rename()：JSONL 物化用 link() ，两进程并发物化同一 id 时 EEXIST 失败而非静默覆盖。
- incarnation 身份：SQLite sessions.incarnation （ randomUUID() ）让 readStoredRevision 区分同一 id 下的不同生命周期。
- store_id： persistence_state.store_id 单例标识存储身份，让 revision 跨重开可比较。

14.12 数据架构组件图

详见 data-architecture.puml。组件分层：



数据架构组件图

- 核心层: Session、SessionStore、SessionEventMap、SessionHeader、SessionEvent
- 持久化协调层: PersistenceCoordinator、SessionWriteBehind、PersistenceBackend 接口
- 持久化后端层: JsonlSessionPersistence、SqliteSessionPersistence
- 派生层: SessionProjectionRegistry、SessionProjectionCache
- 检索层: SessionQueryEngine、SqliteSessionQueryEngine、SessionCorpus
- 辅助存储层: attachment-local、storage / storage-domain、spill-local、settings-file、credentials-local

14.13 关键发现总结

1. **事件溯源是核心**：会话状态是 `SessionEvent` 追加只写日志的派生物；所有派生视图（模型历史、投影、FTS 索引）从事件流折叠，丢失可重建。
2. **双后端等价语义**：JSONL 与 SQLite 实现同一 `PersistenceBackend` 接口，崩溃撕裂尾修复语义一致（最后 `turn/end` 之后的洞容忍，之前的洞抛错）。
3. **Pre-release 无迁移**：所有 schema 版本不匹配即拒绝，派生索引丢弃重建——优先正确基础而非兼容垫片。
4. **写透发布平台分化**：POSIX 用 `link()` + 目录 fsync；Windows 用原生 `MoveFileExW(MOVEFILE_WRITE_THROUGH)` ——因为 Node 不暴露 Windows 父目录 fsync 契约。
5. **模型可见即已记录**：`Session.append` 用 `isJsonValue` 在源头拒绝非可序列化数据，保证耐久日志回放相同事件。
6. **内容寻址附件**：附件 ID 即 sha256，读取时 digest 校验，dedup 路径处理 `EEXIST`，目录 fsync 关闭并发桶创建窗口。
7. **投影缓存 fail-soft**：派生状态检查点失败仅 warning，下次冷读从全日志重放自愈——缓存是快捷方式非权威。
8. **凭据分层信任**：进程环境胜出（不可写显式意图）> 托管 `.credentials.yaml`（可写）> `.env`（只读 fallback）；严格映射而非 dotenv，避免影子非 secret 条目。
9. **FTS5 派生索引可弃**：`SESSION_QUERY_SQLITE_SCHEMA_VERSION` 不匹配即 `resetDerivedSchema` 重建；live-preferred corpus 优先 live session 避免可选后端故障。
10. **双版本体系**：`SESSION_FORMAT_VERSION`（事件词汇表，per session）与 `SCHEMA_VERSION`（SQLite 表布局，per database）正交，分别盖戳到头行/`sessions.version` 列与 `PRAGMA user version`。

15 第三方集成总览

本文档基于 AST 符号工具对源码符号进行验证分析生成。代码引用处的行号均来自符号工具验证。

15.1 集成概述

DeepSeek Harness (dsh) 是「一切皆插件」的 Agent 框架。第三方集成的核心设计模式是 **能力接缝 (Capability Seam) 三角色**：

- **Service Definition (接口声明)**：抽象 service + 词汇 + 错误分类，零运行时依赖。
- **Service Provider (实现)**：注册具体 adapter / provider 到 `ctx.<capability>`。
- **Consumer (消费方)**：通过 `ctx.<capability>` 调用能力（通常是面向模型的工具）。

每个第三方服务集成严格遵循此模式，让 dsh 在不修改核心代码的前提下扩展新 LLM provider、新沙箱 backend、新协议、新 hook dialect。

dsh 的第三方集成可分为四大类：

类别	包数	文档
LLM 与 AI 框架	5 个子包	01-llm-integration.md
沙箱与执行环境	4 个子包 + 1 个原生 addon	02-sandbox-execution.md
协议集成	8 个子包	03-protocols.md
框架与构建工具	3 个 vendored + 多个工具链	04-framework-vendor.md

15.2 集成类别列表

15.2.1 LLM 与 AI 框架 (详见 01-llm-integration.md)

包	角色	第三方依赖
@deepseek-ai/dsh-llm	Service Definition	(无第三方)
@deepseek-ai/dsh-llm-deepseek	Provider (直连 fetch + SSE)	eventsourcing-parser@3.1.0
@deepseek-ai/dsh-llm-pi-ai	Provider (pi-ai 库多协议)	@earendil-works/pi-ai@0.82.1 (含 @google/genai)
@deepseek-ai/dsh-llm-retry	Consumer (agent loop 失败恢复)	(无第三方)
@deepseek-ai/dsh-token-meter	Consumer (replay-aware 计量)	zod@4.4.3

核心抽象： `LlmAdapter` (`packages/llm/llm/src/index.ts:180`) 只有一个必需方法 `stream(options)`。

15.2.2 沙箱与执行环境（详见 02-sandbox-execution.md）

包	角色	第三方依赖
@deepseek-ai/dsh-sandbox	Service Definition	(无第三方)
@deepseek-ai/dsh-sandbox-local	Provider (本地多 runner 链)	@deepseek-ai/node-addon-landlock-run@workspace:^
@deepseek-ai/dsh-sandbox-windows-acl	Provider (Windows ACL FFI)	koffi@^3.1.0
@deepseek-ai/dsh-sandbox-policy	Consumer (per-call policy 解析)	(无第三方)
@deepseek-ai/node-addon-landlock-run	原生 addon (C + 预编译二进制)	BSD-3-Clause, Linux only
@deepseek-ai/dsh-e2b + dsh-fs-e2b + dsh-subprocess-e2b	Provider (远程沙箱 POC)	e2b@2.29.1
(node-pty)	持久 PTY 后端	node-pty@1.1.0 (patched)

核心抽象: SandboxProvider.confine(argv, policy) 返回 ConfinedArgv , fail-closed 不允许 unconfined passthrough。

15.2.3 协议集成（详见 03-protocols.md）

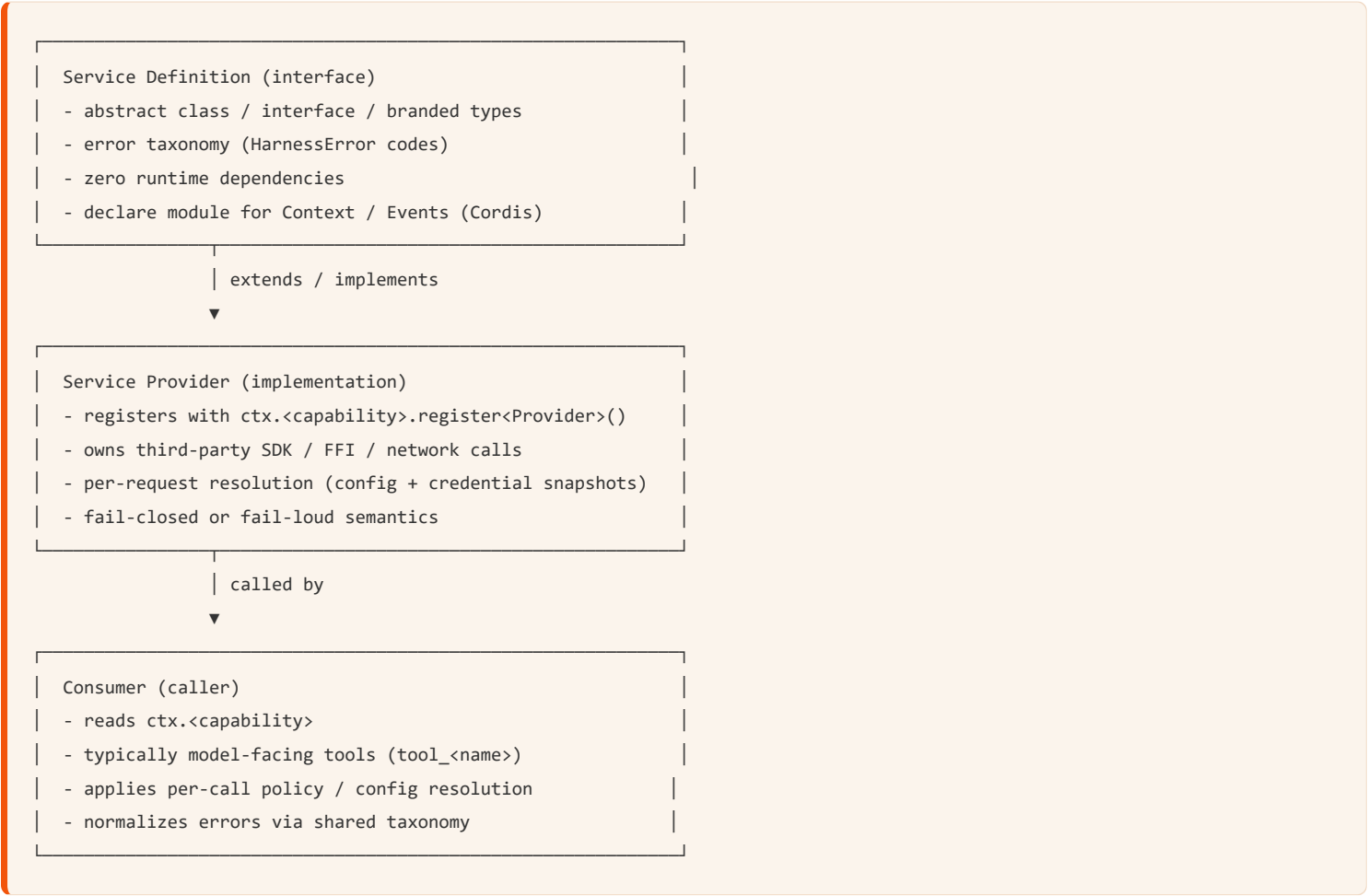
包	角色	第三方依赖
@deepseek-ai/dsh-acp	Provider (ACP server)	@agentclientprotocol/sdk@0.25.1
@deepseek-ai/dsh-sdk-protocol	Service Definition (共享 wire)	(无第三方)
@deepseek-ai/dsh-sdk-client	Client (TS)	(无第三方)
@deepseek-ai/dsh-sdk-jsonrpc-server	Provider (SDK server)	(无第三方)
@deepseek-ai/dsh-hook-protocol	共享 hook 库	(无第三方)
@deepseek-ai/dsh-hooks-claude-code	Bridge (Claude Code hooks)	(无第三方)
@deepseek-ai/dsh-hooks-codex	Bridge (Codex hooks)	(无第三方)
(Python SDK)	Client (Python twin)	(在 python/sdk)

15.2.4 框架与构建工具（详见 04-framework-vendor.md）

类别	包	角色
Vendored 框架	@deepseek-ai/cordis@4.0.1	Meta-Framework（插件骨架）
Vendored 框架	@deepseek-ai/cosmokit@1.8.2	Common utilities
Vendored 框架	@deepseek-ai/schemastery@3.18.1	Schema validator
工具链	pnpm 11.7.0	Workspace monorepo
工具链	TypeScript 6.0.3 + tsx 4.22.4	类型 + ESM 执行
工具链	tsdown 0.22.2 + esbuild	库打包
工具链	oxlint 1.76.0 + oxlint-tsgolint 7.0.2001	Linters
工具链	vitest 4.1.8	测试框架
工具链	knip 6.16.1 + publint 0.3.21 + jscpd 5.0.12	卫生检查
工具链	lefthook 2.1.9	git hooks

15.3 统一接口设计：能力接缝三角色

15.3.1 三角色定义



15.3.2 各能力接缝对照

能力	Service 类	Provider 注册方法	ctx 字段	典型 Consumer
LLM	LlmAdapter (llm/src/index.ts:180)	ctx.llm.registerAdapter([providers], adapter) (:337)	ctx.llm	dsh-agent-loop , dsh-token-meter
LLM configurable provider dir	(无独立类)	ctx.llm.registerConfigurableProviders(entries) (:430)	(同上)	settings page
LLM model discovery	(callback)	ctx.llm.registerModelDiscovery(ns, fn) (:503)	(同上)	settings page
Sandbox	SandboxProvider (sandbox/src/index.ts:158)	(extends + super(ctx, 'sandbox'))	ctx.sandbox	dsh-shell , dsh-subprocess , dsh-code-runtime
Sandbox policy	(callback)	(consumer 解析 per-call policy)	ctx.sandboxPolicy	(consumer 内部)
E2B	E2BRuntime (e2b/src/index.ts:74)	(extends + super(ctx, 'e2b'))	ctx.e2b	dsh-fs-e2b , dsh-subprocess-e2b
Token meter	TokenMeter (token-meter/src/index.ts:74)	(extends + super(ctx, ...))	ctx.tokenMeter	dsh-compaction , UI
Web search	WebSearchProvider	ctx.web.registerSearchProvider(...)	ctx.web	dsh-tool-web
Web fetch	WebFetchProvider	ctx.web.registerFetchProvider(...)	ctx.web	dsh-tool-web
LSP	LspProvider	ctx.lsp.registerProvider(...)	ctx.lsp	dsh-tool-lsp
ACP server	(function plugin)	apply(ctx, config)	(无 ctx field)	外部 ACP 客户端
SDK server	HarnessSdkJsonRpcServer (sdk/server/src/server.ts:52)	(plugin apply)	(无 ctx field)	外部 SDK 客户端
Hook bridges	(function plugin)	apply(ctx, config) 注册到 interception 扩展点	(无独立 ctx field)	agent loop, tool exec

15.3.3 注册与销毁

所有 provider 注册都通过 `ctx.effect()` / `registration handle` 实现可逆 **effect**:

- 注册时返回 `disposer` (如 `AdapterRegistrationHandle` , `llm/src/index.ts:238`)。
- `disposer` 既可作函数调用 (释放所有路由) , 也提供 `replace(providers)` 原子替换路由集。
- plugin 卸载时 `cordis` 自动调用 `disposer` , 无需手动 `cleanup`。
- `ctx.on(event, listener)` 注册的事件监听同样可逆。

15.3.4 凭据引用模式

dsh 的核心安全设计: **配置永不承载 secret**。

- 配置字段 `apiKeyEnv: string` (role: `'credential-ref'`) 只存环境变量名。
- 运行时通过 `ctx.get('credentials')` 接缝或 `launchEnvironmentOf(ctx).get(ref)` 解析。
- `secret` 永不进入 `settings` 文档、永不入日志、永不入 `session event`。
- `assertUsableApiKey(raw, pkg, ref)` (`llm/src/index.ts:137`) 共享校验逻辑, 错误消息引导用户修改 `ref` 而非回显 `secret`。

15.4 环境变量清单

环境变量	类别	作用	默认 / fallback
<code>DEEPSEEK_API_KEY</code>	LLM	DeepSeek chat-completions API key	必填
<code>DEEPSEEK_BASE_URL</code>	LLM	DeepSeek chat-completions endpoint	<code>https://api.deepseek.com</code>
<code>DEEPSEEK_SEARCH_BASE_URL</code>	Web	DeepSeek web search endpoint (Anthropic-compatible)	(独立于 chat-completions)
<code>OPENAI_API_KEY</code>	LLM (pi-ai)	OpenAI 路由凭据 (profile 指定)	—
<code>ANTHROPIC_API_KEY</code>	LLM (pi-ai)	Anthropic 路由凭据	—
<code><provider>_API_KEY</code>	LLM (pi-ai)	自定义 provider 凭据	—
<code>E2B_API_KEY</code>	Sandbox	E2B 远程沙箱 API key (POC)	必填
<code>DSH_BUILD_FACE</code>	Build	tsdown 构建目标 face	<code>host</code> / <code>client</code>
<code>DSH_SNAPSHOT</code>	Test	snapshot 测试模式	<code>record</code> / <code>refresh</code> / <code>replay</code>

注: 沙箱 native addon (`landlock-run`) **故意不提供任何环境变量 override**, 防止 ambient env 控制哪个 binary 限制进程。

15.5 扩展指南

15.5.1 添加新 LLM Provider

参考 `01-llm-integration.md §7`。

15.5.2 添加新沙箱 Backend

参考 `02-sandbox-execution.md §7`。

15.5.3 添加新协议或 Hook Dialect

参考 03-protocols.md §7。

15.5.4 通用扩展流程

无论添加什么新能力，遵循以下步骤：

1. **新建 Service Definition 子包** `packages/<group>/<capability>/` :
 - 抽象 `Service` 子类或纯 interface。
 - 错误 taxonomy (继承 `HarnessError`)。
 - `declare module '@deepseek-ai/cordis'` 扩展 `Context` 接口。
 - 零运行时第三方依赖。
2. **新建 Provider 子包** `packages/<group>/<capability>-<provider>/` :
 - 继承抽象类，实现必需方法。
 - 在 `apply(ctx, config)` 中注册到 `ctx.<capability>`。
 - 用 `schemastory` 定义 `Config` schema。
 - 凭据字段用 `role: 'credential-ref'`，secret 字段用 `role: 'secret'`。
3. **新建 Consumer 子包** (如需模型可见工具) `packages/<group>/tool-<capability>/` :
 - 通过 `ctx.<capability>` 调用能力。
 - 应用 per-call policy (来自 `ctx.sandboxPolicy` 或类似)。
 - 错误用 `HarnessError` 包装为 `tool/result` structured error。
4. **加入 bundle**：在对应 `cordis.yml` (如 `apps/cli/cordis.yml` 或 `examples/<bundle>/cordis.yml`) 加 plugin entry。
5. **测试**：
 - 用 `dsh-test-support` 的 testkit / invariant 检查。
 - 跨平台 gate (Linux + Windows + Wine)。
 - snapshot 测试 (如适用)。
6. **文档**：
 - 子包 `README.md` + `README.zh.md` (i18n)。
 - 加入 `gen-catalog` 系列 (如 `gen-tool-catalog`)。
 - 加入 `verify-package-invariants` 校验。
7. **卫生检查**： `npm run hygiene` 全套 (knip + publint + constraints + licenses + invariants + cordis-config + node-types + runtime-closure + vendored-links)。

15.6 关键设计原则

15.6.1 接缝优先

核心包零运行时第三方依赖，所有 SDK / FFI / 网络调用下沉到 Provider 子包。这让接缝稳定、Provider 可独立演进。

15.6.2 Per-request Resolution

LLM adapter 的连接事实和凭据每请求解析一次，in-flight 流不观察配置变化。沙箱 ACL grant per-session 隔离。配置错误保留 lastGood 而非 crash running session。

15.6.3 Fail-Closed / Fail-Loud

- 沙箱：无可 backend 抛 `SANDBOX_UNAVAILABLE`，命令永不 unconfined 运行。
- 凭据：缺失抛 `MISSING_CREDENTIAL`，不静默 fallback。
- AclGrant：半 materialized 失败时撤销 + 清理后才抛。

- 重试: `retryableCodes` 配置不可为空。

15.6.4 Atomic Replace

- LLM `AdapterRegistrationHandle.replace(providers)`: 原子替换路由集, 无空窗。
- Windows ACL `AclWriteGrant`: standing workspace ACE 跨 session 复用, 避免 dispose-then-register 留空窗。

15.6.5 No Secret in Config

所有 `apiKeyEnv` 是环境变量名, secret 永不入配置 / 日志 / session event。错误消息引导用户修改 `ref` 而非回显 secret。

15.6.6 Contracts-Ready Pattern

`build:lib:host && gate:contracts-ready` 模式贯穿 typecheck / lint / doc-typecheck, 因为 dsh 的 type contracts 跨包生成 (declaration merging、`SessionEventMap` 等)。

15.7 关键发现

- Vendored Cordis**: scope 从 `@cordiverse/*` 改为 `@deepseek-ai/*`, `linkWorkspacePackages: true` + `overrides` 确保本地源码 override 远程。便于在框架层做 in-tree 修复。
- allowBuilds 严格模式**: pnpm 10+ 默认拒绝所有 install/build script, dsh 显式 allow 仅需要的 (esbuild / lefthook / node-pty / koffi), 减少供应链攻击面。
- patchedDependencies**: node-pty@1.1.0 通过 patch 文件修复, 无需 fork 维护。
- landlock-run 二进制无 env override**: 故意设计, 防止 ambient env 控制哪个 binary 限制进程 (安全考虑)。
- dsh-llm-pi-ai 是 design-verification twin**: 与 dsh-llm-deepseek 共享 `LlmAdapter` 契约, 覆盖 OpenAI / Anthropic / 自定义网关, 是设计验证而非主流。
- dsh-sdk 与 ACP 平行**: ACP 是 automation-only 标准 (外部 trusted 客户端), SDK 是 dsh 自有完整客户端协议 (含 subagent 一等支持)。
- Hook bridge 是兼容层**: 执行未修改的 Claude Code / Codex hook 命令, shared execution 在 `dsh-hook-protocol`, 每个 dialect bridge 只 own payload / env / mapping。
- restrictive merge**: 多 hook 输出合并采取最严格决策 (block 优先), 保证安全。
- SDK client 关闭 ladder**: EOF → SIGTERM → SIGKILL, 加 grace 让 stdio 沉淀——子进程关闭稳健模式。
- 大量 verify 脚本构成卫生检查网**: 从 `verify-md-links` 到 `verify-runtime-closure`, 确保 monorepo 健康度但需维护成本。

16 LLM 与 AI 框架集成

本文档基于 AST 符号工具对源码符号进行验证分析生成。代码引用处的行号均来自符号工具验证。

16.1 集成概述

dsh 的 LLM 能力是「能力接缝三角色」模式的典型示例:

- Service Definition (接口声明)**: `@deepseek-ai/dsh-llm` 声明 `LlmAdapter` 抽象类、`LlmRuntime` 注册表与 `llm/stream` waterfall 事件。
- Service Provider (实现)**: `dsh-llm-deepseek` (直连 fetch + SSE)、`dsh-llm-pi-ai` (基于 pi-ai 库的多协议适配器) 注册具体 Adapter。
- Consumer (消费方)**: `dsh-agent-loop`、`dsh-token-meter`、`dsh-llm-retry` 等通过 `ctx.llm` 调用模型流。

每个 Provider 都有自己的 retryPolicy, 但统一的执行器 `dsh-llm-retry` 在 agent loop 失败恢复扩展点上调度重试。

16.1.1 设计哲学

- 接缝优先: 核心包 `dsh-llm` 完全 `provider-neutral`, 不含任何 HTTP 或 SDK 依赖。
- 凭据引用: 配置仅承载 `apiKeyEnv` (环境变量名), 永不承载 `secret` 本身。
- 每请求解析: 连接事实和凭据在一次 `stream` 调用内冻结, 避免 `in-flight` 流观察到配置变化。
- 可重启路由: `AdapterRegistrationHandle.replace()` 原子替换路由集, 无空窗。

16.2 支持的服务/产品

服务名	SDK / API	版本	状态	说明
DeepSeek Chat Completions	直连 fetch + SSE	OpenAI 兼容	GA	默认 Provider, <code>deepseek-official</code> 路由, V4-Flash / V4-Pro
OpenAI	<code>@earendil-works/pi-ai</code>	0.82.1	可选	通过 <code>pi-ai</code> 适配器, <code>openai-completions</code> 协议
Anthropic	<code>@earendil-works/pi-ai</code>	0.82.1	可选	<code>anthropic-messages</code> 协议, Claude 模型
OpenAI Responses	<code>@earendil-works/pi-ai</code>	0.82.1	可选	<code>openai-responses</code> 协议
Bedrock / Vertex / Azure	<code>pi-ai catalog</code>	0.82.1	受限	<code>pi-ai</code> 自带 <code>catalog</code> provider, 但本适配器无 <code>SigV4/OAuth/ADC</code> 配置形状, 需 <code>route</code> 自带 <code>auth</code>
自定义 OpenAI 兼容网关	<code>pi-ai createProvider</code>	—	可选	<code>acme-gateway</code> 路由示例: 自定义 <code>baseURL</code> + <code>api: openai-completions</code>
DeepSeek Web Search	Anthropic Messages API	<code>web_search_20250305</code>	可选	见 <code>dsh-web-search-deepseek</code> , 与 <code>chat-completions</code> 使用不同的 <code>baseURL</code>

16.3 集成方式

16.3.1 Service Definition: LlmRuntime 与 LlmAdapter

`LlmRuntime` (`packages/llm/llm/src/index.ts:283`) 是注册表中枢。它声明 `ctx.llm` (`index.ts:48`) 和 `llm/stream` `waterfall` 事件 (`index.ts:64`), 核心方法:

- `registerAdapter(providers, adapter): AdapterRegistrationHandle` (`index.ts:337`): 注册一个 `Adapter` 实例到多个 `provider` 路由, 返回带 `replace()` 的 `disposer`。
- `registerConfigurableProviders(entries)` (`index.ts:430`): 声明可配置 `Provider` 目录 (settings page 显示)。
- `registerModelDiscovery(namespace, fn)` (`index.ts:503`): 注册端点模型发现回调。
- `prepareCall(options)` (`index.ts:778`): 返回 `PreparedLlmCall`, 捕获 `config` + `retryPolicy` + `adapterDefaults`。
- `stream(options)` (`index.ts:912`): 触发 `llm/stream` `waterfall` 后调用 `adapter`。

LlmAdapter (index.ts:180) 抽象类只有一个必需方法:

```
abstract stream(options: GenerateOptions): AsyncIterable<StreamChunk>
```

可选 override: providerInfo (index.ts:186) 、 providerRetryPolicy (index.ts:195) 、 listModels (index.ts:206) 、 resolveModel (index.ts:219) 。

16.3.2 Provider: DeepSeekAdapter (直连 fetch)

DeepSeekAdapter (packages/llm/llm-deepseek/src/adapter.ts:158) 继承 LlmAdapter , 是第一个原生实现:

- **transport-only**: 连接事实通过 thunk 每操作解析一次, bearer token 通过 per-request resolver, 注册插件拥有验证、分层、凭据策略。
- 不依赖任何 SDK, 直接 fetch() DeepSeek 的 /chat/completions 。
- 使用 eventsource-parser 解析 SSE 流 (packages/llm/llm-deepseek/src/sse.ts:14) 。
- stream(options) (adapter.ts:213) : 单次 resolution 冻结连接事实和凭据快照, 复用 idleWatchdog 监控空闲。
- httpErrorCode(status, error) (adapter.ts:138) : 把 HTTP 状态码映射到稳定 LlmError code (401/403→ AUTH、429→ RATE_LIMIT、5xx→ SERVER、quota_deceeded → QUOTA_EXCEEDED) 。

构造选项 DeepSeekAdapterOptions (adapter.ts:74) :

```
interface DeepSeekAdapterOptions {
  options: () => DeepSeekConnectionOptions // 每操作调用一次
  resolveApiKey: (connection: DeepSeekConnectionOptions) => Promise<string>
  resolveUserId: () => AnonymousUserId
}
```

连接事实 DeepSeekConnectionOptions (adapter.ts:49) 包含 baseUrl 、 apiKeyEnv: CredentialRef 、 defaults 、 maxTokens 、 defaultContextWindow 、 models 、 streamIdleTimeoutMs 、 retryPolicy 。

16.3.3 Provider: PiAiAdapter (pi-ai 库)

PiAiAdapter (packages/llm/llm-pi-ai/src/adapter.ts:186) 是 dsh-llm-deepseek 的 design-verification twin:

- 基于 @earendil-works/pi-ai 库, 支持多 provider 路由共享一个 adapter 实例。
- 路由命名 pi-ai 已安装的 provider 时继承其 endpoint/protocol/catalog; 命名 pi-ai 未识别的 key 时整体声明。
- current() (adapter.ts:199) : memoize snapshot, profiles 引用相等即复用。
- stream(options) (adapter.ts:275) : 路由到对应 pi-ai provider 的 stream。
- providerInfo (adapter.ts:227) : 使用配置的 displayName 而非路由 key。

支持协议表 (packages/llm/llm-pi-ai/src/provider.ts:47) :

```
const PROTOCOLS: Readonly<Record<string, () => ProviderStreams>> = {
  'openai-completions': openAICompletionsApi,
  'openai-responses': openAIResponsesApi,
  'anthropic-messages': anthropicMessagesApi,
}
```

buildProvider(spec) (provider.ts:167) 构造 pi-ai Provider: catalog 路由 + 同协议 → reuseCatalogProvider (provider.ts:144) 保留原生实现; 其他 → createProvider 通过 PROTOCOLS 表构造。

16.3.4 凭据解析

两个 adapter 共用 assertUsableApiKey(raw, pkg, ref) (packages/llm/llm/src/index.ts:137) :

- 从 `dsh-credentials` 接缝或环境变量获取原始字符串。
- 拒绝空字符串和非 HTTP-header 安全字符。
- 错误消息引导用户修改 `ref` 指向的环境变量，绝不回显 `secret`。
- 缺失时抛 `LlmError('MISSING_CREDENTIAL')`。

`dsh-llm-pi-ai` 多了一个 detail: profile 不命名 `apiKeyEnv` 时返回 `undefined`，让 pi-ai 的 provider-native ambient discovery 生效 (OAuth 等)。一旦命名就 fail-loud，避免 pi-ai 拾取无关的 ambient key (如 `OPENAI_API_KEY`)。

16.4 代码实现

16.4.1 关键类与文件

类 / 模块	路径	行号	角色
<code>LlmRuntime</code>	<code>packages/llm/llm/src/index.ts</code>	283-927	Adapter 注册表、waterfall 调度
<code>LlmAdapter</code>	<code>packages/llm/llm/src/index.ts</code>	180-232	抽象 Adapter 基类
<code>LlmError</code>	<code>packages/llm/llm/src/index.ts</code>	83	类型化错误 (AUTH/RATE_LIMIT/NO_ADAPTER 等)
<code>assertUsableApiKey</code>	<code>packages/llm/llm/src/index.ts</code>	137	凭据校验共享逻辑
<code>AdapterRegistrationHandle</code>	<code>packages/llm/llm/src/index.ts</code>	238	注册句柄 (disposer + replace)
<code>DeepSeekAdapter</code>	<code>packages/llm/llm-deepseek/src/adapter.ts</code>	158-345	直连 fetch 适配器
<code>DeepSeekConnectionOptions</code>	<code>packages/llm/llm-deepseek/src/adapter.ts</code>	49-71	连接事实快照
<code>httpErrorCode</code>	<code>packages/llm/llm-deepseek/src/adapter.ts</code>	138	HTTP 状态到 LlmError code 映射
<code>parseSse</code>	<code>packages/llm/llm-deepseek/src/sse.ts</code>	28	SSE 流解析 (基于 eventsource-parser)
<code>PiAiAdapter</code>	<code>packages/llm/llm-pi-ai/src/adapter.ts</code>	186-357	pi-ai 多协议适配器
<code>buildProvider</code>	<code>packages/llm/llm-pi-ai/src/provider.ts</code>	167	pi-ai Provider 构造
<code>TokenMeter</code>	<code>packages/llm/token-meter/src/index.ts</code>	73-310	Replay-aware token 计量
<code>resolveRetryPolicy</code>	<code>packages/llm/llm/src/retry-policy.ts</code>	145	Retry policy 解析

16.4.2 Provider 插件 apply 流程

`dsh-llm-deepseek` 的 `apply(ctx, config)` (`packages/llm/llm-deepseek/src/index.ts:200`)：

1. **memoized resolver**: `options()` 函数捕获 `lastRaw` + `lastGood`，settings snapshot 失败时保留上一个 good 配置。

2. `resolveApiKey`: 通过 `ctx.get('credentials')` 接缝或 `launchEnvironmentOf(ctx).get(ref)` 解析。
3. 构造 `adapter`: `new DeepSeekAdapter({ options, resolveApiKey, resolveUserId })`。
4. `registerConfigurableProviders`: 声明 `deepseek-official` 路由。
5. `registerAdapter`: 注册 `[PROVIDER]` 路由, 捕获 `registration`。
6. `installSettingsSection`: 挂载 `llm-deepseek` settings namespace, `onChange` 触发 `ensureRegistrationFacts` (`retryPolicy` 变化时 `registration.replace([PROVIDER])` 原子替换)。
- `dsh-llm-pi-ai` 的 `apply` (`packages/llm/llm-pi-ai/src/index.ts:150`) 多一步 `directoryEntries` 维护: 合并 catalog provider 和 hand-declared route, 通过 `directory.replace(entries)` 原子更新 configurable-provider 目录。

16.5 配置与环境变量

16.5.1 环境变量

环境变量	作用	默认值 / fallback
<code>DEEPSEEK_API_KEY</code>	DeepSeek chat-completions 的 API key	必填 (无默认)
<code>DEEPSEEK_BASE_URL</code>	DeepSeek chat-completions endpoint	<code>https://api.deepseek.com</code> (<code>PUBLIC_BASE_URL</code> , <code>llm-deepseek/src/index.ts:104</code>)
<code>DEEPSEEK_SEARCH_BASE_URL</code>	DeepSeek web search endpoint (Anthropic-compatible)	见 <code>web-search-deepseek/src/index.ts:82</code> , 区别于 chat-completions
<code>E2B_API_KEY</code>	E2B 远程沙箱 API key (POC)	见 <code>dsh-e2b</code> , 非 LLM
<code><provider>_API_KEY</code> (如 <code>OPENAI_API_KEY</code> 、 <code>ANTHROPIC_API_KEY</code>)	pi-ai 路由凭据	profile 的 <code>apiKeyEnv</code> 字段指定

16.5.2 Plugin Config (`dsh-llm-deepseek`)

`Config` interface (`packages/llm/llm-deepseek/src/index.ts:62`) :

字段	类型	默认	说明
apiKeyEnv	string (credential-ref)	DEEPSEEK_API_KEY	凭据引用
baseUrl	string	DEEPSEEK_BASE_URL 或 PUBLIC_BASE_URL	endpoint base
thinking	'enabled' \ 'disabled'	—	部署 thinking 策略
reasoningEffort	'off' \ 'high' \ 'max'	high	默认 thinking 强度
maxTokens	number	DEFAULT_MAX_TOKENS = 256_000	每请求输出上限
defaultContextWindow	number	DEFAULT_CONTEXT_WINDOW = 1_000_000	模型未声明时使用
models	DeepSeekCatalogModel[]	V4-Flash + V4-Pro	公告模型列表
streamIdleTimeoutMs	number	DEFAULT_STREAM_IDLE_TIMEOUT_MS = 300_000	流读空闲超时
retryPolicy	RetryPolicyConfig	normal defaults	Provider-owned 重试策略

16.5.3 Retry Policy Schema

RetryPolicySchema (packages/llm/llm/src/retry-policy.ts:100) 是 union:

- **normal** (NormalRetryPolicyConfig , retry-policy.ts:36) : mode: 'normal' + maxRetries (默认 2) + retryableCodes (默认 [EMPTY_RESPONSE, RATE_LIMIT, SERVER, TIMEOUT, TRANSPORT]) + backoff 。
- **always** (AlwaysRetryPolicyConfig , retry-policy.ts:48) : mode: 'always' 无限重试直到成功/取消/dispose。
- **BackoffConfig** (retry-policy.ts:27) : initialDelayMs (500) + maxDelayMs (10_000) + jitterRatio (0.1) 对称抖动。

16.6 错误处理

16.6.1 LlmError Code 表

Code	含义	触发场景
AUTH	认证失败	HTTP 401/403
RATE_LIMIT	速率限制	HTTP 429
QUOTA_EXCEEDED	配额耗尽	错误体含 <code>quota</code> 关键字
CONTEXT_WINDOW_EXCEEDED	上下文超长	HTTP 400 + <code>context window</code> 关键字
INVALID_REQUEST	请求无效	HTTP 400 其他情况
SERVER	服务端错误	HTTP ≥ 500
HTTP_<status>	其他 HTTP 错误	未识别状态码
MISSING_CREDENTIAL	凭据缺失	<code>assertUsableApiKey</code> 失败
INVALID_CREDENTIAL_CODE	凭据格式错误	key 含非 HTTP-header 字符
NO_ADAPTER	无适配器	provider 路由未注册
UNKNOWN_MODEL	未知模型	pi-ai provider 无该 model
STREAM_CLOSED	流异常关闭	SSE 未见 <code>[DONE]</code> (<code>sse.ts:39</code>)
LLM_STREAM_IDLE_TIMEOUT	流空闲超时	<code>streamIdleTimeoutMs</code> 触发
EMPTY_RESPONSE_CODE	空响应	见 <code>error.ts</code>
TRANSPORT	传输错误	网络层失败
TIMEOUT	超时	整体请求超时
REGISTRATION_DISPOSED	注册已释放	<code>replace()</code> 在 <code>dispose</code> 后调用

16.6.2 重试策略

`dsh-llm-retry` (`packages/llm/llm-retry/src/index.ts:99`) 在 agent loop 的 `request-error` 扩展点执行：

- `apply(ctx, config, internals)` 注入 `agents` 服务。
- `localDelay(policy, retry, random)` (`index.ts:58`) ：指数退避 + 对称抖动，`Math.min(initialDelayMs * 2^exponent, maxDelayMs) * jitter` 。
- `cancellableDelay(delayMs, signal)` (`index.ts:78`) ：可被 abort signal 取消的 delay。
- 每次调度重试在 wait 前持久化 (durable before cancellable wait) 。
- `retryPolicyKey(policy)` (`index.ts:65`) ：序列化 policy 用于去重 / 比较。

16.6.3 降级：保留上一个 good 配置

`dsh-llm-deepseek` 的 `options()` resolver (`index.ts:204`) 在 settings snapshot 解析失败时：

- 首次失败直接抛出 (无 lastGood 可用) 。
- 后续失败：保留上一个 `lastGood` ，记录 `logger.error` ，继续服务 in-flight 请求。
- 这样配置错误不会让运行中的会话崩溃。

16.7 扩展指南：添加新 LLM Provider

16.7.1 直连方式（参考 dsh-llm-deepseek）

1. 新建子包 `packages/llm/llm-<your-provider>/`，`peerDependencies` 含 `@deepseek-ai/dsh-llm`。
2. 实现 `LlmAdapter` 子类：必填 `stream(options)`，可选 override `providerInfo` / `providerRetryPolicy` / `listModels` / `resolveModel`。
3. 构造 `AdapterOptions`：包含 `options: () => ConnectionOptions`、`resolveApiKey: (conn) => Promise<string>`、`resolveUserId`。
4. 在 `apply(ctx, config)`：
 - 调用 `ctx.llm.registerConfigurableProviders([...])`。
 - 调用 `ctx.llm.registerAdapter([PROVIDER], adapter)` 拿到 `registration`。
 - `installSettingsSection(ctx, NS, Config, config, { setSource, onChange })`。
 - 在 `onChange` 中根据 `retryPolicy` 变化调用 `registration.replace([...])`。
5. 配置 `schema`：用 `schemastery` 定义 `Config`，包含 `apiKeyEnv`（role: 'credential-ref'）、`baseUrl`、`retryPolicy: RetryPolicySchema`。
6. 加入 `bundle`：在 `apps/cli/cordis.yml` 或对应 `cordis.yml` 加入 `- id: llm-<your-provider>`。
7. 测试：用 `dsh-llm-mock-server`（见 `packages/test-support`）做集成测试。

16.7.2 pi-ai 库方式（参考 dsh-llm-pi-ai）

如果新 provider 已被 `@earendil-works/pi-ai` 收录 (catalog provider)：

1. 不需要新 `adapter`：直接在 `dsh-llm-pi-ai` 的 `Config.providers` 下加一条路由，命名 pi-ai 的 provider key 即可继承 `endpoint` / `protocol` / `catalog`。
2. 覆盖字段：用 `models` 数组缩小 `catalog`、`apiKeyEnv` 指定凭据、`api` 覆盖协议、`baseUrl` 重指 `endpoint`。
3. 新增协议：若 pi-ai 未收录，在 `provider.ts:47` 的 `PROTOCOLS` 表加 entry（如 `bedrock` 需要 SigV4 auth，超出当前配置形状不能表达）。

16.7.3 Token 计量

`TokenMeter`（`packages/llm/token-meter/src/index.ts:73`）是 replay-aware service：

- 自动 fold 每个 `StreamChunk` 的 `TokenUsage`，无需新 provider 干预。
- `_estimateProviderAssistant`（`index.ts:276`）在 provider 未返回 usage 时用 `estimateMessage` 估算。
- 新 provider 只需在 `StreamChunk` 中正确产出 `usage` 字段。

16.8 关键 SDK 依赖版本

依赖	版本	用途
<code>@agentclientprotocol/sdk</code>	0.25.1	ACP（非 LLM 直接相关，但 llm-retry 测试用）
<code>@earendil-works/pi-ai</code>	^0.82.1	pi-ai 适配器后端（ <code>llm-pi-ai/package.json</code> ）
<code>@google/genai</code>	(transitive)	由 pi-ai 拉入， <code>allowBuilds: false</code> 拒绝其 lifecycle 脚本
<code>eventsourcing-parser</code>	^3.1.0	DeepSeek SSE 解析（ <code>llm-deepseek/package.json</code> ）
<code>zod</code>	^4.4.3	token-meter schema

16.9 关键发现

1. **唯一必填方法**: `LlmAdapter.stream` 是唯一抽象方法 (`index.ts:232`) , 其他都有默认实现, 最小化新 provider 实现成本。
2. **retryPolicy 在注册时捕获**: runtime 在 `registerAdapter` 时 freeze policy, 运行时改变只能通过 `registration.replace()` 原子重注册。
3. **dsh-llm-pi-ai 是 dsh-llm-deepseek 的 design-verification twin**: 两者共享同一 `LlmAdapter` 契约, pi-ai 路径覆盖 OpenAI / Anthropic / 自定义网关, 是设计验证。
4. **凭据永不入配置**: 所有 `apiKeyEnv` 都是环境变量名 (`CredentialRef` brand) , 通过 `dsh-credentials` 接缝或 `launchEnvironmentOf(ctx)` 解析, secret 永不进入 settings 文档。
5. **流式响应空闲超时**: `DEFAULT_STREAM_IDLE_TIMEOUT_MS = 300_000` (5 分钟) , `idleWatchdog` 监控单次读操作间隔, 避免 hang。
6. **pi-ai catalog drift gate**: `MODALITY_GATE` (`catalog.ts:42`) 和 `THINKING_LEVEL_GATE` (`catalog.ts:69`) 用 `Record<...>` 编译期检查 pi-ai 升级引入的 modality / thinking level 漂移。

17 沙箱与执行环境集成

本文档基于 AST 符号工具对源码符号进行验证分析生成。代码引用处的行号均来自符号工具验证。

17.1 集成概述

dsh 的沙箱能力负责在执行模型触发的 shell / subprocess 命令时施加文件系统约束。它同样遵循能力接缝三角色:

- **Service Definition**: `@deepseek-ai/dsh-sandbox` 声明 `SandboxProvider` 抽象类、`SandboxPolicy` / `ConfinedArgv` 词汇、`SandboxUnavailableError` fail-closed 错误。
- **Service Provider**:
 - `dsh-sandbox-local`: 本地进程沙箱 (Linux bwrap/Landlock、macOS Seatbelt、Windows ACL restricted-token runner) 。
 - `dsh-sandbox-windows-acl`: Windows ACL 后端独立子包。
 - `@deepseek-ai/node-addon-landlock-run`: 原生 addon (C 代码 + 静态预编译二进制) 。
 - `dsh-e2b` + `dsh-fs-e2b` + `dsh-subprocess-e2b`: E2B 远程沙箱 POC。
- **Consumer**: `dsh-shell` (bash 执行)、`dsh-subprocess` (子进程树)、`dsh-code-runtime` (worker-thread)、`dsh-terminal` (PTY) 通过 `ctx.sandbox.confine(argv, policy)` 包装 argv。

17.1.1 设计哲学

- **fail-closed**: 无可用 backend 时拒绝运行 unconfined 命令, 抛 `SANDBOX_UNAVAILABLE` 。
- **per-call policy**: `SandboxPolicy` 携带 `mode` / `workspaceRoot` / `sessionId` , 不同 consumer 可同时使用不同 policy。
- **denial dialect**: 每个 backend 报告自己的 `denialSignatures` (EROFS / EACCES / EPERM) , consumer 只匹配本 backend 的方言而非并集。
- **runner failure 区分**: `RunnerFailureRule` 结构化 stderr 匹配规则, 先识别 runner 自身失败 (命令未运行) , 再识别 denial (confinement 工作并阻止了) 。

17.2 支持的服务/产品

服务名	版本 / API	平台	状态	说明
Bubblewrap (<code>bwrap</code>)	命令行	Linux	GA	Linux 首选 runner, mount profile 接近 mode 词汇
Landlock	Linux Kernel ABI	Linux	GA	<code>landlock-run</code> 原生 launcher, allow-list 语义
Seatbelt (<code>sandbox-exec</code>)	macOS 内核	macOS	GA	macOS 唯一候选, <code>sandbox-exec -p</code> profile
Windows ACL	Win32 API	Windows	GA (partial)	restricted-token + capability-SID ACE, 需 koffi FFI
E2B	<code>e2b</code> SDK 2.29.1	远程 Linux	POC	远程 Linux 沙箱, 通过 <code>Sandbox.create()</code>
node-pty	1.1.0 (patched)	跨平台	GA	持久 PTY 后端 (非沙箱, 但执行环境相关)
koffi	^3.1.0	Windows	GA	Win32 FFI 加载器, 用于 Windows ACL
runnerCommand override	自定义	任意	可选	operator 断言已配置 runner, 跳过 probe

17.3 集成方式

17.3.1 Service Definition: `SandboxProvider` 与 `SandboxPolicy`

`SandboxProvider` (`packages/sandbox/sandbox/src/index.ts:158`) 是抽象 Service:

```
abstract class SandboxProvider extends Service {
  constructor(ctx: Context) { super(ctx, 'sandbox') }
  abstract confine(argv: readonly string[], policy: SandboxPolicy): ConfinedArgv
}
```

`SandboxMode` (`index.ts:29`) : `'read-only' | 'workspace-write' | 'danger-full-access'` 。 `danger-full-access` 不进入 `SandboxPolicy` (不是 confinement) 。

`SandboxExecutionPolicy` (`index.ts:39`) : - `mode: SandboxMode` - `workspaceRoot: string` (绝对路径) - `sessionId?: SessionId` (branded, 用于 per-session 状态隔离)

`SandboxPolicy extends SandboxExecutionPolicy` (`index.ts:69`) : 把 `mode` 收窄到 `ConfinedSandboxMode` 。

`ConfinedArgv` (`index.ts:95`) 是 `confine()` 的返回: - `argv: string[]` : wrapped argv (runner + profile args + `--` + 原 argv) - `enforcement: 'full' | 'partial'` : 本次执行的强制完整性 - `denialSignatures: readonly string[]` : 本 backend 的 denial stderr 子串 - `runnerFailureRules: readonly RunnerFailureRule[]` : runner 失败识别规则

`SandboxUnavailableError` (`index.ts:131`) 携带 `SANDBOX_UNAVAILABLE` code, message 列出平台对应的可用 backend 安装建议。

17.3.2 Provider: LocalSandboxProvider (多 runner 链)

LocalSandboxProvider (packages/sandbox/sandbox-local/src/index.ts:250) 是 SandboxProvider 的本地实现。它选择平台 runner 链 (index.ts:159) :

```
const PLATFORM_CHAINS: Record<string, readonly SelectedRunner['runner'][]> = {
  linux: ['bwrap', 'landlock'],
  darwin: ['seatbelt'],
  win32: ['windows-acl'],
}
```

每个平台的链接按优先级排序, **sole candidate 直接选用不 probe** (其运行时拒绝仍 fail-closed), 多 candidate 才 probe 仲裁。

STATIC_ENFORCEMENT (index.ts:177) : bwrap / landlock / seatbelt = 'full', windows-acl = 'partial' (因 WRITE_RESTRICTED 需保留 Everyone + NTFS hard link 可跨路径别名)。

DENIAL_SIGNATURES (index.ts:205) : - bwrap: ['read-only file system'] - landlock: ['permission denied'] - seatbelt: ['operation not permitted'] - windows-acl: ['access is denied', 'access to the path', 'permission denied']

RUNNER_FAILURE_RULES (index.ts:231) : 每个 runner 的 fatal stderr 前缀 + exit code 限制: - bwrap: { fatalSignatures: ['bwrap: '] } - landlock: { allowedExitCodes: [125], fatalSignatures: ['landlock-run:'], informationalLines: ['landlock-run: partial enforcement (older Landlock ABI)'] } - seatbelt: { fatalSignatures: ['sandbox-exec: '] } - windows-acl: { allowedExitCodes: [127], fatalSignatures: ['windows-acl-run: '] }

confine(argv, policy) (index.ts:316) : - 若有 runnerCommand override: 直接 [...runnerCommand, ...bwrapProfileArgs(policy), '--', ...argv] + enforcement: 'full'。 - 否则 selectRunner(policy.mode) 选 rung, 调用对应 runnerArgv() 拼装。

17.3.3 Provider: dsh-sandbox-windows-acl (koffi FFI)

独立子包 @deepseek-ai/dsh-sandbox-windows-acl (依赖 koffi: ^3.1.0) :

- ffi.ts (packages/sandbox/sandbox-windows-acl/src/ffi.ts) : lazy koffi bindings, 非 Windows 进程永不加载 Win32 库。每个函数签名对照 MinGW headers 验证, struct layouts 在 verify/abi-probe.cpp 加载时断言。
- runner.ts (runner.ts:44) : argv-prefix wrapper, 创建 WRITE_RESTRICTED token + workspace write-SID allowlist, spawn 子进程并 mirror exit code, temp grant 在 exit 时撤销。稳定 argv 契约: [node, runner.js, '--workspace', <dir>, '--temp', <dir>, '--mode', <mode>, ['--write-sid', <sid>, '--temp-write-sid', <sid>], '--', <argv...>]。
- AclWriteGrant : standing workspace-root grant per workspace (cross-session 复用缓存) + revocable private-temp grant per live session/workspace pair。
- workspaceWriteSid(workspaceRoot) / tempWriteSid(tempDir) : 基于路径派生稳定 capability-SID。

LocalSandboxProvider.materializeAclGrant(sessionId, workspaceRoot) (sandbox-local/src/index.ts:392) : fail-closed, 半 materialized 的 temp grant 被撤销 + 目录删除后才抛错。

17.3.4 Native Addon: @deepseek-ai/node-addon-landlock-run

native/landlock-run/packages/entry/src/index.ts 提供 JS API over 预编译 landlock-run 二进制:

- LAUNCHER_BIN = 'landlock-run' (index.ts:22)
- LAUNCHER_FAILURE_EXIT = 125 (index.ts:31) : 每个 launcher 级失败的 exit code (usage error、unenforcing kernel、unopenable grant root、failed exec)。
- LandlockEnforcement = 'full' | 'partial' | 'unusable' (index.ts:41) : probe 的裁决结果。
- LauncherGrants (index.ts:47) : readOnly / readWrite 路径列表 (Landlock 是 allow-list)。
- launcherPath() (index.ts:69) : 从 @deepseek-ai/node-addon-landlock-run-\${process.platform}-\${process.arch} 解析二进制路径; npm 的 os / cpu 字段让安装只 fetch 匹配平台的 optional dependency。
- probe() : spawnSync 启动 launcher, 依据 exit code 与 stderr 判断 enforcement level。

关键设计：本模块故意不提供任何环境变量 **override** (`index.ts:14` 注释) —— 「which binary confines a process must never be decidable by the ambient environment」。测试注入通过函数参数。

二进制构建：

- `native/landlock-run/packages/entry/src/main.c` 是 C 源码。
- 平台预编译包：`packages/linux-arm64` / `packages/linux-x64`。
- BSD-3-Clause license (与 MIT 主体不同)。

17.3.5 Provider: E2B 远程沙箱 (POC)

`dsh-e2b` (`packages/e2b/e2b/src/index.ts`) 使用 `e2b: 2.29.1` SDK:

`E2BRuntime` (`index.ts:74`) extends `Service` :

- `static Config` (`index.ts:75`) : `apiKey` (默认读 `E2B_API_KEY`)、`cwd` (默认 `/home/user/workspace`)、`timeoutMs` (默认 `300_000`)。
- `validate()` (`index.ts:139`) : `apiKey` 必填、`cwd` 必须是绝对 Linux 路径、`timeoutMs` 正有限。
- `open()` (`index.ts:151`) : `Sandbox.create({ apiKey, timeoutMs, secure: true, lifecycle: { onTimeout: 'kill' } })`，然后 `files.mkdir` 创建 `cwd` 和 `.dsh-e2b runtimeRoot`，`chmod 700`。
- `getSandbox()` (`index.ts:130`) : 返回共享 live SDK handle; adapters await 此 promise 后才执行首次操作。
- `ctx.effect(() => async () => { sandbox.kill() })` : dispose 时 kill 沙箱。
- `quoteE2BShellArg(value)` (`index.ts:27`) : 为 SDK 的 `/bin/bash -l -c` 强制层正确引用参数。
- `e2bControlEnvs(overrides)` (`index.ts:36`) : 随机化 `HOME` 路径隔离 E2B 的硬编码 login shell。

E2B 是 POC: `fs-e2b` 提供 filesystem 实现，`subprocess-e2b` 提供 subprocess 实现，三者共享同一 `E2BRuntime` handle 进入同一远程 Linux world。

17.3.6 node-pty (patched 1.1.0)

`pnpm-workspace.yaml` 配置 (第 71-72 行) :

```
patchedDependencies:
  node-pty@1.1.0: patches/node-pty@1.1.0.patch
```

`allowBuilds: node-pty: true` 允许其 install/build 脚本运行 (含 ConPTY Windows)。patch 文件 `patches/node-pty@1.1.0.patch` 自定义修改。

`dsh-subprocess-local` 的 `postinstall` (`pnpm-workspace.yaml:55`) 也允许: `@deepseek-ai/dsh-subprocess-local@file:packages/subprocess/subprocess-local: true`，用于 Python runtime deploy 时恢复 macOS spawn helper 的 executable bit。

17.4 代码实现

17.4.1 关键类与文件

类 / 模块	路径	行号	角色
<code>SandboxProvider</code>	<code>packages/sandbox/sandbox/src/index.ts</code>	158-176	抽象 service
<code>SandboxUnavailableError</code>	<code>packages/sandbox/sandbox/src/index.ts</code>	131	fail-closed 错误
<code>LocalSandboxProvider</code>	<code>packages/sandbox/sandbox-local/src/index.ts</code>	250-565	多 runner 链本地 provider
<code>PLATFORM_CHAINS</code>	<code>packages/sandbox/sandbox-local/src/index.ts</code>	159	平台到 runner 链映射
<code>DENIAL_SIGNATURES</code>	<code>packages/sandbox/sandbox-local/src/index.ts</code>	205	各 backend denial 方言
<code>RUNNER_FAILURE_RULES</code>	<code>packages/sandbox/sandbox-local/src/index.ts</code>	231	runner 失败识别规则
<code>confine()</code>	<code>packages/sandbox/sandbox-local/src/index.ts</code>	316	argv 包装入口
<code>materializeAclGrant()</code>	<code>packages/sandbox/sandbox-local/src/index.ts</code>	392	Windows ACL grant 物化
<code>selectRunner()</code>	<code>packages/sandbox/sandbox-local/src/index.ts</code>	492	链选择 + probe 仲裁
<code>windowsAclRunnerArgv()</code>	<code>packages/sandbox/sandbox-local/src/index.ts</code>	358	Windows ACL runner argv
<code>ffi.ts</code>	<code>packages/sandbox/sandbox-windows-acl/src/ffi.ts</code>	1	koffi lazy bindings
<code>runner.ts</code>	<code>packages/sandbox/sandbox-windows-acl/src/runner.ts</code>	44	Windows ACL runner
<code>E2BRuntime</code>	<code>packages/e2b/e2b/src/index.ts</code>	73-179	E2B 共享沙箱生命周期
<code>LAUNCHER_BIN</code>	<code>native/landlock-run/packages/entry/src/index.ts</code>	22	landlock-run 二进制名
<code>launcherPath()</code>	<code>native/landlock-run/packages/entry/src/index.ts</code>	69	平台二进制路径解析
<code>probe()</code>	<code>native/landlock-run/packages/entry/src/index.ts</code>	—	landlock 功能探针

17.4.2 平台选择与 probe 流程

```
confine(argv, policy) [sandbox-local/src/index.ts:316]
↓
selectRunner(mode) [index.ts:492]
↓
chainVerdict() [index.ts:499]
- 读 PLATFORM_CHAINS[platform]
- sole candidate → 直接返回 + STATIC_ENFORCEMENT
- 多 candidate → 顺序 probeRunner() 直到首个非 'unusable'
- 全部 unusable → 'unavailable' → throw SandboxUnavailableError
↓
runnerArgv(selected.runner, policy) [index.ts:336]
- bwrap: ['bwrap', ...bwrapProfileArgs(policy)]
- landlock: [landlockLauncher(), ...landlockProfileArgs(policy)]
- seatbelt: [seatbeltExec(), ...seatbeltProfileArgs(policy)]
- windows-acl: windowsAclRunnerArgv(policy) [index.ts:358]
↓
return { argv, enforcement, denialSignatures, runnerFailureRules }
```

17.5 配置与环境变量

17.5.1 环境变量

环境变量	作用	默认
E2B_API_KEY	E2B 远程沙箱 API key	必填 (仅 POC 使用)

沙箱 deliberately 不提供环境变量 override (landlock-run/packages/entry/src/index.ts:14) : runner 选择由 process.platform + probe 决定, 不允许 ambient env 干预。

17.5.2 Plugin Config (dsh-sandbox-local)

Config interface (packages/sandbox/sandbox-local/src/index.ts:43) :

字段	类型	默认	说明
runnerCommand	string[]	[]	自定义 runner argv (非空则跳过 probe, 断言 full enforcement)
runnerFailureSignatures	string[]	[]	runner 拒绝 profile 时的 stderr 子串 (与 runnerCommand 互为必填)
probeTimeoutMs	number	5_000	每次 functional probe 超时

约束: - runnerCommand 非空时 runnerFailureSignatures 必填, 反之亦然 (index.ts:283-291)。 - runnerFailureSignatures 每条必须非空、单行 (无 \r\n)。 - probeTimeoutMs 必须正有限 (assertPositiveFinite , index.ts:194)。

17.5.3 Plugin Config (dsh-e2b)

Config interface (packages/e2b/e2b/src/index.ts:43) :

字段	类型	默认	说明
apiKey	string	process.env.E2B_API_KEY	E2B API key (永不转发入沙箱)
cwd	string	/home/user/workspace	远程共享工作目录
timeoutMs	number	300_000	沙箱生命周期，到期自动 kill

17.6 错误处理

17.6.1 错误码表

Code	含义	触发
SANDBOX_UNAVAILABLE	无可用的 backend	SandboxUnavailableError (sandbox/src/index.ts:131)
SANDBOX_DENIED (隐式)	confinement 拒绝写操作	通过 stderr denialSignatures 匹配
Runner exit 125	landlock-run launcher 失败	LAUNCHER_FAILURE_EXIT
Runner exit 127	windows-acl runner 失败	WINDOWS_ACL_RUNNER_FAILURE_EXIT (sandbox-local/src/index.ts:216)

17.6.2 Runner Failure vs Denial 区分

consumer 在收到非零 exit code 时按顺序处理：

- 应用 runnerFailureRules[].allowedExitCodes 过滤（如 landlock 只在 exit 125 时考虑 runner failure）。
- 移除 informationalLines（如 landlock 的 partial enforcement (older Landlock ABI) 提示，非 fatal）。
- 在剩余 stderr 行中 case-insensitive 匹配 fatalSignatures（如 landlock-run: ）。
- 命中 → runner failure（命令从未运行）；未命中 → 检查 denialSignatures → 命中 → confinement denial（命令运行了但被阻止写）。

17.6.3 Probe 失败 = fail-closed

每个 backend 的 probe（probeRunner， sandbox-local/src/index.ts:513）返回 'full' | 'partial' | 'unusable'：

- bwrap: spawnSync('bwrap', [...profileArgs, '--', 'true']) exit 0 = full。
- landlock: 调用 @deepseek-ai/node-addon-landlock-run 的 probe()，返回 full / partial / unusable。
- seatbelt: spawnSync('sandbox-exec', [...profileArgs, '--', 'true']) exit 0 = full。
- windows-acl: spawnSync(runnerInvocation, [..., '--mode', 'read-only', '--', 'cmd', '/c', 'exit', '0']) exit 0 = partial (Windows 默认 partial)。

链上所有 rung unusable → chainVerdict() 返回 'unavailable' → confine() throw SandboxUnavailableError，命令永不 unconfined 运行。

17.6.4 ACL Grant 清理

revokeAclGrants()（ sandbox-local/src/index.ts:454）在 provider dispose 时：

- 撤销每个 tempCapabilities 的 temp ACE + 删除 temp 目录。
- workspaceGrants（standing workspace ACE）保留（cross-session 复用缓存，跨 provider 生命周期）。
- 清理失败被收集后 logger.warn，不抛（cordis teardown 不能被 grant cleanup 中断）。
- 崩溃时跳过 cleanup，但新 provider 永不复用旧路径或 SID（随机性保证）。

17.7 扩展指南：添加新沙箱 Backend

17.7.1 新增本地 runner

- 在 `dsh-sandbox-local/src/profiles.ts` 添加 `<new-runner>ProfileArgs(policy)`：构造新 runner 的 profile argv。
- 扩展 `SelectedRunner['runner']` 类型：加入新成员。
- 更新 `PLATFORM_CHAINS`：在对应平台链加入新 runner（保持优先级）。
- 更新 `STATIC_ENFORCEMENT`：若 sole candidate 则填入 enforcement level。
- 更新 `DENIAL_SIGNATURES` + `RUNNER_FAILURE_RULES`：定义本 backend 的 stderr 方言。
- 在 `runnerArgv()` switch 加入 case：返回 `[<runner>, ...profileArgs]`。
- 更新 `probeRunner()`：调用 functional probe（spawnSync 一个 `-- true` 类似的最小命令）。
- 测试：用 `SandboxInternals` 注入 fake chain / fake probe 跨平台验证。

17.7.2 新增独立 Provider 包（参考 E2B）

- 新建 `packages/<cap>/sandbox-<name>/`，`peerDependencies` 含 `@deepseek-ai/dsh-sandbox`。
- 继承 `SandboxProvider`：实现 `confine(argv, policy)`。
- (可选) 实现 `fs-<name>` / `subprocess-<name>`：如果新沙箱需要替代 host filesystem 或子进程。
- 加入 bundle：在 `cordis.yml` 加 `- id: sandbox-<name>`。

17.7.3 SandboxPolicy 设计原则

新 consumer 在调用 `ctx.sandbox.confine(argv, policy)` 前：

- per-call 解析 policy：从 `ctx.sandboxPolicy`（`dsh-sandbox-policy` 包）解析 mode + workspaceRoot + sessionId。
- 不假设 enforcement 完整性：检查 `ConfinedArgv.enforcement`，`'partial'` 时告知用户边界。
- 匹配 denial dialect：使用 `ConfinedArgv.denialSignatures` 而非硬编码。
- 应用 `runnerFailureRules`：在分类 stderr 时严格按规则。

17.8 关键 SDK 依赖版本

依赖	版本	用途
<code>koffi</code>	^3.1.0	Win32 FFI (<code>sandbox-windows-acl/package.json</code>)
<code>e2b</code>	2.29.1	E2B SDK (<code>e2b/package.json</code>)
<code>node-pty</code>	1.1.0 (patched)	PTY 后端 (patches/node-pty@1.1.0.patch)
<code>@deepseek-ai/node-addon-landlock-run</code>	0.1.1 (workspace)	landlock launcher JS API + 二进制
<code>@deepseek-ai/node-addon-landlock-run-linux-x64</code>	0.1.4 (workspace)	x64 预编译二进制
<code>@deepseek-ai/node-addon-landlock-run-linux-arm64</code>	0.1.4 (workspace)	arm64 预编译二进制

`pnpm-workspace.yaml` 关键 `allowBuilds` 配置：

```
allowBuilds:
  esbuild: true      # 原生 binary
  lefthook: true     # git hooks
  node-pty: true     # 跨平台 PTY 后端 (含 ConPTY)
  '@google/genai': false # pi-ai 拉入但脚本 no-op, 拒绝
  protobufjs: false  # 同上
  koffi: true        # Win32 FFI 用于 MoveFileExW
  '@deepseek-ai/dsh-subprocess-local@file:packages/subprocess/subprocess-local': true
```

patchedDependencies : node-pty@1.1.0: patches/node-pty@1.1.0.patch。

17.9 关键发现

- fail-closed 是硬约束**：SandboxProvider.confine 必须返回 enforcing argv 或在 wrap/runner-execution 时失败，禁止 silent unconfined passthrough (sandbox/src/index.ts:154 注释)。
- per-platform 链 + 优先级 probe**：Linux 上 bwrap 优先于 landlock (mount profile 更接近 mode 词汇)，多 candidate 才 probe。
- Windows ACL 的 partial 强制**：WRITE_RESTRICTED token 需保留 Everyone (进程初始化需要) + NTFS hard link 可跨路径别名，所以永远报告 partial。
- standing vs revocable grant 分离**：workspace-root ACE 跨 session 复用 (O(1) exact-ACE skip)，private-temp ACE per-session 随机化并 dispose 时撤销。
- landlock-run 二进制不存在环境变量 override**：故意设计，防止 ambient env 控制哪个 binary 限制进程 (安全考虑)。
- E2B 是 POC**：当前只提供 E2BRuntime 共享 handle + fs-e2b + subprocess-e2b，未集成进 cordis.yml 默认 bundle。
- node-pty 1.1.0 是 patched 版本**：项目通过 patchedDependencies 自定义修复 (patch 文件可读但具体修改未在本次分析范围)。
- runner argv 契约稳定**：windows-acl 的 argv 契约文档化 (runner.ts:9-14)，未来 native-exe 替换保持同契约只需替换 [node, runner.js] 前缀。

18 协议集成

本文档基于 AST 符号工具对源码符号进行验证分析生成。代码引用处的行号均来自符号工具验证。

18.1 集成概述

dsh 通过三类协议集成实现「自动化客户端」与「外部 hook 生态」对接：

- ACP (Agent Client Protocol)**：@agentclientprotocol/sdk 0.25.1，JSON-RPC over stdio，让外部 trusted 客户端驱动 dsh agent。
- JSON-RPC SDK 协议**：dsh-sdk-protocol + dsh-sdk-client + dsh-sdk-server 自定义实现的 newline-delimited JSON-RPC 协议，提供比 ACP 更完整的 SDK 客户端能力 (subagent、session tree 等)。
- Hook 桥接协议**：dsh-hook-protocol + dsh-hooks-claude-code + dsh-hooks-codex 让 dsh 复用 Claude Code 和 Codex 的 hook 配置和命令生态。

三者都基于「Service Definition / Provider / Consumer」三角色模式，但 ACP 和 SDK 是协议层集成 (对外暴露 dsh 能力)，hook 桥是兼容层集成 (消费外部 hook 生态)。

18.1.1 设计哲学

- 进程外 SDK 优先**：所有协议均 over stdio JSON-RPC，无 HTTP 监听端口。
- 复用 wire 协议库**：dsh-sdk-protocol 把 newline-delimited JSON-RPC transport 抽成共享包，server 与 client 共享。
- bridge 而非 reimplementaion**：hook 桥执行未修改的 Claude Code / Codex hook 命令，不重新实现其语义。

- 共享执行库 + 独立方言: `dsh-hook-protocol` 提供 `matcher / codec / merge / runner`, 每个 dialect bridge 只 own 自己的 `payload / env / substitution / mapping`。

18.2 支持的服务/产品

协议 / 服务	SDK / 库	版本	状态	说明
ACP (Agent Client Protocol)	<code>@agentclientprotocol/sdk</code>	0.25.1	GA	JSON-RPC over stdio, automation-only
DeepSeek Harness SDK	自实现	0.1.0-rc.5	GA	自定义 JSON-RPC, 比 ACP 多 subagent 支持
Python SDK	Python 客户端	—	GA	<code>python/sdk</code> , 与 TS <code>HarnessClient</code> 是 design twin
Claude Code Hooks	自实现 bridge	—	GA	兼容未修改的 <code>hooks.json / settings</code>
Codex Hooks	自实现 bridge	—	GA	兼容未修改的 Codex <code>hooks.json</code>
JSON-RPC wire	自实现 transport	—	GA	<code>JsonRpcLineTransport</code> , newline-delimited

18.3 集成方式

18.3.1 ACP Server (`dsh-acp`)

`apply(ctx, config)` (`packages/acp/acp/src/index.ts:105`) :

- `inject = ['agents']` : ACP bridge 创建并 own agents, 其他能力由 agent 组合承载。
- `Config` (`index.ts:70`) : `provider` / `model` (创建 agent 时使用) / `stream` (test-only transport override) 。
- 使用 `AgentSideConnection` (来自 `@agentclientprotocol/sdk`) 作为 SDK 连接。
- 处理的方法: `initialize` / `authenticate` / `newSession` / `prompt` / `cancel` 。
- 通过 `session/notification` 推送 turn 事件。
- **ACP 是 automation-only**: 只携带 prompt text / committed assistant text / cancellation / one-shot permission, presentation 和 human-interaction 留给 harness UI 模块。

错误转换 (`index.ts:60-67`) :

```
function invalidParams(detail: string): RequestError {
  return RequestError.invalidParams(undefined, detail)
}
function internalError(detail: string): RequestError {
  return RequestError.internalError(undefined, detail)
}
```

`PROTOCOL_VERSION` 从 SDK 导入 (`index.ts:21`) , ACP 客户端必须匹配。

18.3.2 SDK Wire Protocol (dsh-sdk-protocol)

packages/sdk/protocol/src/index.ts:11 导出:

- `JsonRpcLineTransport` / `JsonRpcResponseError` (来自 `transport.ts`)
- `JsonRpcTransportPeer` (type)
- 类型: `HarnessSdkRequestMap` / `HarnessSdkResultMap` / `HarnessSdkNotificationMap` / `InitializeParams` / `InitializeResult` / `SessionPromptParams` / `SessionPromptResult` / `SessionEventNotification` / `SessionStatusNotification` / `SubagentStartedNotification` / `SubagentFinishedNotification` / `SdkRunStatus` (来自 `types.ts`)

newline-delimited JSON-RPC: 每行一个 JSON 消息, stdio transport。Server 与 client 共享此 protocol 包, 确保两端 type 一致。

18.3.3 SDK Server (dsh-sdk-jsonrpc-server)

`HarnessSdkJsonRpcServer` (`packages/sdk/server/src/server.ts:52`) :

- 构造接收 `ctx` + `transport: JsonRpcTransportPeer` + `options`。
- 订阅 `session/event` → `transport.notify('session.event', ...)` (`server.ts:71`)。
- 订阅 `agent/status` → `transport.notify('session.status', ...)` (`server.ts:75`)。
- 订阅 `session/created` → 处理 subagent `session/start` notification。
- 关键方法:
 - `initialize(params)` (`server.ts:110`) : 握手。
 - `prompt(params)` (`server.ts:131`) : 发起 turn。
 - `shutdown()` / `performShutdown()` (`server.ts:149` / `154`) : 优雅关闭。
 - `getOrCreateSession(sessionId)` (`server.ts:202`) + `createSession(sessionId)` (`server.ts:217`)。
 - `handleRequest(method, params)` (`server.ts:189`) : 分发。
 - `hasAdapterFor(provider)` (`server.ts:236`) : 检查 LLM adapter 可用性。

`maxTokensAsSuccess` 选项: deployment 可让 `max-tokens` 终止报告为 `ok` 而非 `error` (`server.ts:43`)。

18.3.4 SDK Client (dsh-sdk-client)

`HarnessClient` (`packages/sdk/client/src/client.ts:183`) 是低层级 client, 运行在 **harness context 之外**, 直接 spawn runtime subprocess:

- `inject` 无 (非 plugin)。
- `start()` (`client.ts:202`) : spawn runtime 子进程。
- `initialize(params)` (`client.ts:267`) : 握手。
- `prompt(params)` (`client.ts:282`) : 发起 prompt。
- `request(method, params)` (`client.ts:300`) : 低层级 JSON-RPC request。
- `subscribe(filter)` (`client.ts:341`) : 订阅 notification 流 (`AsyncIterable<HarnessNotification>`)。
- `close()` / `performClose()` (`client.ts:379` / `384`) : 通过私有 EOF → SIGTERM → SIGKILL ladder 关闭子进程。
- `subscribeSessionTree(sessionId)` (`client.ts:360`) : 订阅整个 session 树 (含 subagent)。

错误类型 (`client.ts:38-65`) :

- `TransportClosedError` : runtime 子进程消失 / unusable。
- `RequestTimeoutError` : 请求超时 (`HarnessClientOptions.requestTimeoutMs`)。
- `SdkProtocolError` : runtime 返回的响应不符合协议 (如 `session/prompt` 缺 `accepted: true`)。

子进程关闭 ladder (`dispose.ts`) : EOF (关闭 stdin) → SIGTERM → SIGKILL, grace `STREAM_SETTLE_MS = 100` 让 stdio 流沉淀。

18.3.5 Hook Protocol Library (dsh-hook-protocol)

packages/hooks/hook-protocol/src/index.ts:1 描述：共享、非 plugin 的 hook 协议库——matching / command 执行 / decoding / restrictive outcome merging / durable event helpers / detached run quiescence。

导出 (index.ts:9-25)：

- 类型： CommandHook / HookDialect / HookOutput / MatcherGroup / MatcherMode (types.ts)
- matcherDiagnostic / matchesMatcher (matcher.ts)
- parseHookOutput (codec.ts)
- DEFAULT_HOOK_TIMEOUT_MS / runHook (runner.ts) + RunHookOptions / RunHookResult
- mergeHookOutputs (merge.ts) + MergedDecision / MergedHookOutcome
- appendHookInvoked / appendHookResult / DEFAULT_STDERR_SUMMARY_MAX_CHARS / summarizeStderr (events.ts) + HookInvocation / HookResultRecord
- createDetachedRuns (detached.ts) + DetachedRuns

核心特性： - **matcher engine**: 支持多种 matcher 模式 (regex / substring 等)，通过 **MatcherMode** 区分。 - **stdin/exit-code/stdout codec**: hook 命令通过 stdin 传 payload, exit code + stdout 表达决策。 - **restrictive merge**: 多 hook 输出合并采用最严格决策 (block 优先)。 - **detached runs**: hook 可声明 detached，在主流程返回后继续运行并 quiesce。

18.3.6 Claude Code Hook Bridge (dsh-hooks-claude-code)

packages/hooks/hooks-claude-code/src/index.ts:9 说明：bridge for unmodified Claude Code command hooks on harness interception extension points。

- inject = ['shell']：需要 bash 执行 hook 命令，其他扩展点 opportunistic 通过 ctx.get 读。
- 支持的扩展点： SessionStart / prompt pre/post / tool pre/post / Stop / subagent start/stop。
- Owns Claude payloads / environment / substitution / decision mapping；shared execution and parsing 在 dsh-hook-protocol 。
- updatedInput 字段被 log + warn 但不 honor (设计取舍)。

Config (index.ts:45)：

字段	类型	默认	说明
configPath	string (required)	—	hooks.json 或 settings 文件路径
pluginRoot	string	—	替换 \${CLAUDE_PLUGIN_ROOT}
projectDir	string	session workspace	替换 \${CLAUDE_PROJECT_DIR} + 导出为 env var
defaultTimeoutMs	number	DEFAULT_HOOK_TIMEOUT_MS (600_000)	hook 默认超时
stderrSummaryMaxChars	number	DEFAULT_STDERR_SUMMARY_MAX_CHARS	持久化 stderr 摘要上限

18.3.7 Codex Hook Bridge (dsh-hooks-codex)

packages/hooks/hooks-codex/src/index.ts:8 说明：bridge for unmodified Codex command hooks。

- 同样 inject = ['shell']。
- 支持五个点： SessionStart / prompt pre/post / tool pre/post / Stop (不支持 subagent start/stop)。
- regex-only matchers, snake_case payloads without trailing newline。
- no hook environment or command substitution。

- no pre-tool approval or rewrite path, 只 blocking decisions 被 honor。
- 每个 payload携带 model 字段 (Config.model 默认 '')。

Config (index.ts:44) :

字段	类型	默认	说明
configPath	string (required)	—	Codex hooks.json 路径
model	string	''	stamp 在每个 payload 上的模型名
defaultTimeoutMs	number	DEFAULT_HOOK_TIMEOUT_MS	hook 默认超时
stderrSummaryMaxChars	number	DEFAULT_STDERR_SUMMARY_MAX_CHARS	摘要字符上限

18.4 代码实现

18.4.1 关键类与文件

类 / 模块	路径	行号	角色
<code>HarnessSdkJsonRpcServer</code>	<code>packages/sdk/server/src/server.ts</code>	52-239	SDK server
<code>HarnessClient</code>	<code>packages/sdk/client/src/client.ts</code>	183-457	SDK client (spawn 子进程)
<code>TransportClosedError</code>	<code>packages/sdk/client/src/client.ts</code>	38	runtime 死亡错误
<code>RequestTimeoutError</code>	<code>packages/sdk/client/src/client.ts</code>	47	请求超时错误
<code>SdkProtocolError</code>	<code>packages/sdk/client/src/client.ts</code>	59	协议响应错误
<code>JsonRpcLineTransport</code>	<code>packages/sdk/protocol/src/transport.ts</code>	—	newline-delimited JSON-RPC transport
<code>AcpConfig</code>	<code>packages/acp/acp/src/index.ts</code>	70	ACP plugin config
<code>apply</code> (ACP)	<code>packages/acp/acp/src/index.ts</code>	105	ACP server mount
<code>runHook</code>	<code>packages/hooks/hook-protocol/src/runner.ts</code>	—	hook 命令执行
<code>parseHookOutput</code>	<code>packages/hooks/hook-protocol/src/codec.ts</code>	—	hook 输出解码
<code>mergeHookOutputs</code>	<code>packages/hooks/hook-protocol/src/merge.ts</code>	—	restrictive outcome 合并
<code>createDetachedRuns</code>	<code>packages/hooks/hook-protocol/src/detached.ts</code>	—	detached run quiescence
<code>apply</code> (Claude Code)	<code>packages/hooks/hooks-claude-code/src/index.ts</code>	—	Claude Code bridge mount
<code>apply</code> (Codex)	<code>packages/hooks/hooks-codex/src/index.ts</code>	—	Codex bridge mount

18.4.2 ACP 与 SDK 的差异

维度	ACP	dsh-sdk
协议源	第三方标准 @agentclientprotocol/sdk 0.25.1	自实现 wire protocol
状态	automation-only	完整 SDK 客户端能力
Transport	stdio JSON-RPC	stdio newline-delimited JSON-RPC
Subagent 支持	通过 session/notification 间接	一等公民: SubagentStartedNotification / SubagentFinishedNotification + subscribeSessionTree
Client 实现	第三方 SDK 提供	自实现 HarnessClient + Python twin
Server 实现	apply(ctx, config) 函数式	HarnessSdkJsonRpcServer class
关闭协议	ACP 标准关闭	EOF → SIGTERM → SIGKILL ladder (dispose.ts)

18.4.3 Hook Bridge 架构

```
外部 hooks.json (Claude Code 或 Codex 格式)
↓
parseClaudeCodeConfig / parseCodexConfig [config.ts]
↓
matchesMatcher(matcherGroup, payload, mode) [hook-protocol/matcher.ts]
↓
runHook(command, stdin, options) [hook-protocol/runner.ts]
- spawn hook 命令
- 通过 stdin 传 payload
- timeout 控制
↓
parseHookOutput(stdout, exitCode) [hook-protocol/codec.ts]
↓
mergeHookOutputs(outputs) [hook-protocol/merge.ts]
- restrictive: block 优先
↓
appendHookInvoked / appendHookResult [hook-protocol/events.ts]
- 持久化到 session log
↓
createDetachedRuns [hook-protocol/detached.ts]
- 主流程返回后继续运行
```

每个 dialect bridge 只 override: - payload 格式 (CamelCase vs snake_case, 是否带 trailing newline) - 环境变量 (CLAUDE_PROJECT_DIR 等) - 命令 substitution (\${CLAUDE_PLUGIN_ROOT} 等) - matcher mode (regex-only vs 多模式) - 决策 mapping (pre-tool approval 是否支持等)

18.5 配置与环境变量

18.5.1 环境变量

环境变量	作用	默认
(无独立 env vars)	协议集成不直接读环境变量	—

注：hook 桥接通过 `CLAUDE_PROJECT_DIR` 等 env var 导出给 hook 子进程，但 dsh 自身不读这些 env var。

18.5.2 Plugin Config

18.5.2.1 ACP (`packages/acp/acp/src/index.ts:70`)

字段	类型	默认	说明
<code>provider</code>	string	—	创建 agent 时使用的 provider route
<code>model</code>	string	—	创建 agent 时使用的 model 名
<code>stream</code>	<code>Stream</code>	—	test-only transport override

18.5.2.2 Claude Code Hook Bridge (`packages/hooks/hooks-claude-code/src/index.ts:45`)

见 §3.6。

18.5.2.3 Codex Hook Bridge (`packages/hooks/hooks-codex/src/index.ts:44`)

见 §3.7。

18.6 错误处理

18.6.1 ACP 错误

- `RequestError.invalidParams(detail)`：保留 invalid-parameter detail (`acp/src/index.ts:60`)。
- `RequestError.internalError(detail)`：failed-turn detail 转为 wire internal error (`acp/src/index.ts:65`)。
- 普通 handler 错误 → generic wire internal error。

18.6.2 SDK Client 错误

错误类型	含义	触发
<code>TransportClosedError</code>	runtime 子进程消失	exit / stdio closed / spawn 失败 (携带 exit code + stderr tail <code>STDERR_TAIL_LIMIT=400</code>)
<code>RequestTimeoutError</code>	请求超时	<code>requestTimeoutMs</code> 触发
<code>SdkProtocolError</code>	协议响应不符	如 <code>session/prompt</code> response 无 <code>accepted: true</code>
<code>JsonRpcResponseError</code>	server 返回的 JSON-RPC error	从 wire 透传

18.6.3 Hook 错误

- timeout**: `DEFAULT_HOOK_TIMEOUT_MS = 600_000` (10 分钟, CC 默认), hook 超时被 collect 而非抛。

- **stderr 摘要**: `summarizeStderr` 截取到 `stderrSummaryMaxChars` 字符持久化到 `hook/result` session event。
- **detached failure**: detached run 失败被记录但不影响主流程返回值。
- **restrictive merge**: 一个 hook 返回 block, 整体 block (即使其他 hook 返回 allow) 。

18.7 扩展指南

18.7.1 添加新 ACP 方法

1. 在 `@agentclientprotocol/sdk` 升级后: dsh 跟随 SDK 版本 (当前 0.25.1) 。
2. 在 `dsh-acp/src/index.ts` `apply` 中: 通过 `conn.on(method, handler)` 注册新方法 handler。
3. **handler 签发** `RequestError` 而非 generic Error, 保留 `invalidParams` / `internalError` 语义。
4. **测试**: 用 ACP client SDK + `mock:llm` server 跑端到端。

18.7.2 添加新 SDK 方法

1. **扩展** `HarnessSdkRequestMap` / `HarnessSdkResultMap` (`packages/sdk/protocol/src/types.ts`) : 添加新 method 名 + params / result 类型。
2. 在 `HarnessSdkJsonRpcServer` 实现 handler 并加入 `handleRequest` dispatch (`server.ts:189`) 。
3. 在 `HarnessClient` 添加 typed 方法 (如 `prompt()` , `client.ts:282`) 。
4. **同步 Python SDK** (`python/sdk`) : 保持 design twin 一致。
5. **测试**: 用 `agent-loop-testkit` 跑端到端。

18.7.3 添加新 Hook Dialect Bridge

参考 Claude Code / Codex 双桥实现:

1. **新建** `packages/hooks/hooks-<dialect>/` , `peerDependencies` 含 `dsh-hook-protocol` 。
2. `config.ts` 实现 `parse<Dialect>Config(path)` : 解析该 dialect 的 hook 配置格式。
3. `index.ts` `apply(ctx, config)` :
 - `inject = ['shell']` (hook 命令通过 bash 执行) 。
 - 在每个支持的扩展点 (SessionStart / prompt pre/post / tool pre/post / Stop) 注册 listener。
 - listener 调用 `matchesMatcher` → `runHook` → `parseHookOutput` → `mergeHookOutputs` 。
 - 用 `appendHookInvoked` / `appendHookResult` 持久化到 session log。
4. **Own 该 dialect 的差异**:
 - payload 格式 (CamelCase / snake_case / trailing newline 等)
 - matcher 模式 (regex-only / 支持其他模式)
 - 环境变量 (导出给 hook 子进程的)
 - 命令 substitution (`${...}` 占位符)
 - 支持的扩展点集合
 - 决策 mapping (pre-tool approval / rewrite 是否支持)
5. **加入 bundle**: 在 `cordis.yml` 加 `- id: hooks-<dialect>` 。
6. **测试**: 参考 `hooks-claude-code` / `hooks-codex` 的 testkit 集成。

18.8 关键 SDK 依赖版本

依赖	版本	用途
@agentclientprotocol/sdk	0.25.1	ACP server (<code>acp/package.json</code> + 根 <code>devDependencies</code>)
@deepseek-ai/dsh-sdk-protocol	workspace: ^	共享 wire 协议
@deepseek-ai/dsh-sdk-client	workspace: ^	TS 客户端
@deepseek-ai/dsh-sdk-jsonrpc-server	workspace: ^	SDK server
@deepseek-ai/dsh-hook-protocol	workspace: ^	hook 共享库
@deepseek-ai/dsh-hooks-claude-code	workspace: ^	Claude Code bridge
@deepseek-ai/dsh-hooks-codex	workspace: ^	Codex bridge

18.9 关键发现

- ACP 与 SDK 是平行而非嵌套**: ACP 是 automation-only 标准 (外部 trusted 客户端), SDK 是 dsh 自有的完整客户端协议 (含 subagent 一等支持); 两者都 over stdio JSON-RPC 但 wire 不同。
- hook bridge 是兼容层而非重写**: 执行未修改的 Claude Code / Codex hook 命令, dsh 只 own 自己的 payload / env / mapping, shared execution 在 `dsh-hook-protocol`。这降低迁移成本。
- restrictive merge**: 多 hook 输出合并采取最严格决策, 确保安全 (一个 block 则整体 block)。
- SDK client 关闭 ladder**: EOF (关闭 stdin) → SIGTERM → SIGKILL, 加 `STREAM_SETTLE_MS = 100` grace 让 stdio 沉淀——这是 process 子进程关闭的稳健模式。
- HarnessClient 运行在 harness context 之外**: 直接 spawn runtime 而非通过 `dsh-subprocess` service (这是 seam 的 documented exception for SDK-managed transports)。
- updatedInput 在 Claude Code bridge 不 honor**: 设计取舍——log + warn 但不应用, 因为重新写 prompt 的语义在 dsh 上下文需要 typed native plugin 而非兼容 bridge。
- Python SDK 是 TS HarnessClient 的 design twin**: 两端驱动同一 runtime 协议, 必须同步演进 (`server.ts:9-10` 注释)。
- Codex 比 Claude Code 受限**: regex-only matchers / snake_case / no env / no substitution / no pre-tool approval / no subagent——这反映 Codex 协议本身更窄, 而非 dsh 实现差距。

19 框架与构建工具集成

本文档基于 AST 符号工具对源码符号进行验证分析生成。代码引用处的行号均来自符号工具验证。

19.1 集成概述

dsh 的「第三方框架与构建工具」分两类:

- Vendored 框架**: 把 Cordis 生态 (`cordis` / `cosmokit` / `schemastery`) 从上游 fork 后 vendored 进 `vendor/` 目录, 重新 scope 为 `@deepseek-ai/*` 并通过 `linkWorkspacePackages` 与本地源码联动。
- 第三方构建工具**: pnpm workspace、tsdown、oxlint、vitest、TypeScript、tsx、knip、publint、lefthook、jscpd、mermaid 等通过根 `package.json` `devDependencies` 集中管理。

这层集成的不变主题是「vendored 框架提供插件化骨架，第三方工具链保证 monorepo 健康度」。

19.1.1 设计哲学

- **Vendored 而非 dependency**: Cordis 三件套 fork 进仓库，scope 改为 `@deepseek-ai/*`，便于 dsh 在框架层做 in-tree 修复与演进。
- **linkWorkspacePackages: true**: 本地源码 override 远程包，确保 `@deepseek-ai/cordis` 始终解析到 workspace pinned 源码而非 npm。
- **allowBuilds 默认拒绝**: pnpm 10+ 严格模式，所有 install/build 脚本默认拒绝，显式 allow 后才运行。
- **patchedDependencies**: 必须修改的第三方包通过 patch 文件管理，避免 fork 维护成本。
- **类型契约先行**: 所有构建前必须 `npm run build:lib:host` 才能跑 typecheck / lint (contracts-ready pattern)。

19.2 支持的服务/产品

19.2.1 Vendored 框架

包名	上游	版本	角色
<code>@deepseek-ai/cordis</code>	<code>cordis</code> (Shigma)	4.0.1	Meta-Framework for Modern JavaScript Applications
<code>@deepseek-ai/cosmokit</code>	<code>cosmokit</code>	1.8.2	Common utilities
<code>@deepseek-ai/schemastery</code>	<code>schemastery</code>	3.18.1	Type driven schema validator
<code>@deepseek-ai/cordis-plugin-include</code>	cordis 生态	—	Cordis 插件
<code>@deepseek-ai/cordis-plugin-loader</code>	cordis 生态	—	Cordis 加载器
其他 vendor 子包	—	—	<code>group</code> / <code>hmr</code> / <code>include</code> / <code>loader</code> / <code>logger-console</code> / <code>timer</code>

19.2.2 第三方构建工具

工具	版本	用途
pnpm	11.7.0	workspace monorepo 包管理器
Node.js	^22.19.0 >=24.0.0	运行时
TypeScript	^6.0.3	类型系统（注意：peerDependency typescript: '>=5 <7' ）
tsx	^4.22.4	ESM TS 执行（CLI bin.ts 入口走 tsx/esm hook）
tsdown	^0.22.2	库打包（基于 esbuild）
esbuild	(transitive)	原生 binary, allowBuilds: true
oxlint	1.76.0	linter
oxlint-tsgolint	7.0.2001	oxlint TypeScript plugin
vitest	^4.1.8	测试框架
@vitest/coverage-v8	^4.1.8	覆盖率
knip	^6.16.1	未使用代码检测
publint	^0.3.21	发布前 package 配置 lint
lefthook	^2.1.9	git hooks, allowBuilds: true
jscpd	^5.0.12	代码重复检测
mermaid	11.16.0	文档图表渲染
jsdom	29.1.1	DOM 测试环境
@testing-library/react	^16.3.2	React 组件测试
fast-check	^4.8.0	property-based testing
execa	^10.0.0	子进程执行
vite-tsconfig-paths	^6.1.1	vite 路径别名
lightningcss	^1.32.0	CSS 优化
smol-toml	^1.7.1	TOML 解析
spdx-expression-parse	^5.0.0	SPDX 许可证解析
js-yaml	^4.2.0	YAML 解析
mdast-util-*	—	Markdown AST 工具链

19.3 集成方式

19.3.1 Vendored Cordis Workspace

`pnpm-workspace.yaml` 第 1-2 行:

```
packages:
  - vendor/*
  - packages/**
```

`vendor/` 下每个子包都是独立 workspace 成员。`linkWorkspacePackages: true` (line 25) 让本地源码 override 远程版本。

`overrides` (line 27-29) 强制所有传递依赖解析到 vendored 版本:

```
overrides:
  '@deepseek-ai/cosmokit': 'link:vendor/cosmokit'
  '@deepseek-ai/schemastery': 'link:vendor/schemastery'
```

`peerDependencyRules.allowedVersions` (line 31-33) 放宽 TypeScript peer:

```
peerDependencyRules:
  allowedVersions:
    typescript: '>=5 <7'
```

19.3.1.1 @deepseek-ai/cordis 4.0.1

`vendor/cordis/package.json` 描述: 「Meta-Framework for Modern JavaScript Applications」。源码在 `vendor/cordis/src/` (含 `context.ts` / `events.ts` / `fiber.ts` / `logger.ts` / `reflect.ts` / `registry.ts` / `service.ts` / `utils.ts`)。

关键能力:

- **Context + Service**: 所有 dsh 能力继承 `Service` (如 `SandboxProvider extends Service`、`TokenMeter extends Service`、`E2BRuntime extends Service`)。
- **declaration merging**: dsh 通过 `declare module '@deepseek-ai/cordis'` 扩展 `Context` 接口 (如 `ctx.llm` / `ctx.sandbox` / `ctx.tokenMeter` / `ctx.e2b`)。
- **waterfall events**: `Events` 接口扩展定义 waterfall (如 `'llm/stream'` 在 `packages/llm/llm/src/index.ts:51-66`)。
- **ctx.effect() / ctx.on()**: 可逆 effect, plugin 卸载时自动回滚。

`peerDependencies` 含可选的 `@deepseek-ai/cordis-plugin-include` 和 `@deepseek-ai/cordis-plugin-loader` (独立 vendor 子包)。

依赖 `@standard-schema/spec: ^1.1.0` + `@deepseek-ai/cosmokit: workspace:^`。

19.3.1.2 @deepseek-ai/cosmokit 1.8.2

`vendor/cosmokit/package.json`: common utilities, 零依赖 (除标准库), 被 cordis 和 schemastery 依赖。

19.3.1.3 @deepseek-ai/schemastery 3.18.1

`vendor/schemastery/package.json`: type-driven schema validator。所有 dsh plugin 的 `Config` schema 用 `z = @deepseek-ai/schemastery` 定义。

关键用法 (dsh 内):

- `z.object({...})`: 定义 plugin Config。
- `z.string().role('credential-ref')`: 标记字段为 credential 引用 (不让 secret 进入配置)。
- `z.string().role('secret')`: 标记字段为 secret 字面量 (web-search-deepseek 支持)。
- `z.number().default(...)` / `z.union([...])` / `z.array(...)`: 标准 schema 组合子。

- `static Config: z<Config>` : 每个 Service plugin 把 schema 暴露为 static 属性供 loader 读取。

依赖 `@standard-schema/spec: ^1.1.0` + `@deepseek-ai/cosmoskit: workspace: ^`。

19.3.2 pnpm Workspace + allowBuilds

`pnpm-workspace.yaml` 的 workspace 成员:

```
packages:
  - vendor/*           # vendored 框架
  - packages/*/*       # dsh 工作区 (49 个 group)
  - native/landlock-run # 原生 addon
  - native/landlock-run/packages/*
  - apps/*             # 产品装配 (CLI、Web)
  - website            # 文档站点
  - examples           # 示例 (仅 dependency resolution)
  - python/sdk-runtime  # 单 exe 部署根
```

`allowBuilds` (line 40-55, pnpm 10+ strict 模式) :

```
allowBuilds:
  esbuild: true           # 原生 binary, 需 install script
  lefthook: true          # git hooks, 需 install script
  node-pty: true          # 跨平台 PTY 后端 (含 ConPTY)
  '@google/genai': false  # pi-ai 拉入但脚本 no-op, 拒绝
  protobufjs: false       # 同上
  node-addon-require-builtin: false # 同上
  koffi: true             # Win32 FFI, MoveFileExW 用于 JSONL durability
  '@deepseek-ai/dsh-subprocess-local@file:packages/subprocess/subprocess-local': true
```

设计原则:

- **默认拒绝**: 未列出的包 install/build 脚本一律拒绝 (hard install error) 。
- **理由**: esbuild (native binary)、lefthook (git hooks)、node-pty (ConPTY)、koffi (FFI) 确实需要 install script。
- **拒绝 no-op**: `@google/genai` / `protobufjs` / `node-addon-require-builtin*` 的 lifecycle script 是 no-op, 拒绝可加速 install。

`minimumReleaseAgeExclude` (line 57-69) : fresh pi-ai release (0.82.1) 和 node-addon-* 0.1.4 排除最小发布年龄限制, 因为这些版本携带 model catalog 更新或 native binary 是发布目的本身。

`patchedDependencies` (line 71-72) :

```
patchedDependencies:
  node-pty@1.1.0: patches/node-pty@1.1.0.patch
```

19.3.3 构建工具链

19.3.3.1 tsdown (库打包)

根 `package.json` 的 build 脚本 (line 21-23) :

```
"build:lib": "npm run build:lib:host && npm run build:lib:client",
"build:lib:host": "tsc -b tsconfig.host.json && tsdown --env.DSH_BUILD_FACE host",
"build:lib:client": "tsc -b tsconfig.client.json && tsdown --env.DSH_BUILD_FACE client",
```

两个 face:

- **host**: 服务端代码, 给 dsh runtime 用。
- **client**: 浏览器侧代码, 给 Web UI 用。

tsdown 基于 esbuild, 单文件 bundle 输出到 `lib/index.js` (每个子包的 `exports['.'].default`)。

19.3.3.2 TypeScript 6.0.3 + tsx 4.22.4

`typescript: ^6.0.3` 是较新版本, `peerDependency typescript: '>=5 <7'` 兼容。

`tsx: ^4.22.4` 提供 ESM TS 执行:

- CLI 入口 `apps/cli/src/bin.ts` 通过 `node --import tsx/esm` 启动 (package.json `dsh` 脚本, line 136)。
- 各包的 `src/*` 通过 `./src/*` export 暴露, 便于开发期直接 import 源码。
- 例如 `dsh-sandbox-windows-acl/runner` 在开发期通过 `[process.execPath, '--import', 'tsx/esm', sourceEntry]` 启动 (`sandbox-local/src/index.ts:563`)。

19.3.3.3 oxlint 1.76.0

`oxlint-tsgolint: 7.0.2001` 是 TypeScript plugin。

Lint 脚本 (package.json line 29-32) :

```
"lint": "npm run build:lib:host && npm run lint:contracts-ready",
"lint:contracts-ready": "tsx scripts/run-oxlint.ts .",
"lint:fix": "npm run build:lib:host && npm run lint:fix:contracts-ready",
"lint:fix:contracts-ready": "tsx scripts/run-oxlint.ts --config .oxlintrc.staged.json
  packages/typert/generator/tests/fixtures/type-model --fix && tsx scripts/run-oxlint.ts . --fix",
```

contracts-ready 模式: 先 `build:lib:host` 让 type contracts 可见, 再 lint。这是 dsh 反复出现的模式。

`.oxlintrc.staged.json` 是 lefthook staged 用的精简配置。

19.3.3.4 vitest 4.1.8

`vitest: ^4.1.8` + `@vitest/coverage-v8: ^4.1.8`。多种 test 配置:

```
"test": "vitest run",
"test:coverage": "vitest run --coverage",
"test:e2e": "vitest run --config vitest.e2e.config.ts",
"test:snapshot": "vitest run --config vitest.snapshot.config.ts",
"test:snapshot:record": "DSH_SNAPSHOT=record vitest run --config vitest.snapshot.config.ts --update",
"test:snapshot:refresh": "DSH_SNAPSHOT=refresh vitest run --config vitest.snapshot.config.ts",
"test:web": "npm run build && npm run test:web:build",
"test:web:build": "vitest run --config vitest.web.config.ts",
"test:web:perf": "npm run build && npm run test:web:perf:build",
"test:web:stress": "npm run build && vitest run --config vitest.web-stress.config.ts",
"test:gui": "vitest run packages/client packages/host",
```

Snapshot 测试支持 `record` / `refresh` / `replay` 三种模式 (通过 `DSH_SNAPSHOT` env var 控制)。

19.3.3.5 knip 6.16.1

`knip: ^6.16.1` 检测未使用的 exports / dependencies / files。

```
"knip": "knip --treat-config-hints-as-errors",
```

`--treat-config-hints-as-errors` 让 config 提示也视为错误，确保严格。

19.3.3.6 publint 0.3.21

```
"publint": "tsx scripts/publint-all.ts",
```

`publint-all.ts` 批量检查所有子包的 `package.json` 配置 (exports / files / main / types 等)，确保发布前正确。

19.3.3.7 lefthook 2.1.9

`lefthook: ^2.1.9` 管理 git hooks。 `postinstall: node scripts/install-lefthook.mjs` (package.json line 142) 。

`allowBuilds: lefthook: true` 让其 install script 运行。

19.3.3.8 jscpd 5.0.12

`jscpd: ^5.0.12` 检测代码重复：

```
"duplication": "jscpd --config .jscpd.json packages scripts",
```

19.3.3.9 mermaid 11.16.0

`mermaid: 11.16.0` 用于文档图表渲染 (如 `analysis/02-architecture/system-component-architecture.puml`，虽然 puml 是 PlantUML 但项目也含 mermaid 图)：

```
"verify-mermaid": "tsx scripts/verify-mermaid.ts",
```

19.4 代码实现

19.4.1 关键脚本

脚本	文件	用途
<code>build:lib:host</code>	<code>tsconfig.host.json</code> + <code>tsdown</code>	构建 host face
<code>build:lib:client</code>	<code>tsconfig.client.json</code> + <code>tsdown</code>	构建 client face
<code>lint:contracts-ready</code>	<code>scripts/run-oxlint.ts</code>	跑 oxlint
<code>publint</code>	<code>scripts/publint-all.ts</code>	批量 publint
<code>knip</code>	(直接 knip)	未使用代码检测
<code>verify-md-links</code>	<code>scripts/verify-md-links.ts</code>	文档链接校验
<code>verify-md-wrap</code>	<code>scripts/verify-md-wrap.ts</code>	markdown 换行校验
<code>verify-mermaid</code>	<code>scripts/verify-mermaid.ts</code>	mermaid 图表校验
<code>verify-package-invariants</code>	<code>scripts/verify-package-invariants.ts</code>	包不变量校验
<code>verify-built-package-invariants</code>	<code>scripts/verify-built-package-invariants.mjs</code>	构建后包不变量校验
<code>verify-vendored-links</code>	<code>scripts/verify-vendored-links.ts</code>	vendored 链接校验
<code>verify-cordis-config</code>	<code>scripts/verify-cordis-config.ts</code>	cordis 配置校验
<code>verify-runtime-closure</code>	<code>scripts/verify-runtime-closure.ts</code>	runtime 闭包校验
<code>verify-node-next-types</code>	<code>scripts/verify-node-next-types.ts</code>	Node next 类型校验
<code>verify-dsh-package-licenses</code>	<code>scripts/verify-dsh-package-licenses.ts</code>	许可证校验
<code>verify-third-party-notices</code>	<code>scripts/gen-third-party-notices.ts --check</code>	第三方声明校验
<code>gen-cordis-catalog</code>	<code>scripts/gen-cordis-catalog.ts</code>	生成 cordis 目录
<code>gen-tool-catalog</code>	<code>scripts/gen-tool-catalog.ts</code>	生成工具目录
<code>gen-config-catalog</code>	<code>scripts/gen-config-catalog.ts</code>	生成配置目录
<code>gen-module-graph</code>	<code>scripts/gen-module-graph.ts</code>	生成模块图
<code>gen-doc-graphs</code>	<code>scripts/gen-doc-graphs.ts</code>	生成文档图
<code>rescope-vendor</code>	<code>scripts/rescope-vendor.ts</code>	重 scope vendored 包
<code>hygiene</code>	(组合脚本)	多重卫生检查

19.4.2 Contracts-Ready 模式

多个 gate 脚本都遵循 `build:lib:host && gate:contracts-ready` 模式：

```
"typecheck": "npm run build:lib:host && npm run typecheck:contracts-ready",
"lint": "npm run build:lib:host && npm run lint:contracts-ready",
"doc-typecheck": "npm run build:lib:host && npm run doc-typecheck:contracts-ready",
```

原因: dsh 的 type contracts (如 `SessionEventMap`、`Context` declaration merging) 跨包生成, 必须先构建 host face 才能让下游包看到完整类型。

19.4.3 Gate 系统

`scripts/run-gates.ts` (package.json line 49-62) 封装多种 gate:

```
"check:all": "tsx scripts/run-gates.ts check-all",
"check:ci": "tsx scripts/run-gates.ts ci-primary",
"check:ci:static": "tsx scripts/run-gates.ts ci-static",
"check:ci:coverage": "tsx scripts/run-gates.ts ci-coverage",
"check:ci:snapshot": "tsx scripts/run-gates.ts ci-snapshot",
"check:ci:artifacts": "tsx scripts/run-gates.ts ci-artifacts",
"check:ci:consumers": "tsx scripts/run-gates.ts ci-consumers",
"check:ci:linux-primary": "tsx scripts/run-gates.ts ci-linux-primary",
"check:ci:windows-blocking": "tsx scripts/run-gates.ts ci-windows-blocking",
"check:ci:windows-complete": "tsx scripts/run-gates.ts ci-windows-complete",
"check:ci:windows-observational": "tsx scripts/run-gates.ts ci-windows-observational",
"check:node-compat": "tsx scripts/run-gates.ts node-compat",
"check:windows-wine": "bash scripts/wine-windows-gates.sh",
```

平台分层:

- **static**: lint / typecheck / docs。
- **linux-primary**: Linux 主流 CI。
- **windows-blocking**: Windows 必跑 gate。
- **windows-observational**: Windows 观察性 gate (不 block)。
- **wine**: 通过 Wine 跑 Windows gate (非 Windows 主机)。

19.4.4 Vendored Scope 重写

`scripts/rescope-vendor.ts` (package.json line 101-102) :

```
"rescope-vendor": "tsx scripts/rescope-vendor.ts",
"rescope-vendor:check": "tsx scripts/rescope-vendor.ts --check",
```

把 vendored 包的 `package.json` 中 `@cordiverse/*` 等 upstream scope 改写为 `@deepseek-ai/*`。 `verify-vendored-links.ts` 校验所有内部 import 都用新 scope。

19.5 配置与环境变量

19.5.1 环境变量

环境变量	作用	默认
DSH_BUILD_FACE	tsdown 构建目标 face	host 或 client (由 build 脚本设置)
DSH_SNAPSHOT	snapshot 测试模式	record / refresh / replay
NODE_OPTIONS	Node.js 启动选项	(隐式含 --import tsx/esm for dev)

19.5.2 关键配置文件

文件	用途
pnpm-workspace.yaml	workspace 配置 (packages / overrides / allowBuilds / patchedDependencies)
tsconfig.host.json	host face TS 配置
tsconfig.client.json	client face TS 配置
tsconfig.base.json	共享 TS 配置
.oxlintrc.json / .oxlintrc.staged.json	oxlint 配置
vitest.config.ts / vitest.e2e.config.ts / vitest.snapshot.config.ts / vitest.web.config.ts / vitest.web.perf.config.ts / vitest.web-stress.config.ts	vitest 配置
.jscpd.json	jscpd 配置
lefthook.yml	lefthook 配置
knip.json / knip.base.json	knip 配置

19.6 错误处理

19.6.1 构建错误

错误	原因	处理
tsc -b 失败	类型错误	必须修复才能继续 tsdown
tsdown 失败	bundle 错误	检查 exports 配置和 lib/index.js
peerDependencyRules 不满足	TypeScript 版本不匹配	检查 allowedVersions: typescript: '>=5 <7'
allowBuilds 拒绝	未列出的 install script	加入 allowBuilds 或修复

19.6.2 CI 错误

gate 系统分层失败：

- **static gate 失败**：lint / typecheck / knip / publint / 链接校验。
- **test gate 失败**：单元测试 / snapshot 不匹配（用 `DSH_SNAPSHOT=record` 更新） / e2e。
- **platform gate 失败**：Linux 主 CI 失败 block，Windows observational 不 block。

19.6.3 Vendored 链接错误

- `verify-vendored-links.ts`：检查 vendored 包内 import 都用 `@deepseek-ai/*` scope。
- `rescope-vendor:check`：CI 跑 `--check` 模式，确保 `rescope-vendor` 已运行。

19.6.4 包不变量错误

`verify-package-invariants.ts` + `verify-built-package-invariants.mjs`：

- 检查每个包的 `exports` / `files` / `main` / `types` 一致性。
- 检查构建产物存在且匹配配置。
- 防止发布配置错误。

19.7 扩展指南

19.7.1 升级 Vendored Cordis

1. 拉取上游变更：从 `cordiverse/cordis` 同步到 `vendor/cordis/src/`。
2. 跑 `rescope-vendor`：重写所有 scope 为 `@deepseek-ai/*`。
3. 跑 `verify-vendored-links:check`：确保 import 正确。
4. 跑 `gen-cordis-catalog --check`：目录生成一致。
5. 跑 `verify-cordis-config`：配置校验。
6. 跑 `check:all`：全量 gate。
7. 更新 `vendor/cordis/package.json` 版本。

19.7.2 添加新子包到 workspace

1. 创建 `packages/<group>/<pkg>/`，写 `package.json`：
 - `name`：`@deepseek-ai/dsh-<pkg>`
 - `version`：`0.1.0-rc.5`（与 workspace 同步）
 - `peerDependencies`：含 `@deepseek-ai/cordis: workspace:^`
 - `dependencies`：含 `@deepseek-ai/schemastery: workspace:^`（如需 schema）
 - `exports['.']` / `exports['./invariant']` / `exports['./src/*']` / `exports['./package.json']`
 - `files`：列出 `lib/index.js` / `lib/invariant.js` / `lib/types/**/*.d.ts`
2. 加入 `cordis.yml`：在对应 bundle 的 plugin 列表加 `- id: <pkg>`。
3. 跑 `verify-package-invariants`：检查配置一致。
4. 跑 `publint`：发布前 lint。

19.7.3 添加新第三方构建依赖

1. 加入根 `package.json` `devDependencies`（不要加到子包）。
2. 若包有 install/build script：
 - 在 `pnpm-workspace.yaml` 的 `allowBuilds` 加 `true`（确实需要）或 `false`（no-op 拒绝）。

- 若是 native binary 类 (如 esbuild) , 考虑 `minimumReleaseAgeExclude` 。
3. 若需 patch:
- 创建 `patches/<pkg>@<version>.patch` (用 `pnpm patch`) 。
 - 在 `pnpm-workspace.yaml` 的 `patchedDependencies` 加 entry。
4. 集成到 gate 脚本: 在 `scripts/run-gates.ts` 加对应 gate。

19.8 关键 SDK 依赖版本

19.8.1 Vendored 框架

包	版本	上游
<code>@deepseek-ai/cordis</code>	4.0.1	cordiverse/cordis
<code>@deepseek-ai/cosmokit</code>	1.8.2	cordiverse/cosmokit
<code>@deepseek-ai/schemastery</code>	3.18.1	Shigma/schemastery
<code>@standard-schema/spec</code>	^1.1.0	standard-schema (被 cordis 和 schemastery 依赖)

19.8.2 构建工具链版本矩阵

工具	版本	必要性
pnpm	11.7.0	包管理器 (<code>packageManager</code> 字段)
Node.js	<code>^22.19.0 >=24.0.0</code>	运行时 (<code>engines.node</code>)
TypeScript	<code>^6.0.3</code>	类型系统
tsx	<code>^4.22.4</code>	ESM TS 执行
tsdown	<code>^0.22.2</code>	库打包
oxlint	1.76.0	linter
oxlint-tsgolint	7.0.2001	oxlint TS plugin
vitest	<code>^4.1.8</code>	测试
@vitest/coverage-v8	<code>^4.1.8</code>	覆盖率
knip	<code>^6.16.1</code>	未使用代码检测
publint	<code>^0.3.21</code>	发布 lint
lefthook	<code>^2.1.9</code>	git hooks
jscpd	<code>^5.0.12</code>	代码重复
mermaid	11.16.0	文档图表
jsdom	29.1.1	DOM 测试
@testing-library/react	<code>^16.3.2</code>	React 测试
fast-check	<code>^4.8.0</code>	property-based testing
execa	<code>^10.0.0</code>	子进程执行
vite-tsconfig-paths	<code>^6.1.1</code>	vite 路径别名
lightningcss	<code>^1.32.0</code>	CSS 优化
smol-toml	<code>^1.7.1</code>	TOML
spdx-expression-parse	<code>^5.0.0</code>	SPDX
js-yaml	<code>^4.2.0</code>	YAML
mdast-util-from-markdown	<code>^2.0.3</code>	Markdown AST
mdast-util-gfm	<code>^3.1.0</code>	GFM 扩展
micromark-extension-gfm	<code>^3.0.0</code>	GFM parser
@types/node	<code>^22.20.0</code>	Node 类型
@types/js-yaml	<code>^4.0.9</code>	js-yaml 类型

工具	版本	必要性
@types/jsdom	^28.0.3	jsdom 类型
@types/mdast	^4.0.4	mdast 类型
@types/spdx-expression-parse	^4.0.0	SPDX 类型
@stylistic/eslint-plugin	^5.10.0	ESLint 风格
eslint-plugin-sonarjs	^4.1.0	SonarJS 规则
@yarnpkg/cli-dist	4.17.1	Yarn CLI (用途未明)
istanbul-lib-report	^3.0.1	覆盖率报告

19.9 关键发现

- Vendored Cordis 是 in-tree fork:** scope 从 `@cordiverse/*` 改为 `@deepseek-ai/*`, `linkWorkspacePackages: true` + `overrides` 确保本地源码 override 远程, 便于 dsh 在框架层快速修复。
- allowBuilds 是安全防线:** pnpm 10+ 默认拒绝所有 install/build script, dsh 显式 allow 仅需要的 (esbuild / lefthook / node-pty / koffi), 拒绝 no-op 的 (`@google/genai` / `protobufjs`), 减少供应链攻击面。
- patchedDependencies 用于 node-pty:** 1.1.0 版本通过 `patches/node-pty@1.1.0.patch` 自定义修复, 无需 fork。
- contracts-ready 模式贯穿:** `build:lib:host` 必须在 typecheck / lint / doc-typecheck 前跑, 因为 dsh 的 type contracts 跨包生成 (declaration merging / SessionEventManager 等)。
- 双 face 构建:** `host` (服务端) 和 `client` (浏览器) 分开构建, 让 Web UI 不携带 Node-only 代码。
- 平台分层 CI:** Linux-primary + Windows-blocking + Windows-observational + Wine (非 Windows 主机跑 Windows gate), 保证跨平台质量。
- 大量 verify 脚本:** `verify-md-links` / `verify-mermaid` / `verify-package-invariants` / `verify-vendored-links` / `verify-cordis-config` / `verify-runtime-closure` 等, 构成强约束的卫生检查网, 但需要维护成本。
- TypeScript 6 + peerDependency 5-7:** 用 TypeScript 6 但 peer 兼容 5+, 下游消费者可灵活选版本。
- gen-catalog 系列:** `gen-cordis-catalog` / `gen-tool-catalog` / `gen-config-catalog` / `gen-persistence-catalog` 等从源码生成目录, 确保文档与代码同步。
- hygiene 是组合脚本:** `rescope-vendor:check && knip && publint && constraints && verify-dsh-package-licenses && verify-package-invariants && verify-built-package-invariants && verify-cordis-config && verify-node-next-types && verify-runtime-closure && verify-vendored-links` ——一次跑完所有卫生检查。

20 相关资源 (Related Resources)

本章汇总理解、运行、扩展与对比 DeepSeek Harness 的一手与延伸资源。编写时间: 2026 年 8 月。Harness 处于 **开发者预览期 (Developer Preview)**, 链接与版本可能随时变动, 访问请以官网为准。

20.1 官方一手资源

资源	链接 / 说明
GitHub 仓库 (MIT, monorepo)	https://github.com/deepseek-ai/deepseek-harness
架构文档 (中英双语)	docs/architecture.zh.md · docs/architecture.md
开发指南	docs/development.md (改动 packages/ 前必读)
Web UI 指南	docs/user/guide/index.md
文档站点 (VitePress)	仓库 website/ 目录构建, 本地 <code>pnpm run website:build</code>
一键运行 (npm)	<code>npx @deepseek-ai/dsh web</code> (默认 http://127.0.0.1:3080)
底层框架 Cordis	https://github.com/cordiverse/cordis
Cordis 设计论文	<i>A Programming Paradigm for Spatiotemporal Composability</i>
许可证	MIT; 第三方依赖见 THIRD_PARTY_NOTICES.md

关键事实 (来自官方 README) : `dsh` 由 DeepSeek AI 开发, 基于 Cordis 的「一切皆插件」架构; 开发者预览阶段 **会出现破坏兼容性的变更**; CLI 与本地 Web UI 双形态; 提供中英双语架构文档。发布时间为 2026-08-13 (MIT, 发布当天开源) 。

20.2 社区与支持

- **GitHub Discussions**: <https://github.com/deepseek-ai/deepseek-harness/discussions> (反馈 / bug 报告首选)
- **插件可发现性**: 为你的插件仓库添加 `dsh-plugin` 话题
- **官方企微信群 / 微信公众号**: 见官方仓库 README 底部二维码 (小助手、入群问卷、公众号)

20.3 橙皮书系列 & 同类框架横向对比

本解析是「读源码」视角, 下面这些资料是互补的「上手实测 / 同类框架」视角:

- **花叔《DeepSeek Harness: 从开机到拆开》** (橙皮书, 非代码视角的上手实测) <https://github.com/alchaincyf/deepseek-harness-orange-book>
- **花叔《OpenAI Codex 从入门到精通》** (橙皮书, 中英双版) <https://github.com/alchaincyf/codex-orange-book>
- **同类 AI Agent 框架** (用于架构/能力对比)
 - Claude Code (Anthropic) : <https://docs.claude.com/en/docs/claude-code/overview>
 - OpenAI Codex: <https://developers.openai.com/codex/>
 - Trae Agent (字节跳动) : <https://github.com/bytedance/trae-agent>
 - OpenClaw (「养一只你自己的 AI」, 见橙皮书系列 / [huasheng.ai](#))
- **本系列其他源码拆解**: [awesome-ai-projects-analysis](#) (Dify / Langfuse / RAGFlow / dify-1.14.2 等)

20.4 本分析的方法与工具链

本章的解析并非「读文档转译」, 而是「贴着代码跑」:

- **AST 符号验证**: 正文中大量「`X 类 L424-757`」「`SessionEventMap 接口 L235-332`」这类行号引用, 由 AST 符号工具对源码符号验证生成, 不是凭记忆手测 (见各章首注) 。
- **图表**: 架构/时序/ER 图以 PlantUML 源 (`.puml`) 编写, 再渲染为 PNG 随文档内联。

- **文档流水线**: Markdown → Pandoc (`--embed-resources` 图片内联) → Google Chrome 无头 `--print-to-pdf` 生成单文件离线 PDF/HTML。
- **数据源**: 本地克隆的 `deepseek-ai/deepseek-harness` 源码 (commit 级快照)。复现请以你本机的 `git log` 与运行输出为准。

20.5 延伸阅读 / 第三方解读

- 社区 wiki / 博客 (如 `deepseek-harness-book` 等第三方梳理)
- `Awesome-AITools` 等工具清单中的 Harness 条目
- 官方 `.agents/` (Agent 工作流 + Agent Notes)、`docs/postmortem/` (事故复盘) 是理解「它踩过哪些坑」的一手材料

20.6 版本与时效说明

- 当前为 **开发者预览**: 版本号形如 `0.1.0-rc.x`, **不承诺兼容性**, 会话格式版本固定为 `0`、SQLite `SCHEMA_VERSION` 单调但无兼容承诺。
- 本文档基于某一 commit 快照编写; 仓库迭代极快 (官方描述「快速迭代」), 复现任何命令或数字时, 请以你本机输出为准, 并在引用处标注取数时间。
- 如要跟进最新, 直接 `git clone` 后 `pnpm dsh web` 跑起来, 比看任何二手资料都准。

下一站: 回到 `README` 查看完整目录与定位; 或直接打开合并版 PDF / HTML (仓库根 `combined/`) 。